

Tumhare hisab se most important react ke note ke express ke aur JavaScript ke कौन-कौन se question hai interview ke Jo ek pressure se puche hi jaate Hain hamesha se har company mein

Haan. Agar **MERN / Full-Stack Developer interview** ke liye time kam hai, to React + JavaScript + Express ke saare questions karne ki zarurat nahi hai. Kuch questions **almost har technical interview mein repeatedly** aate hain.

Main priority ke hisaab se bata raha hoon:

1. JavaScript — sabse important

Must-do

1. **var, let, const mein difference**
2. **Hoisting kya hai?**
3. **Scope — global, function, block scope**
4. **Closure kya hai? ★**
5. **Callback function kya hai?**
6. **Promise kya hai?**
7. **async/await kya hai?**
8. **Promise.all(), Promise.allSettled(), Promise.race()**
9. **Synchronous vs Asynchronous JavaScript**
10. **Event Loop kya hai? ★**
11. **Call Stack, Callback Queue, Microtask Queue**
12. **== vs ===**
13. **null vs undefined**
14. **Primitive vs Reference data types**
15. **Shallow copy vs Deep copy**
16. **Spread vs Rest operator**
17. **map(), filter(), reduce()**
18. **forEach() vs map()**
19. **find() vs filter()**
20. **Destructuring**
21. **this keyword**
22. **Arrow function vs normal function**
23. **Higher Order Function**

- 24. **Prototype / Prototype Chain**
- 25. **Object.create()**
- 26. **ES6 features**
- 27. **Debouncing vs Throttling**
- 28. **LocalStorage vs SessionStorage vs Cookies**
- 29. **Error handling — try/catch**
- 30. **Fetch API / Axios**

★★★★★ Coding bhi pakka karo

- Reverse string
- Palindrome
- Anagram
- Character frequency
- Remove duplicates
- Find duplicate
- Largest/smallest element
- Second largest
- Two Sum
- Array sorting
- Flatten array
- Frequency of elements
- Missing number
- FizzBuzz

JS mein sabse zyada focus:

Closure + Promise + Async/Await + Event Loop + Array methods + this + Hoisting

2. React — interview ke liye

★★★★★ Must-do

1. **React kya hai?**
2. **React use karne ke advantages**
3. **Component kya hota hai?**
4. **Functional vs Class Component**
5. **Props kya hain?**

6. State kya hai?
7. Props vs State ★
8. useState()
9. useEffect() ★★
10. useEffect dependency array
11. useEffect cleanup function
12. useRef()
13. useMemo()
14. useCallback()
15. useMemo vs useCallback ★
16. useContext()
17. Context API
18. Prop Drilling
19. Controlled vs Uncontrolled Components
20. Forms in React
21. Conditional Rendering
22. Lists and keys
23. Why is key important in React?
24. React Router
25. useNavigate()
26. useParams()
27. useLocation()
28. SPA kya hai?
29. Virtual DOM kya hai? ★
30. Real DOM vs Virtual DOM
31. Reconciliation
32. React component re-render kab hota hai? ★
33. State update asynchronous kyu lagta hai?
34. Lifting State Up
35. Custom Hooks
36. React.memo()
37. Lazy loading
38. Code splitting
39. Error Boundary
40. Performance optimization in React

★★★★★ Practical questions

Ye bhi interviewer coding/practical round mein de sakta hai:

- Counter component
- Todo app
- Search/filter list
- API se data fetch karna
- Loading/error state
- Login form
- Registration form
- Parent → child data
- Child → parent data
- Protected route
- Pagination
- Debounced search

React mein sabse zyada focus:

Props/State → useState → useEffect → useContext → useRef → React Router → API calling
→ Re-render → Virtual DOM → Performance

3. Express.js — comparatively chhota but important

Express mein **theory se zyada practical understanding** dekhi jaati hai.

★★★★★ Must-do

1. Express.js kya hai?
2. Node.js vs Express.js
3. Server kaise create karte ho?
4. Routing kya hai?
5. GET, POST, PUT, PATCH, DELETE
6. Middleware kya hai? ★★★
7. Application-level middleware
8. Router-level middleware
9. Error-handling middleware

10. Built-in middleware
11. Custom middleware
12. req.params vs req.query vs req.body ★
13. Request aur Response kya hain?
14. Status codes
15. res.send() vs res.json()
16. app.use() kya karta hai?
17. express.json() kya hai?
18. CORS kya hai?
19. REST API kya hai?
20. RESTful API kaise banate ho?
21. Controller kya hota hai?
22. Routes aur Controllers ko separate kyu karte hain?
23. MVC architecture
24. Authentication vs Authorization
25. JWT authentication
26. Protected routes
27. Password hashing
28. bcrypt
29. Environment variables / .env
30. API error handling

Express ke practical questions

Interviewer bol sakta hai:

“Create an API for user registration.”

Tumhe flow pata hona chahiye:

Route → Controller → Validation → Database → Response

Ya:

“How will you protect an API?”

Answer ka flow:

JWT → middleware → token verification → req.user → protected controller

Agar bahut hi kam time hai

Main tumhare liye priority **aise** rakhunga:

JavaScript

Top 15

1. var/let/const
2. Hoisting
3. Scope
4. Closure
5. Callback
6. Promise
7. async/await
8. Event Loop
9. == vs ===
10. map/filter/reduce
11. this
12. Arrow vs normal function
13. Spread/Rest
14. Shallow/Deep copy
15. Debounce/Throttle

React

Top 15

1. React basics
2. Components
3. Props
4. State
5. Props vs State
6. useState
7. useEffect
8. Dependency array
9. Cleanup
10. useRef
11. Context API

12. Prop Drilling
13. React Router
14. Virtual DOM
15. Re-rendering

Express

Top 15

1. Express kya hai
 2. Node vs Express
 3. Routing
 4. HTTP methods
 5. Middleware
 6. Types of middleware
 7. app.use()
 8. req.params
 9. req.query
 10. req.body
 11. Status codes
 12. REST API
 13. CORS
 14. JWT authentication
 15. Error handling
-

100 Aur ek important baat

Interview mein sirf definition nahi puchte.

For example, useEffect kya hai? ke baad interviewer immediately pooch sakta hai:

“Dependency array empty ho to kya hoga?”

phir:

“Cleanup function kyu use karte ho?”

phir:

“Component unmount hone par kya hota hai?”

Isliye tumhe **question ke saath 2–3 follow-up questions** bhi prepare karne chahiye.

Agar tum MERN interview ki preparation kar rahi ho, to main personally **JavaScript → React → Node/Express → MongoDB → Projects** isi order mein karunga. **JavaScript ko sabse strong** rakhna, kyunki React ke bahut saare interview questions indirectly JavaScript hi test karte hain.

To fir aise kaise preparation Karun yaar

Haan yaar 🤔 **problem yehi hai ki agar hum “1000 interview questions” dekhne lagenge to preparation kabhi khatam nahi hogi.** Tumhe questions yaad nahi karne, **patterns prepare karne hain.**

Tumhare case mein main preparation ko **3 layers** mein karunga:

🔥 Layer 1 — Core Questions

Har technology ke sirf **most repeated 30–40 questions.**

JavaScript → पहले

Pehle ye 5 blocks complete karo:

Block 1: Basics

- var/let/const
- scope
- hoisting
- data types
- == vs ===
- null vs undefined

Block 2: Functions

- function
- arrow function
- callback
- higher-order function
- closure
- this

Block 3: Arrays & Objects

- map
- filter
- reduce
- forEach
- find
- spread/rest
- destructuring
- shallow/deep copy

Block 4: Async JS

- synchronous/asynchronous
- callback
- Promise
- async/await
- Promise.all
- event loop
- call stack
- microtask/callback queue

Block 5: Browser

- DOM
- event bubbling
- event delegation
- localStorage/sessionStorage/cookies
- debounce/throttle

👉 **Bas ye 5 blocks strong = JavaScript interview ka major portion covered.**

Layer 2 — React

JS ke baad React ko is order mein:

Block 1

- React kya hai

- component
- JSX
- props
- state
- props vs state

Block 2

- useState
- useEffect
- dependency array
- cleanup
- re-render

Block 3

- useRef
- useContext
- Context API
- prop drilling
- lifting state

Block 4

- React Router
- useNavigate
- useParams
- protected routes

Block 5

- Virtual DOM
- reconciliation
- React.memo
- useMemo
- useCallback
- performance

Block 6 — Practical

- API calling
- loading

- error handling
 - forms
 - authentication
 - CRUD
-

Layer 3 — Express

Express ko unnecessarily huge mat banana.

Block 1 — Basics

- Express
- server
- routing
- HTTP methods
- request/response

Block 2 — Middleware

- middleware kya hai
- `app.use()`
- custom middleware
- router middleware
- error middleware

Block 3 — API

- `req.body`
- `req.params`
- `req.query`
- status codes
- REST API
- CORS

Block 4 — Authentication

- JWT
- bcrypt
- authentication vs authorization
- protected routes

- role-based authorization

Block 5 — Architecture

- routes
 - controllers
 - models
 - middleware
 - MVC
-

Lekin preparation ka actual method ye hoga

Sirf question → answer yaad nahi karna.

Har question ke liye 4 cheezein:

1. **Definition**
2. **Simple example**
3. **Difference/use case**
4. **Follow-up question**

Example:

useEffect kya hai?

Tumhe pata hona chahiye:

Definition:

useEffect side effects perform karne ke liye use hota hai.

Example:

API call, event listener, timer etc.

Follow-up:

- Dependency array kya hai?
- Empty dependency array?
- Cleanup function?
- Component re-render hone par kya hota hai?

Bas. Isse interviewer ke 4–5 questions ek saath cover ho jaate hain.

Tumhare liye best strategy

Tum already bahut saare questions collect kar chuki ho. **Ab naye 500 questions collect karna bilkul band.** 🤔

Hum ek fixed interview bank bana sakte hain:

Technology	Questions
JavaScript	50
React	40
Node.js	25
Express	25
MongoDB	30
SQL	30
Git/GitHub	15
Projects	25

Total ≈ 240 questions

Lekin important baat:

Ye 240 alag-alag ratne wale questions nahi honge.

Ek JS topic ke andar uske **follow-ups** honge. Isliye actual preparation manageable rahegi.

Aur tumhare liye ek rule

Interview answer English mein hoga, preparation Hindi + English mix mein.

Tumhe English sentences ratne ki zarurat nahi.

Example:

Q: What is closure?

Tum apne mind mein pehle samjho:

“Function apne outer scope ke variables ko yaad rakhta hai.”

Phir interview mein simple English:

“A closure is a function that remembers and can access variables from its outer scope even after the outer function has finished executing.”

Bas. Fancy English ki zarurat nahi.

♥ To ab hum exactly aise kar sakte hain:

Phase 1 → JavaScript ke 50 most important interview questions

Phase 2 → React ke 40

Phase 3 → Node + Express ke 50

Phase 4 → MongoDB + SQL

Phase 5 → Tumhare projects ke questions

Phase 6 → Mock interview

Aur har question mein main tumhe **interview-level English answer + simple Hindi explanation + likely follow-up** dunga.

Isse tumhara preparation scattered nahi hoga.

Dekho yaar main asal mein sare question cover karna chahti Hun ki matlab Main Nahin chahti ki interview mujhse koi sawal puche to main uska jawab Na De paaun isliye tum mujhe freshers ke liye most important jitne bhi question hai FIR chahe vah basic Hai chahe vah advance hai aur pressure sare hi question mere ko batao theek hai pyara Sa answer do FIR unke followup question bhi do is type Se mujhe fark Nahin padta hai ki questions kitne jyada ho jaenge but mujhe important interview question do sare ke sare a to z ki isase bahar guarantee ke sath kisse Bahar kuchh Nahin aane Wala 100%

Haan yaar, ab tumhari requirement **100% clear** hai. ♥

Tumhe **“top 20 questions”** wali preparation nahi chahiye. Tumhe **Freshers MERN/Full-Stack interview ka complete question bank** chahiye — basic se advanced tak — taaki koi interviewer normal range mein kuch bhi pooche to tum blank na ho.

Bas ek honest correction: **kisi bhi company ke liye 100% guarantee nahi di ja sakti** ki is list ke bahar kuch nahi aayega. Interviewer apne project, company aur

interviewer ke experience ke hisaab se unexpected question pooch sakta hai.
Lekin hum syllabus ko itna comprehensive bana sakte hain ki standard fresher MERN interviews ka almost पूरा surface area cover ho.

Aur ek aur important cheez: **500 questions ko 500 answers ki tarah ratna nahi hai.** Hum related questions ko concepts ke andar organize karenge.



COMPLETE FRESHER MERN INTERVIEW MASTER BANK

Hum roughly ye coverage rakhenge:

Section	Approx. Questions
JavaScript	100+
React	80+
Node.js	50+
Express.js	50+
MongoDB/Mongoose	60+
SQL/DBMS	60+
HTML/CSS	50+
Git/GitHub	25+
REST API	25+
Authentication/Security	35+
Project-based	50+
HR/Technical mixed	30+

Total: 600+ questions/follow-ups easily ho sakte hain.

Aur tumhari preparation ka format fixed rahega:

**Question → Interview-level English answer → Simple Hindi meaning
→ Example/code where needed → Follow-up questions**

PART 1 — JAVASCRIPT

JavaScript ko sabse zyada depth mein karenge because **React + Node dono JavaScript par depend karte hain.**

A. JavaScript Fundamentals

1. What is JavaScript?
2. Features of JavaScript?
3. JavaScript vs Java?
4. Is JavaScript interpreted or compiled?
5. What are JavaScript engines?
6. What is ECMAScript?
7. ES5 vs ES6?
8. What are ES6 features?
9. What are variables?
10. var, let, and const difference?
11. What is scope?
12. Global scope?
13. Function scope?
14. Block scope?
15. What is lexical scope?
16. What is scope chain?
17. What is hoisting?
18. What gets hoisted?
19. Are let and const hoisted?
20. What is Temporal Dead Zone?
21. What are primitive data types?
22. What are reference data types?
23. Primitive vs reference?
24. What is dynamically typed language?
25. null vs undefined?
26. NaN?
27. Why is typeof null object?
28. What is type coercion?
29. Implicit vs explicit conversion?
30. == vs ===?

- 31. != vs !==?
- 32. Truthy and falsy values?
- 33. What is Object.is()?
- 34. What is strict mode?
- 35. What is "use strict"?

Follow-ups

- Why does var allow redeclaration?
 - Why can't const be reassigned?
 - Can a const object be modified?
 - Why is NaN !== NaN?
 - What is the difference between null and an empty object?
-

B. Functions

- 36. What is a function?
- 37. Function declaration vs function expression?
- 38. What is an anonymous function?
- 39. What is an arrow function?
- 40. Arrow function vs normal function?
- 41. What is a callback function?
- 42. What is a higher-order function?
- 43. What is a first-class function?
- 44. What are default parameters?
- 45. Rest parameter?
- 46. Spread operator?
- 47. Rest vs spread?
- 48. What is destructuring?
- 49. Array destructuring?
- 50. Object destructuring?
- 51. What is a nested function?
- 52. What is a closure?
- 53. Why are closures useful?
- 54. What is lexical environment?
- 55. What is an IIFE?

- 56. What is recursion?
- 57. What is currying?
- 58. What is function composition?
- 59. What is memoization?

Follow-ups

- Can you give a real-world example of closure?
 - Where are closures used in React?
 - What problems can closures cause?
 - What is a callback hell?
 - How do Promises solve callback hell?
-



C. this

- 60. What is this?
- 61. How is this determined?
- 62. this in global scope?
- 63. this inside a normal function?
- 64. this inside an object?
- 65. this inside an arrow function?
- 66. call()?
- 67. apply()?
- 68. bind()?
- 69. call vs apply?
- 70. bind vs call?

Follow-up

Why doesn't an arrow function have its own this?



D. Arrays & Objects

- 71. What is an array?
- 72. map()?
- 73. filter()?
- 74. reduce()?

75. `forEach()`?
76. `find()`?
77. `findIndex()`?
78. `some()`?
79. `every()`?
80. `includes()`?
81. `indexOf()`?
82. `slice()` vs `splice()`?
83. `sort()`?
84. `reverse()`?
85. `concat()`?
86. `flat()`?
87. `flatMap()`?
88. Mutable vs immutable methods?
89. How to remove duplicates from an array?
90. How to find frequency?
91. How to find max/min?
92. How to flatten an array?
93. How to merge arrays?
94. What is an object?
95. `Object.keys()`?
96. `Object.values()`?
97. `Object.entries()`?
98. `hasOwnProperty()`?
99. Object destructuring?
100. Shallow copy?
101. Deep copy?
102. `structuredClone()`?
103. Spread copy?
104. `Object.assign()`?
105. What is immutability?

Follow-ups

- Does `map()` modify the original array?
- Does `sort()` modify the original array?
- Why should we avoid mutation in React?

- Difference between map() and forEach()?
-

E. Asynchronous JavaScript

106. Synchronous vs asynchronous?
107. What is asynchronous programming?
108. What is the call stack?
109. What is the event loop?
110. What is callback queue?
111. What is microtask queue?
112. What is task/macrotask queue?
113. What is a Promise?
114. Promise states?
115. resolve() vs reject()?
116. .then()?
117. .catch()?
118. .finally()?
119. async function?
120. await?
121. Promise chaining?
122. Callback vs Promise?
123. Promise vs async/await?
124. Promise.all()?
125. Promise.allSettled()?
126. Promise.race()?
127. Promise.any()?
128. Error handling with async/await?
129. What happens if an awaited Promise rejects?
130. What is callback hell?

★ Very important follow-up

Interviewer:

“What will be the output?”

and gives you:

```
console.log("A");
```

```
setTimeout(() => console.log("B"), 0);
```

```
Promise.resolve().then(() => console.log("C"));
```

```
console.log("D");
```

Tumhe **event loop** + **microtask/macrotask order** explain karna aana chahiye.

F. Advanced JavaScript

131. Prototype?
132. Prototype chain?
133. __proto__?
134. prototype property?
135. Constructor function?
136. Classes?
137. Class vs constructor function?
138. Inheritance?
139. Encapsulation?
140. Polymorphism?
141. Getters/setters?
142. new keyword?
143. Garbage collection?
144. Memory leak?
145. Debouncing?
146. Throttling?
147. Event bubbling?
148. Event capturing?
149. Event delegation?
150. DOM?
151. BOM?
152. Cookies?
153. localStorage?
154. sessionStorage?

- 155. Cookies vs localStorage?
 - 156. JSON?
 - 157. JSON.stringify?
 - 158. JSON.parse?
 - 159. Fetch API?
 - 160. XMLHttpRequest?
 - 161. CORS?
 - 162. Web APIs?
 - 163. Modules?
 - 164. CommonJS?
 - 165. ES Modules?
 - 166. import/export?
 - 167. require()?
 - 168. Dynamic import?
 - 169. Optional chaining?
 - 170. Nullish coalescing?
 - 171. Logical operators?
 - 172. Optional chaining vs &&?
 - 173. ?? vs ||?
-

PART 2 — REACT

React ke questions ko bhi **basic** → **hooks** → **rendering** → **routing** → **API** → **optimization** → **advanced** karenge.

React Fundamentals

- 1. What is React?
- 2. Why React?
- 3. Features of React?
- 4. Library vs framework?
- 5. React vs Angular?
- 6. React vs vanilla JS?
- 7. What is JSX?
- 8. Why JSX?
- 9. Is JSX HTML?

10. How does JSX work?
 11. What is a component?
 12. Functional component?
 13. Class component?
 14. Functional vs class?
 15. Props?
 16. State?
 17. Props vs state?
 18. Component reusability?
 19. Component composition?
 20. Parent-child relationship?
 21. Conditional rendering?
 22. Lists?
 23. Keys?
 24. Why should keys be unique?
 25. What happens if we use array index as key?
-

React Hooks

26. What are Hooks?
27. Rules of Hooks?
28. `useState()`?
29. How does state update work?
30. Functional state update?
31. Multiple state updates?
32. `useEffect()`?
33. Why `useEffect`?
34. Dependency array?
35. Empty dependency array?
36. No dependency array?
37. Specific dependencies?
38. Cleanup function?
39. When does cleanup execute?
40. `useRef()`?
41. `useRef` vs `useState`?

- 42. DOM reference using useRef?
 - 43. useContext()?
 - 44. Context API?
 - 45. Prop drilling?
 - 46. Lifting state up?
 - 47. Custom hooks?
 - 48. useMemo()?
 - 49. useCallback()?
 - 50. React.memo()?
 - 51. useMemo vs useCallback?
 - 52. useCallback vs React.memo?
 - 53. useReducer()?
 - 54. useReducer vs useState?
-

React Rendering

- 55. What is Virtual DOM?
 - 56. Virtual DOM vs Real DOM?
 - 57. Reconciliation?
 - 58. Diffing algorithm?
 - 59. What causes a re-render?
 - 60. Does changing a normal variable cause re-render?
 - 61. Does changing props cause re-render?
 - 62. Does changing state cause re-render?
 - 63. Parent re-render effect on child?
 - 64. How to prevent unnecessary re-renders?
 - 65. What is batching?
 - 66. Why state updates may appear asynchronous?
 - 67. What is StrictMode?
 - 68. Why does useEffect sometimes run twice in development?
-

React Router

- 69. What is React Router?

- 70. Client-side routing?
 - 71. SPA?
 - 72. BrowserRouter?
 - 73. Routes?
 - 74. Route?
 - 75. Link?
 - 76. NavLink?
 - 77. useNavigate?
 - 78. useParams?
 - 79. useLocation?
 - 80. Dynamic routes?
 - 81. Nested routes?
 - 82. Protected routes?
 - 83. Authentication-based routing?
-

React API & Practical

- 84. How to call API in React?
 - 85. Fetch vs Axios?
 - 86. Loading state?
 - 87. Error state?
 - 88. CRUD in React?
 - 89. Form handling?
 - 90. Controlled component?
 - 91. Uncontrolled component?
 - 92. Form validation?
 - 93. Authentication flow?
 - 94. JWT with React?
 - 95. Token storage?
 - 96. Logout?
 - 97. Protected API?
 - 98. Pagination?
 - 99. Search/filter?
 - 100. Debounced search?
-

PART 3 — NODE.JS

Node ko bhi complete cover karenge:

1. What is Node.js?
2. Why Node.js?
3. Node vs browser JavaScript?
4. Node.js architecture?
5. V8 engine?
6. Event-driven architecture?
7. Non-blocking I/O?
8. Single-threaded?
9. Is Node actually single-threaded?
10. Event loop in Node?
11. libuv?
12. npm?
13. package.json?
14. package-lock.json?
15. dependencies vs devDependencies?
16. npm install?
17. npm ci?
18. npm scripts?
19. CommonJS?
20. ES Modules?
21. require vs import?
22. Node modules?
23. Built-in modules?
24. fs?
25. path?
26. http?
27. events?
28. process?
29. environment variables?
30. .env?
31. nodemon?
32. streams?

- 33. buffers?
 - 34. middleware concept?
 - 35. error handling?
 - 36. synchronous vs asynchronous Node APIs?
 - 37. callback APIs?
 - 38. Promise-based APIs?
 - 39. EventEmitter?
 - 40. worker threads?
 - 41. clustering?
 - 42. scalability?
 - 43. blocking code?
 - 44. process.nextTick?
 - 45. setImmediate?
-

PART 4 — EXPRESS

- 1. What is Express?
- 2. Why Express?
- 3. Node vs Express?
- 4. Create Express server?
- 5. Routing?
- 6. HTTP methods?
- 7. Middleware?
- 8. Why middleware?
- 9. Application middleware?
- 10. Router middleware?
- 11. Built-in middleware?
- 12. Third-party middleware?
- 13. Custom middleware?
- 14. Error middleware?
- 15. app.use()?
- 16. express.json()?
- 17. req?
- 18. res?
- 19. next()?

- 20.req.body?
 - 21.req.params?
 - 22.req.query?
 - 23.params vs query?
 - 24.res.send?
 - 25.res.json?
 - 26.res.status?
 - 27.Status codes?
 - 28.REST API?
 - 29.REST principles?
 - 30.CRUD API?
 - 31.Controller?
 - 32.Routes vs controllers?
 - 33.MVC?
 - 34.CORS?
 - 35.Authentication?
 - 36.Authorization?
 - 37.JWT?
 - 38.JWT middleware?
 - 39.bcrypt?
 - 40.Password hashing?
 - 41.Protected route?
 - 42.Role-based authorization?
 - 43.Error handling?
 - 44.Async error handling?
 - 45.Validation?
 - 46.API versioning?
 - 47.HTTP headers?
 - 48.Content-Type?
 - 49.Authorization header?
 - 50.Rate limiting?
-



PART 5 — MONGODB + MONGOOSE

Ye MERN interview mein **bahut important** hai.

MongoDB

1. What is MongoDB?
2. NoSQL?
3. MongoDB vs SQL?
4. Database?
5. Collection?
6. Document?
7. BSON?
8. _id?
9. ObjectId?
10. Embedded documents?
11. References?
12. Embedded vs referenced?
13. CRUD?
14. insertOne?
15. find?
16. findOne?
17. updateOne?
18. updateMany?
19. deleteOne?
20. deleteMany?
21. MongoDB operators?
22. \$set
23. \$push
24. \$pull
25. \$inc
26. \$in
27. \$or
28. \$and
29. \$regex
30. Aggregation?
31. \$match

- 32. \$group
- 33. \$lookup
- 34. \$project
- 35. Index?
- 36. Why indexes?
- 37. Compound index?
- 38. Unique index?
- 39. Transactions?
- 40. MongoDB Atlas?

Mongoose

- 41. What is Mongoose?
 - 42. Schema?
 - 43. Model?
 - 44. Schema vs model?
 - 45. Validation?
 - 46. required?
 - 47. unique?
 - 48. default?
 - 49. enum?
 - 50. timestamps?
 - 51. populate?
 - 52. find() vs findOne()?
 - 53. lean()?
 - 54. middleware/hooks?
 - 55. pre/post hooks?
 - 56. ObjectId reference?
 - 57. Mongoose validation vs MongoDB validation?
-

PART 6 — AUTHENTICATION & SECURITY

Ye tumhare MERN project ke liye **especially important** hai.

- 1. Authentication?
- 2. Authorization?

3. Authentication vs authorization?
 4. JWT?
 5. JWT structure?
 6. Header/payload/signature?
 7. JWT verification?
 8. Access token?
 9. Refresh token?
 10. Password hashing?
 11. bcrypt?
 12. Hashing vs encryption?
 13. Salt?
 14. Why never store plain passwords?
 15. Protected routes?
 16. Role-based authorization?
 17. Cookies?
 18. HttpOnly cookie?
 19. Secure cookie?
 20. SameSite?
 21. XSS?
 22. CSRF?
 23. CORS?
 24. SQL injection?
 25. NoSQL injection?
 26. Input validation?
 27. Sanitization?
 28. Rate limiting?
 29. Helmet?
 30. Environment secrets?
-

PART 7 — SQL + DBMS

Tumne SQL alag se prepare karne ka bhi plan banaya hai, isliye isko skip nahi karna.

Important areas:

- DBMS vs RDBMS
- Primary key
- Foreign key
- Candidate key
- Super key
- Unique constraint
- NOT NULL
- DEFAULT
- CHECK
- Composite key
- Normalization
- 1NF
- 2NF
- 3NF
- BCNF
- Denormalization
- ACID
- Transactions
- COMMIT
- ROLLBACK
- JOINS
- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN
- GROUP BY
- HAVING
- WHERE
- ORDER BY
- DISTINCT
- Subqueries
- Correlated subquery
- UNION
- UNION ALL
- Index

- View
 - Stored procedure
 - Trigger
 - DELETE vs TRUNCATE vs DROP
 - DELETE vs DROP
 - NULL handling
 - Aggregate functions
 - Window functions
 - Constraints
 - Referential integrity
-

PART 8 — HTML + CSS

Full-stack interview mein bhi basic frontend questions aa sakte hain.

HTML

- What is HTML?
- HTML5?
- Semantic HTML?
- div vs section?
- article?
- header/footer/nav?
- form?
- input types?
- GET vs POST?
- id vs class?
- block vs inline?
- accessibility?
- alt attribute?
- SEO basics?
- meta tags?
- viewport?
- localStorage?
- cookies?

CSS

- CSS?
 - Inline/internal/external?
 - Box model?
 - margin vs padding?
 - display?
 - block/inline/inline-block?
 - flexbox?
 - grid?
 - position?
 - relative/absolute/fixed/sticky?
 - z-index?
 - specificity?
 - pseudo-class?
 - pseudo-element?
 - media queries?
 - responsive design?
 - units: px, %, rem, em, vh, vw?
 - CSS variables?
 - Tailwind CSS?
 - Bootstrap?
 - Flexbox vs Grid?
-

● PART 9 — GIT/GITHUB

1. Git?
2. GitHub?
3. Git vs GitHub?
4. Repository?
5. Working directory?
6. Staging area?
7. Commit?
8. Branch?
9. Merge?

10. Rebase?
11. Pull?
12. Push?
13. Fetch?
14. Clone?
15. Fork?
16. Pull request?
17. Merge conflict?
18. .gitignore?
19. git stash?
20. git reset?
21. git revert?
22. git restore?
23. amend commit?
24. How to remove secret accidentally committed?
25. Why should .env not be pushed?

Tumhare actual project mein GitHub push protection wali situation bhi hui hai, isliye **secret management** + **.env** ka answer tumhe particularly strong hona chahiye.

PART 10 — REST API

- API kya hai?
- REST kya hai?
- REST API?
- REST principles?
- HTTP methods?
- GET vs POST?
- PUT vs PATCH?
- DELETE?
- Idempotency?
- Statelessness?
- HTTP status codes?
- Headers?

- Request body?
 - Query parameters?
 - Path parameters?
 - Authentication header?
 - JSON?
 - API versioning?
 - Pagination?
 - Filtering?
 - Sorting?
 - Rate limiting?
 - Error response structure?
-

PART 11 — PROJECT QUESTIONS

Ye section tum bilkul skip mat karna.

Tumhare project ke baare mein interviewer ye pooch sakta hai:

1. Explain your project.
2. Why did you choose this project?
3. What technologies did you use?
4. Why MERN?
5. What was your role?
6. Explain project architecture.
7. Frontend-backend kaise connected hai?
8. API kaise banayi?
9. Database kaise design ki?
10. Authentication kaise implement ki?
11. JWT kahan use kiya?
12. Password kaise secure kiya?
13. Protected routes kaise banaye?
14. Error handling kaise ki?
15. Validation kaise ki?
16. CORS issue kaise solve kiya?
17. Environment variables kyu?
18. Deployment kaise ki?

19. Biggest challenge?
 20. Kisi bug ko kaise solve kiya?
 21. What would you improve?
 22. Scalability kaise improve karoge?
 23. Database slow ho to?
 24. API slow ho to?
 25. Security kaise improve karoge?
 26. Agar 10,000 users aa jayein?
 27. Why MongoDB?
 28. Why React?
 29. Why Express?
 30. Why Node?
 31. AI/API integration kaise ki?
 32. Third-party API fail ho jaye to?
 33. Project mein sabse difficult feature?
 34. Future improvements?
 35. What did you learn?
-

Aur ab sabse important

Hum ye sab ek hi baar mein nahi padhenge.

Tumhare liye main isko **proper interview course** banaunga:

PHASE 1

JavaScript — complete A to Z

PHASE 2

React — complete A to Z

PHASE 3

Node.js

PHASE 4

Express.js

PHASE 5

MongoDB + Mongoose

PHASE 6

SQL + DBMS

PHASE 7

HTML + CSS

PHASE 8

Git/GitHub + REST API + Security

PHASE 9

Tumhare projects ke 50+ questions

PHASE 10

Mock Technical Interview

Aur har phase mein basic se advanced tak kuch intentionally skip nahi karenge.

Sabse important: **main questions ko random list ki tarah nahi dunga.** Har topic ke andar:

Main Question → Answer → Follow-ups → Follow-up Answers → Practical/Coding Question

Is tarah tumhe interviewer agar ek answer se doosre concept par le jaaye, tab bhi tum handle kar paogi.

Agar tumhara goal genuinely “mujhse jo normal fresher MERN interview mein poocha ja sakta hai, uska answer mujhe aana chahiye” hai, to यही approach sahi hai. 💪

Filhal tumhen DBMS a steamer CSS gate gate vagaira batane ki jarurat Nahin Hai because iska to obviously limited seen hota h mujhe sirf mern ki kamaal ki easy preze language with all follow up questions chaiye first start with javascript and ha same question ko interview aur kaise puch skta h ye bhi btana and question ki wording se related hi full interview level easy English answer hona chahiye ki interviewer bhi bole ki ye ladki hi right fit h freshers ke liye jo bhi important

question Hai sare karana aur yes Dhyan rakhna ki faltu questions mat Lana bich mein faltu means Aisa mat kar dena ki jo 10 sal experience logo se related hai vo bhi freshers mha important questions saare k saare a to z aur answer bhi interview level hi dena kbhi bhot lamba ya chota sa answer hi dedo jitna enough hota h sirf utna hi dena bad follow up questions ke bhi answers Dena Dena

Bilkul. **Ab scope fixed hai:** sirf MERN — aur filhaal JavaScript se A→Z. DBMS, SQL, HTML, CSS, aptitude etc. beech mein nahi launga.

Aur ek important rule: “A→Z” ka matlab har obscure JavaScript feature nahi, balki fresher MERN interview mein realistically pooche ja sakne wale basic + intermediate + important advanced questions. 10-year-experience wale niche questions nahi.

Har question ka format:

Q → interviewer ise aur kaise pooch sakta hai → interview-level easy English answer → likely follow-ups → unke answers

Answers na ratne layak unnecessarily long honge, na one-line incomplete.

JAVASCRIPT — FRESHER MERN INTERVIEW MASTER SERIES

PART 1 — JAVASCRIPT BASICS

Q1. What is JavaScript?

Interviewer is question ko aise bhi pooch sakta hai:

- What do you know about JavaScript?
- Why is JavaScript used?
- What is the role of JavaScript in web development?
- Why is JavaScript important in MERN?

Answer:

JavaScript is a high-level, dynamically typed programming language used to make web applications interactive and dynamic. It can run in

the browser as well as on the server using Node.js. In the MERN stack, JavaScript is used for both frontend and backend development.

Follow-up: Is JavaScript only used for frontend?

Answer:

No. JavaScript is used for both frontend and backend development. In the browser we use it with technologies like React, and on the backend we can use Node.js.

Follow-up: What is the difference between JavaScript and Node.js?

Answer:

JavaScript is the programming language, while Node.js is a runtime environment that allows JavaScript to run outside the browser, mainly on the server.

Q2. What are the main features of JavaScript?

Also asked as:

- Why is JavaScript popular?
- What are the characteristics of JavaScript?
- Why do we use JavaScript in web applications?

Answer:

JavaScript is dynamically typed, object-based, single-threaded, event-driven and supports asynchronous programming. It also supports first-class functions and can run on both the client side and server side.

Follow-up: What does dynamically typed mean?

It means we don't have to specify the data type of a variable explicitly. The type is determined at runtime.

```
let value = 10;  
value = "Hello";
```

Q3. What is ECMAScript?

Also asked as:

- What is ES?
- What is ES6?
- Is JavaScript and ECMAScript the same?

Answer:

ECMAScript is the standard specification on which JavaScript is based. JavaScript is an implementation of ECMAScript along with additional features provided by its environment. ES6 is the sixth major version of ECMAScript, also known as ECMAScript 2015.

Follow-up: What important features came with ES6?

Some important features are let, const, arrow functions, template literals, destructuring, spread/rest operators, classes, promises and modules.

Q4. What is a variable in JavaScript?

Also asked as:

- How do you declare variables in JavaScript?
- What are the ways to create variables?

Answer:

A variable is a named storage location used to hold a value. In JavaScript, we mainly declare variables using var, let, and const.

Q5. Difference between var, let and const?

 **Very frequently asked.**

Also asked as:

- Why do we use let and const instead of var?
- Which one do you prefer and why?
- Can you explain variable declarations in JavaScript?

Answer:

var is function-scoped and can be redeclared and reassigned. let and const are block-scoped. let can be reassigned but cannot be redeclared in the same scope, while const cannot be reassigned after initialization. In modern JavaScript, I prefer const by default and use let when the value needs to change.

Follow-up: Can const object properties be changed?

Answer:

Yes. const prevents reassignment of the variable, but it does not make the object immutable.

```
const user = { name: "Pranjal" };
```

```
user.name = "Rahul"; // allowed
```

Follow-up: Can we declare a const variable without assigning a value?

No. A const variable must be initialized when it is declared.

```
const x = 10;
```

Q6. What is scope in JavaScript?

Also asked as:

- What is the scope of a variable?
- Explain different types of scope.
- Where can a variable be accessed?

Answer:

Scope defines where a variable can be accessed in a program. JavaScript mainly has global scope, function scope and block scope.

Follow-up: What is global scope?

A variable declared outside functions or blocks generally has global scope and can be accessed from different parts of the program, subject to the environment and module rules.

Follow-up: What is function scope?

A variable declared with `var` inside a function is accessible only within that function.

Follow-up: What is block scope?

Variables declared using `let` and `const` inside a block `{}` are accessible only within that block.

Q7. What is hoisting?

 **Very common.**

Also asked as:

- Does JavaScript move declarations to the top?
- Why can we sometimes access a variable before declaration?
- How does hoisting work?

Answer:

Hoisting is JavaScript's behavior where declarations are processed before the code is executed. Function declarations can be called before their declaration, while variables declared with `var` are hoisted and initialized with `undefined`. `let` and `const` are also hoisted, but they remain in the Temporal Dead Zone until their declaration is reached.

Follow-up: What happens with `var`?

```
console.log(x);  
var x = 10;
```

It prints `undefined` because the declaration is hoisted, but the assignment happens later.

Follow-up: What happens with let?

```
console.log(x);  
let x = 10;
```

It gives a ReferenceError because x is in the Temporal Dead Zone before its declaration.

Q8. What is the Temporal Dead Zone?

Also asked as:

- Why does let give ReferenceError before declaration?
- What is TDZ?

Answer:

The Temporal Dead Zone is the period between entering a scope and reaching the declaration of a let or const variable. During this period, the variable exists but cannot be accessed.

Q9. What are data types in JavaScript?

Also asked as:

- How many data types are there?
- What are primitive and non-primitive data types?

Answer:

JavaScript has **seven primitive data types**: String, Number, BigInt, Boolean, Undefined, Null and Symbol. The main non-primitive type is Object, which includes arrays, functions and other objects.

Follow-up: What is the difference between primitive and reference types?

Primitive values are stored and copied as individual values, while objects are reference-based. So copying an object can make both variables refer to the same object.

Q10. What is the difference between null and undefined?

Also asked as:

- When do we use null?
- What is undefined?
- Are null and undefined the same?

Answer:

undefined usually means a value has not been assigned or is missing, while null is an intentional empty value assigned by the developer.

```
let a;  
console.log(a); // undefined
```

```
let b = null;  
console.log(b); // null
```

Follow-up: What is typeof null?

typeof null returns "object". This is a historical behavior in JavaScript and is considered a language quirk.

Q11. What is NaN?

Also asked as:

- What does NaN mean?
- Is NaN a number?
- How do you check for NaN?

Answer:

NaN means Not-a-Number. It represents an invalid numeric result. Interestingly, its type is number.

```
console.log(typeof NaN); // "number"
```

Follow-up: Is NaN equal to itself?

No. `NaN === NaN` is false. We can use `Number.isNaN()` to check whether a value is actually NaN.

Q12. What is type coercion?

Also asked as:

- How does JavaScript convert data types?
- What is implicit type conversion?
- Explain type conversion in JavaScript.

Answer:

Type coercion is the automatic or explicit conversion of one data type into another. JavaScript can perform implicit conversion during operations, and we can also explicitly convert values using functions such as `Number()`, `String()` and `Boolean()`.

Follow-up:

```
console.log("5" + 2);  
console.log("5" - 2);
```

Answer:

"5" + 2 gives "52" because + performs string concatenation in this case, while "5" - 2 gives 3 because subtraction converts the string to a number.

Q13. Difference between == and ===?

Must know.

Also asked as:

- Loose equality vs strict equality?
- Which equality operator do you prefer?

Answer:

`==` compares values after type conversion when necessary, while `===` compares both value and type without implicit conversion. I generally prefer `===` because it gives more predictable results.

```
5 == "5" // true
5 === "5" // false
```

Q14. What are truthy and falsy values?

Also asked as:

- What values are considered false in JavaScript?
- Explain truthy/falsy.

Answer:

A **truthy** value behaves like true in a Boolean context, while a **falsy** value behaves like false. Common falsy values are false, 0, -0, 0n, "", null, undefined, and NaN.

Follow-up: Is an empty array truthy?

Yes. Arrays and objects are truthy, even when they are empty.

```
Boolean([]); // true
Boolean({}); // true
```

Q15. What is the difference between primitive and reference values?

Also asked as:

- How are objects copied?
- Why does changing one object affect another?
- Value vs reference?

Answer:

Primitive values are copied by value. Objects and arrays are reference-based, so assigning one object to another variable copies the reference to the same object rather than creating a new independent object.

```
const a = { name: "A" };  
const b = a;
```

```
b.name = "B";
```

```
console.log(a.name); // B
```

Follow-up: How do you create an independent copy?

For a simple object, we can use the spread operator. For deeply nested data, we can use `structuredClone()` when appropriate.

Q16. What is an immutable value?

Also asked as:

- What does immutability mean?
- Why is immutability important in React?

Answer:

Immutability means we don't directly modify an existing value. Instead, we create a new value with the required changes. It is especially important in React because state updates work best when we treat state as immutable.

Q17. What is strict mode?

Also asked as:

- What does "use strict" do?
- Why do we use strict mode?

Answer:

Strict mode enables stricter JavaScript rules. It helps catch common mistakes, prevents some unsafe operations and makes certain behaviors more predictable.

Q18. What is the difference between function declaration and function expression?

Also asked as:

- How can functions be defined in JavaScript?
- What is a function expression?

Answer:

A function declaration is declared directly with the function keyword, while a function expression stores a function inside a variable. Function declarations are hoisted differently and can be called before their declaration.

```
function add(a, b) {  
  return a + b;  
}
```

```
const subtract = function (a, b) {  
  return a - b;  
};
```

Q19. What is an arrow function?

Also asked as:

- Why do we use arrow functions?
- How is an arrow function different from a normal function?

Answer:

An arrow function is a shorter syntax for writing functions. Its most important difference is that it does not have its own this; it inherits this from its surrounding lexical scope.

```
const add = (a, b) => a + b;
```

Follow-up: Can an arrow function be used as a constructor?

No. Arrow functions do not have their own prototype and cannot be used with the new keyword.

Q20. What is a callback function?

Also asked as:

- What is callback?
- Why are callbacks used?
- What is a callback in asynchronous JavaScript?

Answer:

A callback is a function passed to another function as an argument so that it can be executed later or when a particular operation is completed.

```
function greet(name, callback) {  
  callback(name);  
}
```

Follow-up: What is callback hell?

Callback hell happens when multiple asynchronous callbacks are nested inside each other, making the code difficult to read and maintain. Promises and async/await provide cleaner alternatives.

Q21. What is a higher-order function?

Also asked as:

- What are higher-order functions?
- Why are map/filter/reduce called higher-order functions?

Answer:

A higher-order function is a function that accepts another function as an argument, returns a function, or does both.

For example, `map()`, `filter()` and `reduce()` are higher-order functions because they accept callback functions.

Q22. What are first-class functions?

Answer:

In JavaScript, functions are first-class values. This means we can store functions in variables, pass them as arguments, return them from other functions and store them in objects or arrays.

Q23. What is a closure?

🔥🔥 **Extremely important for interviews.**

Also asked as:

- Explain closure with an example.
- How does a function remember variables after execution?
- What is lexical closure?
- Where are closures useful?

Answer:

A closure is created when a function remembers and can access variables from its outer lexical scope even after the outer function has finished executing.

```
function counter() {  
  let count = 0;  
  
  return function () {  
    count++;  
    return count;  
  };  
}
```

```
const increment = counter();
```

```
console.log(increment()); // 1  
console.log(increment()); // 2
```

Follow-up: Why is closure useful?

Closures are useful for data privacy, maintaining state and creating functions with remembered values. They are also commonly used in callbacks and asynchronous code.

Follow-up: What is the main idea behind the example?

The inner function keeps access to count, even though counter() has already finished executing.

Q24. What is lexical scope?

Also asked as:

- What is lexical environment?
- How does a function access variables from outside?

Answer:

Lexical scope means that the accessibility of variables is determined by where functions and blocks are written in the source code, not where they are called.

Q25. What is an IIFE?

Also asked as:

- What is Immediately Invoked Function Expression?
- Why do we use IIFE?

Answer:

IIFE stands for Immediately Invoked Function Expression. It is a function that is defined and executed immediately. It was commonly used to create a private scope and avoid polluting the global scope.

```
(function () {  
  console.log("Executed immediately");  
})();
```

STOP POINT

Yahan tak humne **JavaScript Fundamentals + Functions** cover kiya hai.

Agla block isi JavaScript series ka hoga:

this → call/apply/bind → arrays → map/filter/reduce → objects →
destructuring → spread/rest → shallow/deep copy → prototypes →
classes

Uske baad:

**Promises → async/await → Event Loop → Call Stack → Microtask/Macrotask →
DOM events → debounce/throttle → storage → modules → error handling →
advanced-but-fresher-relevant JS → coding questions.**

Aur haan — **har topic mein isi exact style mein follow-ups + alternate
interview wording + answers** rahenge.

Next

JavaScript Interview — Part 2

this → call/apply/bind → Arrays → Objects → Destructuring →
Spread/Rest → Copying

Q26. What is `this` in JavaScript?

Interviewer is tarah bhi pooch sakta hai:

- What does this refer to?
- How does this work in JavaScript?
- How is the value of this determined?
- What is the difference between this in normal and arrow functions?

Answer:

this is a special keyword that refers to the object associated with the current function execution. Its value depends on how the function is called. In an object method, it usually refers to that object, while in an arrow function, it is inherited from the surrounding scope.

Follow-up: What is this inside an object method?

```
const user = {  
  name: "Pranjal",  
  greet() {  
    console.log(this.name);  
  }  
};
```

```
user.greet();
```

Answer:

Here, this refers to the user object because the function is called as user.greet().

Follow-up: Does an arrow function have its own this?

No. An arrow function does not create its own this. It inherits this from its surrounding lexical scope.

Q27. What is the difference between this in a normal function and an arrow function?

Also asked as:

- Arrow function vs normal function regarding this
- Why is this different in arrow functions?

Answer:

A normal function gets its this value based on how it is called, while an arrow function inherits this from its surrounding scope.

```
const user = {
  name: "Pranjal",

  normal() {
    console.log(this.name);
  },

  arrow: () => {
    console.log(this.name);
  }
};
```

The normal method gets this from user, but the arrow function does not get its own this.

Q28. What are call(), apply(), and bind()?

 **Common follow-up question.**

Also asked as:

- How can you manually set this?
- Difference between call, apply and bind?
- How do you borrow a method from another object?

Answer:

call(), apply(), and bind() are methods used to control the value of this. call() and apply() immediately execute the function, while bind() returns a new function that can be executed later.

Example

```
function greet(city) {
  console.log(this.name, city);
}

const user = {
  name: "Pranjal"
};
```

```
greet.call(user, "Roorkee");
```

```
greet.apply(user, ["Roorkee"]);
```

```
const newGreet = greet.bind(user, "Roorkee");  
newGreet();
```

Follow-up: call vs apply?

The main difference is how arguments are passed. `call()` takes arguments individually, while `apply()` takes them as an array or array-like object.

Follow-up: bind?

`bind()` does not execute the function immediately. It returns a new function with the specified this value.

Q29. What is an array in JavaScript?

Also asked as:

- How do arrays work in JavaScript?
- Can an array contain different data types?

Answer:

An array is an ordered collection used to store multiple values in a single variable. JavaScript arrays can contain values of different data types and are zero-indexed.

```
const data = [10, "Hello", true, { name: "A" }];
```

Q30. What is the difference between `map()` and `forEach()`?

 Very frequently asked.

Answer:

Both iterate over an array, but `map()` returns a new array containing the transformed values, while `forEach()` does not return a new array. I use `map()` when I need a transformed array and `forEach()` when I only need to perform an action for each element.

```
const nums = [1, 2, 3];
```

```
const result = nums.map(num => num * 2);  
// [2, 4, 6]
```

```
nums.forEach(num => console.log(num));
```

Follow-up: Does `map` modify the original array?

No, `map()` creates a new array. However, if the array contains objects, modifying those objects inside the callback can still affect the original objects.

Q31. What is `filter()`?

Also asked as:

- How do you filter an array?
- Difference between `map` and `filter`?

Answer:

`filter()` creates a new array containing only the elements that satisfy a given condition.

```
const nums = [1, 2, 3, 4];
```

```
const even = nums.filter(num => num % 2 === 0);
```

```
console.log(even); // [2, 4]
```

Follow-up: `map` vs `filter`?

`map()` transforms every element and returns the transformed array, while `filter()` returns only the elements that satisfy a condition.

Q32. What is `reduce()`?

🔥 Very common in MERN interviews.

Also asked as:

- How does reduce work?
- Where do you use reduce?
- Can reduce return an object?

Answer:

`reduce()` processes all elements of an array and combines them into a single result. That result can be a number, string, object, array or any other value.

```
const nums = [1, 2, 3, 4];
```

```
const sum = nums.reduce((total, num) => {  
  return total + num;  
}, 0);
```

```
console.log(sum); // 10
```

Follow-up: What is the second argument of reduce?

It is the initial value of the accumulator. In the example, 0 is the initial value.

Follow-up: Can reduce be used for frequency counting?

Yes.

```
const arr = ["a", "b", "a"];
```

```
const frequency = arr.reduce((acc, item) => {  
  acc[item] = (acc[item] || 0) + 1;  
  return acc;  
}, {});
```

Q33. Difference between map(), filter(), and reduce()?

Answer:

map() transforms every element, filter() selects elements based on a condition, and reduce() combines the elements into a single result.

Method	Main purpose
map()	Transform
filter()	Select
reduce()	Combine

Q34. What is find()?

Also asked as:

- find() vs filter()?
- How do you find an object in an array?

Answer:

find() returns the first element that satisfies the condition. If no element matches, it returns undefined.

```
const users = [  
  { id: 1, name: "A" },  
  { id: 2, name: "B" }  
];
```

```
const user = users.find(user => user.id === 2);
```

Follow-up: find vs filter?

find() returns the first matching element, while filter() returns an array containing all matching elements.

Q35. What is the difference between `some()` and `every()`?

Answer:

`some()` checks whether **at least one** element satisfies the condition, while `every()` checks whether **all** elements satisfy the condition.

```
[1, 2, 3].some(x => x > 2); // true
[1, 2, 3].every(x => x > 0); // true
```

Q36. Difference between `slice()` and `splice()`?

🔥 Common trap question.

Answer:

`slice()` returns a portion of an array without modifying the original array, while `splice()` can add, remove or replace elements and modifies the original array.

```
const arr = [1, 2, 3, 4];

arr.slice(1, 3); // [2, 3]

arr.splice(1, 2); // removes 2 and 3
```

Follow-up: Which one is immutable?

`slice()` does not modify the original array, while `splice()` does.

Q37. What is destructuring?

Also asked as:

- What is object destructuring?
- What is array destructuring?
- Why do we use destructuring?

Answer:

Destructuring is a convenient way to extract values from arrays or properties from objects and store them in variables.

Object:

```
const user = {  
  name: "Pranjal",  
  age: 21  
};
```

```
const { name, age } = user;
```

Array:

```
const numbers = [10, 20];
```

```
const [first, second] = numbers;
```

Follow-up: Where is destructuring commonly used in React?

It is commonly used with props and hooks.

```
function User({ name, age }) {  
  return <h1>{name}</h1>;  
}
```

Q38. What is the spread operator?

Also asked as:

- What does ... mean?
- How do you copy or merge arrays?
- How do you copy an object?

Answer:

The spread operator ... expands the elements of an iterable or the properties of an object. It is commonly used to copy and merge arrays or objects without directly modifying the original.

```
const a = [1, 2];  
const b = [...a, 3, 4];
```

```
const user = { name: "A" };  
const updatedUser = { ...user, age: 21 };
```

Follow-up: Is spread a deep copy?

No. Spread creates a **shallow copy**.

Q39. What is the rest operator?

Also asked as:

- What is the difference between spread and rest?
- How do you collect multiple arguments?

Answer:

The rest operator also uses ..., but instead of expanding values, it collects multiple values into an array or object.

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b, 0);  
}
```

Follow-up: Spread vs rest?

Spread expands values, while rest collects values. The same ... syntax gets its meaning from where it is used.

Q40. What is shallow copy?

Also asked as:

- What happens when you copy an object using spread?
- Is spread operator a deep copy?

Answer:

A shallow copy creates a new outer object or array, but nested objects are still shared by reference.

```
const user = {  
  name: "A",  
  address: {  
    city: "Roorkee"  
  }  
};  
  
const copy = { ...user };  
  
copy.address.city = "Delhi";  
  
console.log(user.address.city); // Delhi
```

Q41. What is deep copy?

Answer:

A deep copy creates an independent copy including nested objects, so changes to the copied object do not affect the original nested data.

A modern option is:

```
const copy = structuredClone(original);
```

Follow-up: Is JSON stringify/parse a perfect deep-copy solution?

No. It has limitations because it cannot correctly preserve values such as functions, undefined, some special objects and certain other data types.

Q42. What is immutability in arrays and objects?

Also asked as:

- How do you update an object without modifying the original?
- Why is immutability important in React?

Answer:

Immutability means creating a new array or object instead of directly modifying the existing one. This makes state changes predictable and is especially important in React.

```
const user = { name: "A", age: 20 };
```

```
const updatedUser = {  
  ...user,  
  age: 21  
};
```

Q43. What is an object in JavaScript?

Also asked as:

- How do you store key-value data?
- What is a JavaScript object?

Answer:

An object is a collection of key-value pairs used to represent related data and behavior. Its values can be primitive values, arrays, functions or other objects.

```
const user = {  
  name: "Pranjal",  
  age: 21,  
  greet() {  
    console.log("Hello");  
  }  
};
```

Q44. What are Object.keys(), Object.values(), and Object.entries()?

Answer:

Object.keys() returns an array of property names, Object.values() returns the property values, and Object.entries() returns key-value pairs as nested arrays.

```
const user = {  
  name: "A",  
  age: 21  
};
```

```
Object.keys(user);  
// ["name", "age"]
```

```
Object.values(user);  
// ["A", 21]
```

```
Object.entries(user);  
// [["name", "A"], ["age", 21]]
```

Q45. What is optional chaining?

Also asked as:

- What does ?. do?
- How do you safely access nested properties?

Answer:

Optional chaining ?. allows us to access a property or call a function without throwing an error when an intermediate value is null or undefined.

```
console.log(user?.address?.city);
```

Q46. What is the nullish coalescing operator ???

Also asked as:

- Difference between ?? and ||?
- How do you provide a default only when a value is null or undefined?

Answer:

The ?? operator returns the right-hand value only when the left-hand value is null or undefined.

```
const name = userName ?? "Guest";
```

Follow-up: ?? vs ||?

|| treats all falsy values such as 0, false, and "" as missing, while ?? only treats null and undefined as missing.

Q47. What is the difference between `Object.freeze()` and `Object.seal()`?

Answer:

`Object.freeze()` prevents adding, deleting and changing properties.
`Object.seal()` prevents adding and deleting properties, but existing properties can still be changed.

Follow-up: Is `Object.freeze()` a deep freeze?

No. It is shallow. Nested objects can still be modified unless they are frozen separately.

Q48. What is a prototype in JavaScript?

🔥 Important, but fresher-level depth enough.

Also asked as:

- What is prototype inheritance?
- How does inheritance work in JavaScript?
- Explain prototype chain.

Answer:

A prototype is an object from which another object can inherit properties and methods. JavaScript uses prototype-based

inheritance, so objects can access properties and methods through their prototype chain.

Follow-up: What is prototype chain?

If a property is not found on an object, JavaScript looks for it in that object's prototype, then the prototype's prototype, and continues until it reaches null. This is called the prototype chain.

Q49. What is a class in JavaScript?

Also asked as:

- How do you implement OOP in JavaScript?
- Does JavaScript support classes?

Answer:

A class is a cleaner syntax for creating objects and implementing object-oriented concepts such as constructors, methods and inheritance. Internally, JavaScript classes still use the prototype-based inheritance system.

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    return `Hello ${this.name}`;  
  }  
}
```

Follow-up: Is JavaScript class-based or prototype-based?

JavaScript is fundamentally prototype-based. Classes provide a convenient syntax over the prototype system.

Q50. What is the `new` keyword?

Answer:

The new keyword creates a new object from a constructor or class, sets up its prototype, binds this to the new object and returns the object.

```
const user = new User("Pranjal");
```

Part 2 ka core revision

Interview se pehle ye differences **without hesitation** bol paane chahiye:

- var vs let vs const
- == vs ===
- null vs undefined
- primitive vs reference
- normal function vs arrow function
- call() vs apply() vs bind()
- map() vs forEach()
- map() vs filter()
- find() vs filter()
- slice() vs splice()
- spread vs rest
- shallow vs deep copy
- ?? vs ||
- class vs prototype

Next JavaScript block:

 **Promises → async/await → callbacks → Event Loop → Call Stack → Microtask Queue → Macrotask Queue → setTimeout → Promise execution order → error handling** — ye MERN interviews ka bahut high-value portion hai.

Ok next

JavaScript Interview — Part 3

Asynchronous JavaScript → Promises → async/await → Event Loop

Ye section **bahut important** hai, especially MERN interviews mein, kyunki isi se interviewer check karta hai ki tumhe JavaScript actually samajh aati hai ya sirf syntax yaad hai.

Q51. What is synchronous JavaScript?

Interviewer is tarah bhi pooch sakta hai:

- What is synchronous execution?
- How does synchronous code execute?
- Does JavaScript execute code line by line?

Answer:

In synchronous JavaScript, code is executed one statement at a time in order. The next statement waits until the current statement finishes.

```
console.log("A");  
console.log("B");  
console.log("C");
```

Output:

```
A  
B  
C
```

Follow-up: What is the disadvantage of synchronous execution?

If a long-running operation blocks the execution, the following code has to wait, which can make the application unresponsive.

Q52. What is asynchronous JavaScript?

Also asked as:

- What is asynchronous programming?
- How does JavaScript handle time-consuming operations?
- Why do we need asynchronous code?

Answer:

Asynchronous JavaScript allows certain operations to start without blocking the execution of the rest of the program. Once the operation completes, its result can be handled later using callbacks, Promises, or async/await.

Examples:

- API calls
 - Timers
 - File operations in Node.js
 - Database operations
-

Q53. Is JavaScript synchronous or asynchronous?

 **Very common trick question.**

Answer:

JavaScript is single-threaded and executes synchronous code sequentially, but it supports asynchronous operations through the runtime environment, event loop, callbacks, Promises and other mechanisms.

Follow-up: So is JavaScript single-threaded?

Yes, JavaScript execution itself uses a single main thread, but the environment such as the browser or Node.js can provide mechanisms for handling asynchronous operations outside the main JavaScript execution flow.

Q54. What is the Call Stack?

Also asked as:

- Where does JavaScript execute functions?
- How does JavaScript keep track of function calls?

Answer:

The Call Stack is a data structure used by JavaScript to keep track of function execution. When a function is called, it is added to the stack, and when it finishes, it is removed.

```
function one() {  
  two();  
}
```

```
function two() {  
  console.log("Hello");  
}
```

```
one();
```

Execution:

```
one()  
↓  
two()  
↓  
console.log()
```

Then functions are removed in reverse order.

Follow-up: What happens if the stack becomes too large?

It can cause a **stack overflow**, commonly due to excessive or infinite recursion.

Q55. What is the Event Loop?

🔥🔥 Extremely important.

Also asked as:

- How does JavaScript handle asynchronous operations?
- How does JavaScript execute callbacks?
- Explain JavaScript's asynchronous behavior.
- How does the event loop work?

Answer:

The Event Loop continuously checks whether the Call Stack is empty and whether there are waiting tasks in the appropriate queues. When the stack is empty, it allows eligible queued callbacks to be moved for execution. This is how JavaScript can handle asynchronous operations without blocking the main execution thread.

Follow-up: What are the main parts involved?

The important parts are the Call Stack, Web APIs or runtime APIs, task queues, microtask queue, and Event Loop.

Q56. What are Web APIs?

Also asked as:

- Who handles setTimeout or fetch?
- Are setTimeout and fetch part of JavaScript?

Answer:

Web APIs are browser-provided features that are available to JavaScript, such as timers, DOM APIs and network APIs. They are provided by the browser environment, not by the core JavaScript language itself.

Examples:

- `setTimeout()`

- DOM APIs
- fetch()
- event handlers

Follow-up: Is setTimeout a JavaScript feature?

The syntax is available to JavaScript code, but the timer functionality is provided by the runtime environment such as the browser or Node.js.

Q57. What is a callback queue?

Also asked as:

- Where do setTimeout callbacks wait?
- What is the task queue?

Answer:

The callback or task queue stores callbacks that are ready to run after their asynchronous operation has completed. The Event Loop moves them to the Call Stack when the stack is available and the scheduling rules allow it.

Q58. What is the Microtask Queue?

🔥 Very important.

Answer:

The Microtask Queue stores high-priority asynchronous callbacks, mainly Promise callbacks such as .then(), .catch(), and .finally(), as well as some other microtasks. After the current synchronous execution finishes, microtasks are processed before the next regular task.

Q59. Microtask Queue vs Callback/Task Queue?

Also asked as:

- Which executes first: Promise or setTimeout?
- Why does Promise execute before setTimeout?

Answer:

Microtasks generally have priority over regular tasks. After the current synchronous code finishes, JavaScript processes the microtask queue before taking the next task from the task queue.

Example:

```
console.log("A");
```

```
setTimeout(() => {  
  console.log("B");  
}, 0);
```

```
Promise.resolve().then(() => {  
  console.log("C");  
});
```

```
console.log("D");
```

Output:

```
A  
D  
C  
B
```

Why?

1. A → synchronous
2. setTimeout → task scheduled
3. Promise .then() → microtask scheduled
4. D → synchronous
5. synchronous code finishes
6. microtask C
7. task B

Q60. What is a Promise?

🔥🔥 **Must know.**

Also asked as:

- What is Promise in JavaScript?
- Why do we use Promises?
- How do Promises handle asynchronous operations?

Answer:

A Promise is an object that represents the eventual completion or failure of an asynchronous operation. It allows us to handle the result using `.then()`, errors using `.catch()`, and cleanup using `.finally()`.

Q61. What are the states of a Promise?

Answer:

A Promise has three states:

1. **Pending** — operation is still in progress.
2. **Fulfilled** — operation completed successfully.
3. **Rejected** — operation failed.

Once a Promise becomes fulfilled or rejected, it is **settled** and cannot change to another state.

Q62. How do you create a Promise?

```
const promise = new Promise((resolve, reject) => {  
  const success = true;  
  
  if (success) {  
    resolve("Success");  
  } else {
```

```
    reject("Something went wrong");  
  }  
});
```

Interview answer:

We create a Promise using the Promise constructor, which receives a function with resolve and reject. We call resolve when the operation succeeds and reject when it fails.

Q63. What are `.then()`, `.catch()`, and `.finally()`?

Answer:

`.then()` handles a successful Promise result, `.catch()` handles rejection or errors, and `.finally()` runs after the Promise settles regardless of whether it succeeded or failed.

```
promise  
  .then(result => console.log(result))  
  .catch(error => console.log(error))  
  .finally(() => console.log("Completed"));
```

Q64. What is Promise chaining?

Also asked as:

- How do you handle multiple asynchronous operations?
- Can one `.then()` return another Promise?

Answer:

Promise chaining means connecting multiple `.then()` calls so that the result of one asynchronous operation can be passed to the next operation.

```
getUser()  
  .then(user => getOrders(user.id))  
  .then(orders => console.log(orders))  
  .catch(error => console.log(error));
```

Follow-up: Why is chaining better than nested callbacks?

It keeps asynchronous code more structured and readable and makes error handling easier.

Q65. What is async/await?

🔥🔥 Extremely common in MERN.

Also asked as:

- How do you handle Promises using async/await?
- Why do developers prefer async/await?
- Is async/await different from Promises?

Answer:

async/await is a cleaner syntax for working with Promises. An async function always returns a Promise, and await pauses that async function until the awaited Promise settles, making asynchronous code easier to read.

```
async function getUser() {  
  const response = await fetch("/api/user");  
  const data = await response.json();  
  
  return data;  
}
```

Q66. Does async function always return a Promise?

🔥 Common follow-up.

Answer:

Yes. An async function always returns a Promise. If we return a normal value, JavaScript automatically wraps it in a fulfilled Promise.

```
async function test() {  
  return 10;  
}
```

This behaves like a Promise that fulfills with 10.

Q67. Can we use await without async?

Answer:

Normally, await is used inside an async function. Modern JavaScript also supports **top-level await** in supported ES module environments.

For a fresher interview, the important rule is:

Use await inside an async function unless the environment supports top-level await.

Q68. What happens when an awaited Promise rejects?

Answer:

If the rejected Promise is not handled, the async function's returned Promise becomes rejected. We commonly handle it using try...catch.

```
async function getData() {  
  try {  
    const data = await fetchData();  
    console.log(data);  
  } catch (error) {  
    console.log(error);  
  }  
}
```

Q69. Promise vs async/await?

🔥 Frequently asked.

Answer:

Promises are the underlying mechanism for handling asynchronous operations, while async/await provides a cleaner syntax for consuming Promises. Both are based on the same Promise system.

Follow-up: Which do you prefer?

I prefer async/await for most application code because it makes asynchronous logic easier to read and allows straightforward error handling with try...catch.

Q70. What is `Promise.all()`?

🔥🔥 Very important for practical interviews.

Also asked as:

- How do you run multiple API calls in parallel?
- How do you wait for multiple Promises?
- What is `Promise.all` used for?

Answer:

`Promise.all()` runs multiple Promises together and returns a single Promise that fulfills when all of them fulfill. If any one of them rejects, `Promise.all()` rejects.

```
const results = await Promise.all([
  getUsers(),
  getProducts(),
  getOrders()
]);
```

Follow-up: Why use `Promise.all`?

When multiple independent operations can run concurrently, `Promise.all()` can avoid waiting for them one by one.

Q71. What is `Promise.allSettled()`?

Answer:

`Promise.allSettled()` waits for all Promises to finish, regardless of whether they fulfill or reject. It returns the status and result of each Promise.

Follow-up: Difference between `all` and `allSettled`?

`Promise.all()` fails as soon as one Promise rejects, while `Promise.allSettled()` waits for every Promise and gives us the result of each one.

Q72. What is `Promise.race()`?

Answer:

`Promise.race()` settles as soon as the first Promise settles, whether it is fulfilled or rejected.

Follow-up: Is the first Promise to finish always successful?

No. The first Promise to **settle** wins, so it can be either fulfilled or rejected.

Q73. What is `Promise.any()`?

Answer:

`Promise.any()` fulfills when the first Promise fulfills. It ignores rejected Promises until a successful one is found. If all Promises reject, it rejects with an `AggregateError`.

Very useful difference:

- `race()` → first **settled**
 - `any()` → first **fulfilled**
 - `all()` → all must fulfill
 - `allSettled()` → wait for all
-

Q74. What is callback hell?

Also asked as:

- What is Pyramid of Doom?
- What problem do nested callbacks create?

Answer:

Callback hell occurs when many asynchronous operations are nested inside callbacks, making the code difficult to read, maintain and handle errors in. Promises and async/await provide cleaner alternatives.

Q75. How do you handle errors in asynchronous JavaScript?

Answer:

With Promises, I use `.catch()`. With `async/await`, I commonly use `try...catch`. I also make sure errors are handled at the appropriate application layer instead of silently ignoring them.

```
async function fetchData() {  
  try {  
    const response = await fetch("/api/data");  
    return await response.json();  
  } catch (error) {  
    console.error(error);  
  }  
}
```

Q76. What is the difference between `throw` and `reject()`?

Answer:

throw immediately throws an exception, while reject() marks a Promise as rejected. Inside an async function, throwing an error also causes the returned Promise to reject.

Q77. What happens if we don't handle a rejected Promise?

Answer:

It can result in an **unhandled Promise rejection**, which can cause warnings or errors depending on the environment. In production applications, rejected Promises should be handled appropriately.

Q78. Does `setTimeout(fn, 0)` execute immediately?

🔥 Classic interview trap.

Answer:

No. `setTimeout(fn, 0)` does not execute immediately. It schedules the callback as a task, and the callback can run only after the current synchronous code and higher-priority microtasks have been processed.

Example:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Output:

```
A
C
B
```

Q79. Predict the output.

```
console.log("1");
```

```
Promise.resolve().then(() => {  
  console.log("2");  
});
```

```
console.log("3");
```

Answer:

```
1  
3  
2
```

Why?

1 and 3 are synchronous. The Promise callback is a microtask, so it runs after the synchronous code finishes.

Q80. Predict the output.

```
console.log("1");
```

```
setTimeout(() => {  
  console.log("2");  
}, 0);
```

```
Promise.resolve().then(() => {  
  console.log("3");  
});
```

```
console.log("4");
```

Answer:

```
1  
4
```

3
2

Reason:

Synchronous code runs first, then microtasks, then the timer task.

Q81. What will be the output?

```
setTimeout(() => console.log("A"), 0);
```

```
Promise.resolve().then(() => console.log("B"));
```

```
console.log("C");
```

Answer:

C
B
A

Interviewer follow-up:

Why does B execute before A even though setTimeout is written first?

Because the Promise callback is a microtask, and microtasks are processed before the next regular task such as the timer callback.

PART 3 — Must-remember differences

Concept	Remember this
Synchronous	Executes sequentially
Asynchronous	Allows work to complete later
Call Stack	Executes JS functions
Event Loop	Coordinates queued work with the stack
Microtask	Promise callbacks etc.
Task	Timers/events etc.

Concept	Remember this
Promise	Represents future result
async	Function returns Promise
await	Waits inside async function
Promise.all()	All must fulfill
Promise.allSettled()	Wait for all
Promise.race()	First settled
Promise.any()	First fulfilled
.then()	Success
.catch()	Failure
.finally()	Always runs after settlement

Interviewer ka favourite chain

Agar tumse **Promise** poocha gaya, mentally ye follow-ups ready rakho:

Promise → states → then/catch/finally → chaining → callback hell → async/await → try/catch → Promise.all → allSettled → race → any → Event Loop → microtask vs task → output questions.

Ye chain strong kar li to asynchronous JavaScript ke **fresher-level interview questions ka major portion** cover ho jata hai.

Next: JavaScript ka Part 4 — DOM, Events, Event Bubbling/Capturing/Delegation, localStorage/sessionStorage/cookies, JSON, Fetch API, modules, error handling, debounce/throttle.

Done

JavaScript Interview — Part 4

DOM → Events → Event Bubbling → Event Delegation →
Storage → JSON → Fetch → Modules → Error Handling →
Debouncing/Throttling

Ab hum JavaScript ke **browser-side + practical interview questions** cover karenge. Ye React ke questions samajhne mein bhi directly help karenge.

Q82. What is the DOM?

Interviewer is tarah bhi pooch sakta hai:

- What is DOM?
- How does JavaScript interact with HTML?
- How can JavaScript manipulate a webpage?
- What is the Document Object Model?

Answer:

DOM stands for Document Object Model. It represents an HTML document as a tree of objects, which allows JavaScript to read, modify, add, or remove elements and their content dynamically.

Follow-up: What is DOM manipulation?

DOM manipulation means using JavaScript to change elements, attributes, styles, text, or structure of a webpage.

Q83. How do you select an element from the DOM?

Answer:

We can select elements using methods such as `getElementById()`, `querySelector()`, and `querySelectorAll()`.

```
document.getElementById("title");
```

```
document.querySelector(".title");
```

```
document.querySelectorAll(".item");
```

Follow-up: `querySelector()` VS `querySelectorAll()`?

`querySelector()` returns the first matching element, while `querySelectorAll()` returns all matching elements as a `NodeList`.

Q84. What is an event in JavaScript?

Also asked as:

- What is event handling?
- How do you handle user actions?
- What are browser events?

Answer:

An event is an action or occurrence detected by the browser, such as a click, key press, form submission, mouse movement, or page load. JavaScript can listen for these events and execute a function in response.

Q85. What is `addEventListener()`?

Answer:

`addEventListener()` attaches an event handler to an element without directly replacing existing handlers. It allows us to specify the event type and the function that should execute.

```
button.addEventListener("click", () => {  
  console.log("Clicked");  
});
```

Follow-up: Why prefer addEventListener?

It separates JavaScript behavior from HTML and allows multiple handlers for the same event.

Q86. What is event bubbling?

🔥 **Very important.**

Also asked as:

- Explain event propagation.
- What happens when a child element is clicked?
- What is bubbling phase?

Answer:

Event bubbling means that an event starts from the target element and then propagates upward through its parent elements.

Example:

```
<div id="parent">  
  <button id="child">Click</button>  
</div>
```

If the button is clicked, the event can propagate:

button → div → body → document

Q87. What is event capturing?

Also asked as:

- What is capture phase?
- Bubbling vs capturing?

Answer:

Event capturing is the opposite direction of event propagation. The event travels from the outermost element toward the target element before reaching the target.

The general phases are:

Capturing → Target → Bubbling

Q88. Event bubbling vs event capturing?

Answer:

Capturing moves from the parent toward the target, while bubbling moves from the target toward its parents.

Capturing:

document → parent → child

Bubbling:

child → parent → document

Q89. What is event delegation?

 **Frequently asked.**

Also asked as:

- How can you handle events for many child elements efficiently?
- Why use event delegation?
- How does event delegation work?

Answer:

Event delegation is a technique where we attach one event listener to a parent element instead of adding separate listeners to each child. Because of event bubbling, the parent can handle events coming from its children.

Example:

```
const list = document.querySelector("#list");
```

```
list.addEventListener("click", (event) => {  
  if (event.target.matches("li")) {  
    console.log(event.target.textContent);  
  }  
});
```

Follow-up: Why is event delegation useful?

It reduces the number of event listeners and works well when child elements are dynamically added or removed.

Q90. What is `event.target`?

Answer:

`event.target` refers to the element on which the event originally occurred.

```
parent.addEventListener("click", (event) => {  
  console.log(event.target);  
});
```

Follow-up: `target` VS `currentTarget`?

`target` is the original element that triggered the event, while `currentTarget` is the element whose event listener is currently executing.

This is especially important in event delegation.

Q91. What is `preventDefault()`?

Also asked as:

- How do you prevent default browser behavior?
- How do you stop form submission?

Answer:

preventDefault() prevents the browser's default behavior for an event.

For example:

```
form.addEventListener("submit", (event) => {  
  event.preventDefault();  
});
```

This prevents the browser from performing the normal form submission.

Q92. What is stopPropagation?

Answer:

stopPropagation() prevents the event from continuing to propagate through the capturing or bubbling phases.

Follow-up: Difference between preventDefault and stopPropagation?

preventDefault() stops the browser's default action, while stopPropagation() stops the event from propagating to other elements.

Q93. What is localStorage?

Also asked as:

- How do you store data in the browser?
- Where can you store data permanently on the client side?

Answer:

localStorage is a browser storage mechanism that stores data as key-value pairs. The data remains available even after the browser is closed, until it is manually removed or cleared.

```
localStorage.setItem("name", "Pranjal");
```

```
const name = localStorage.getItem("name");
```

```
localStorage.removeItem("name");
```

Q94. What is sessionStorage?

Answer:

sessionStorage also stores key-value data in the browser, but the data is associated with the current browser tab/session and is generally cleared when that tab is closed.

Q95. localStorage vs sessionStorage?

Answer:

localStorage persists until it is explicitly cleared, while **sessionStorage** is tied to the current browser tab/session and is generally cleared when the tab is closed.

localStorage	sessionStorage
Persists longer	Tab/session based
Shared by same-origin tabs according to storage semantics	Separate per tab
Manually cleared	Usually cleared when tab closes

Q96. Can localStorage store objects directly?

Answer:

No. Web Storage stores strings, so objects are usually converted using `JSON.stringify()` and converted back using `JSON.parse()`.

```
const user = {  
  name: "Pranjal",  
  age: 21  
};
```

```
localStorage.setItem("user", JSON.stringify(user));
```

```
const data = JSON.parse(localStorage.getItem("user"));
```

Q97. What are cookies?

Also asked as:

- What is a cookie?
- How are cookies different from localStorage?
- Where is authentication data commonly stored?

Answer:

Cookies are small pieces of data associated with a website that can be sent with HTTP requests. They are commonly used for session management and authentication. They can also have security attributes such as HttpOnly, Secure, and SameSite.

Q98. localStorage vs cookies?

 Important for authentication interviews.

Answer:

localStorage is client-side storage and is not automatically sent with HTTP requests. Cookies can be automatically included in matching HTTP requests and can be configured with security attributes such as HttpOnly and Secure.

Follow-up: What is HttpOnly?

An HttpOnly cookie cannot be accessed through JavaScript, which helps reduce the risk of client-side JavaScript stealing the cookie through an XSS vulnerability.

Q99. What is JSON?

Also asked as:

- What is JSON used for?
- Why is JSON commonly used in APIs?

Answer:

JSON stands for JavaScript Object Notation. It is a lightweight text format commonly used to exchange structured data between a client and a server.

Example:

```
{  
  "name": "Pranjal",  
  "age": 21  
}
```

Q100. What is JSON.stringify()?

Answer:

JSON.stringify() converts a JavaScript value, usually an object or array, into a JSON string.

```
const user = { name: "Pranjal" };  
  
const json = JSON.stringify(user);
```

Q101. What is JSON.parse()?

Answer:

JSON.parse() converts a valid JSON string into a JavaScript value, such as an object or array.

```
const json = '{"name":"Pranjal"}';
```

```
const user = JSON.parse(json);
```

Follow-up: What happens if invalid JSON is passed?

JSON.parse() throws a SyntaxError, so it should be handled when the input may be invalid.

Q102. What is the Fetch API?

🔥 Very important for MERN.

Also asked as:

- How do you call an API using JavaScript?
- How do you make HTTP requests?
- How do you fetch data from a backend?

Answer:

The Fetch API is a browser API used to make HTTP requests and work with responses asynchronously. It returns a Promise.

```
const response = await fetch("/api/users");  
const data = await response.json();
```

Q103. What is Axios?

Also asked as:

- Fetch vs Axios?
- Why use Axios instead of fetch?

Answer:

Axios is a Promise-based HTTP client commonly used for making API requests. It provides features such as request/response interceptors, convenient JSON handling, and centralized configuration.

Q104. Fetch vs Axios?

Answer:

Both can make HTTP requests. Fetch is a built-in Web API, while Axios is a separate library. Axios provides convenient features such as interceptors and automatic JSON transformation, while Fetch does not require installing an additional package.

Follow-up: Does fetch reject the Promise for HTTP 404 or 500?

🔥 Trick question.

No. Fetch normally rejects only when the request itself fails at the network level. HTTP error responses such as 404 or 500 still produce a Response object, so we should check response.ok or response.status ourselves.

```
const response = await fetch("/api/users");
```

```
if (!response.ok) {  
  throw new Error("Request failed");  
}
```

Q105. What is an HTTP request?

Answer:

An HTTP request is a message sent by a client to a server to perform an operation or retrieve data. It commonly contains a method, URL, headers, and sometimes a request body.

Q106. What are HTTP methods commonly used in REST APIs?

Answer:

The main methods are:

- **GET** — retrieve data
 - **POST** — create data
 - **PUT** — replace/update a resource
 - **PATCH** — partially update a resource
 - **DELETE** — delete data
-

Q107. What is the difference between PUT and PATCH?

🔥 Common.

Answer:

PUT is generally used to replace the complete representation of a resource, while PATCH is used for a partial update.

Example:

PUT /users/10

could send the complete user object.

PATCH /users/10

could send only:

```
{  
  "name": "Pranjal"  
}
```

Q108. What is an HTTP status code?

Answer:

An HTTP status code tells the client the result of an HTTP request.

Important ones:

200 → OK

201 → Created

204 → No Content
400 → Bad Request
401 → Unauthorized
403 → Forbidden
404 → Not Found
409 → Conflict
500 → Internal Server Error

Follow-up: 401 vs 403?

401 means the request lacks valid authentication credentials. 403 means the server understands the request but refuses to authorize it.

Q109. What are JavaScript modules?

Also asked as:

- What is modular JavaScript?
- Why do we use import/export?
- How do you share code between files?

Answer:

Modules allow us to split JavaScript code into separate files and explicitly share functionality using export and import. This makes applications easier to organize and maintain.

```
// math.js  
export const add = (a, b) => a + b;
```

```
// app.js  
import { add } from "./math.js";
```

Q110. Named export vs default export?

Answer:

A named export allows multiple specific exports from a module and must be imported using the exported name. A default export

represents the module's main export and can be imported with any local name.

```
export const add = () => {};  
export default User;
```

Q111. CommonJS vs ES Modules?

Very relevant for Node/MERN.

Answer:

CommonJS commonly uses `require()` and `module.exports`, while ES Modules use `import` and `export`. Modern Node.js supports ES Modules when the project is configured appropriately.

```
// CommonJS  
const express = require("express");
```

```
// ES Module  
import express from "express";
```

Q112. What is error handling in JavaScript?

Also asked as:

- How do you handle runtime errors?
- How do you prevent application crashes due to errors?

Answer:

JavaScript provides mechanisms such as `try...catch`, `finally`, and `throw` for handling errors. In asynchronous code, we can also handle rejected Promises using `.catch()` or `try...catch` with `await`.

Q113. What is try...catch?

Answer:

try...catch allows us to execute code that may throw an error and handle that error without stopping the entire flow unexpectedly.

```
try {  
  const data = JSON.parse("invalid");  
} catch (error) {  
  console.log("Invalid JSON");  
}
```

Follow-up: What is finally?

finally executes after the try and catch blocks regardless of whether an error occurred.

Q114. What is the `throw` keyword?

Answer:

throw is used to manually create and raise an exception.

```
if (!user) {  
  throw new Error("User not found");  
}
```

Q115. What is debouncing?

🔥🔥 **Very common practical question.**

Also asked as:

- How do you optimize a search input?
- How can you avoid calling an API on every keystroke?
- What is debounce?

Answer:

Debouncing delays function execution until the user stops triggering the event for a specified period. It is commonly used for search inputs so that an API request is made only after the user stops typing.

Example:

User types: H

He

Hel

Hell

Hello

API call → only after the user stops typing

Follow-up: Where would you use debounce?

Search boxes, autocomplete, validation and other events where frequent triggers should be reduced.

Q116. What is throttling?

Also asked as:

- Debounce vs throttle?
- How do you optimize scroll events?

Answer:

Throttling limits how frequently a function can execute during a continuous stream of events. For example, a scroll handler may be allowed to execute at most once every specified interval.

Q117. Debouncing vs throttling?

🔥 Must know.

Answer:

Debouncing waits until the event stops occurring, while throttling allows the function to execute at a controlled maximum frequency during continuous events.

Easy example:

Search box → Debounce

PART 4 — Interview traps you should remember

1. fetch() + 404

Fetch does **not automatically reject** for HTTP 404/500.

2. localStorage

Stores **strings**, so objects need JSON.stringify().

3. Arrow function

Does **not have its own this**.

4. Event delegation

Uses **event bubbling**.

5. target VS currentTarget

target = original event element

currentTarget = element whose listener is executing

6. preventDefault() VS stopPropagation()

preventDefault() = stops browser's default action

stopPropagation() = stops event propagation

7. Debounce vs throttle

Debounce = wait for silence

Throttle = limit frequency

Next JavaScript block

Ab JavaScript ke **last major interview blocks** bach rahe hain:

Prototypes → inheritance → OOP → new → generators/iterators (sirf fresher-relevant) → Symbols → Set/Map → WeakMap/WeakSet → garbage

collection/memory leaks → optional chaining/nullish coalescing ke deeper questions → JavaScript coding questions + output-based questions.

Uske baad JavaScript ka complete revision + rapid-fire interview round, aur phir React A→Z start karenge.

Next

JavaScript Interview — Part 5

Prototypes → Inheritance → OOP → Set/Map → Iterators → Generators → Memory

Ab hum JS ke **advanced-but-fresher-relevant** topics cover karenge. Niche 10-year-experience wali cheezein nahi la raha.

Q118. What is prototype-based inheritance?

Also asked as:

- How does inheritance work in JavaScript?
- Explain inheritance in JavaScript.
- What is prototype inheritance?

Answer:

JavaScript uses prototype-based inheritance. An object can inherit properties and methods from another object through its prototype. If a property is not found on the object itself, JavaScript looks for it in the prototype chain.

Follow-up: What is the prototype chain?

The prototype chain is the sequence of objects JavaScript checks when looking for a property or method. It continues until it reaches null.

Q119. What is `prototype`?

Also asked as:

- What is a prototype property?
- What is `Function.prototype`?
- Where are shared methods stored?

Answer:

prototype is a property of constructor functions and classes that is used as the prototype for objects created from them. Methods placed on the prototype can be shared by all instances instead of creating a separate copy for each object.

```
function User(name) {  
  this.name = name;  
}
```

```
User.prototype.greet = function () {  
  return `Hello ${this.name}`;  
};
```

```
const user1 = new User("Pranjal");
```

Q120. What is the difference between `__proto__` and `prototype`?

🔥 Very common tricky question.

Answer:

prototype is a property mainly found on constructor functions and is used when creating objects. `__proto__` is an accessor that refers to an object's prototype.

Easy way to remember:

Function → prototype

Object → its prototype can be accessed via `__proto__`

Follow-up: Should we use `__proto__` in normal code?

Generally, no. It is better to use standard APIs such as `Object.getPrototypeOf()` and `Object.setPrototypeOf()` when prototype manipulation is actually needed.

Q121. Why do we put methods on the prototype?

Answer:

Because prototype methods are shared among instances. This avoids creating a separate function for every object and can save memory.

```
function User(name) {  
  this.name = name;  
}
```

```
User.prototype.greet = function () {  
  console.log("Hello");  
};
```

Multiple User objects can use the same greet function.

Q122. What is inheritance?

Also asked as:

- What is the purpose of inheritance?
- How can one class inherit another class?

Answer:

Inheritance allows one class or object to reuse properties and methods from another. It helps reduce code duplication and represents an “is-a” relationship.

```
class User {  
  login() {  
    console.log("Login");  
  }  
}
```

```
class Admin extends User {  
  deleteUser() {  
    console.log("Delete");  
  }  
}
```

Q123. What does `extends` do?

Answer:

`extends` creates a subclass that inherits from another class.

```
class Admin extends User {}
```

Here, Admin inherits accessible methods from User.

Q124. What does `super` do?

Also asked as:

- How do you call the parent constructor?
- How do you access parent methods?

Answer:

`super` is used inside a derived class to call the parent class constructor or access methods of the parent class.

```
class Admin extends User {  
  constructor(name, role) {  
    super(name);  
    this.role = role;  
  }  
}
```

Follow-up: Why must `super()` be called before using `this` in a derived constructor?

Because the parent constructor must initialize the derived instance before the subclass can use `this`.

Q125. What are the main OOP concepts?

🔥 **Common fresher question.**

Answer:

The four main OOP concepts are:

Encapsulation, Abstraction, Inheritance, and Polymorphism.

Follow-up: Explain them briefly.

Encapsulation:

Bundling data and related methods together and controlling access to internal details.

Abstraction:

Showing only the necessary details and hiding unnecessary implementation details.

Inheritance:

Reusing properties and methods from another class or object.

Polymorphism:

The same interface or method name can behave differently depending on the object or context.

Q126. What is encapsulation in JavaScript?

Answer:

Encapsulation means keeping related data and behavior together and controlling how internal data is accessed or modified. JavaScript supports this using classes, closures and private class fields.

Example:

```
class BankAccount {  
  #balance = 0;  
  
  deposit(amount) {  
    this.#balance += amount;  
  }  
}
```

Here #balance cannot be directly accessed from outside the class.

Q127. What are private class fields?

Also asked as:

- How do you create private properties in JavaScript?
- What does # mean in a class?

Answer:

A property starting with # is a private class field. It can only be accessed from within the class that declares it.

```
class User {  
  #password;  
  
  constructor(password) {  
    this.#password = password;  
  }  
}
```

Q128. What is polymorphism in JavaScript?

Answer:

Polymorphism means that the same method or interface can produce different behavior depending on the object using it.

```
class Animal {  
  sound() {  
    console.log("Animal sound");  
  }  
}
```

```
}  
}  
  
class Dog extends Animal {  
  sound() {  
    console.log("Bark");  
  }  
}
```

Q129. What is a Set?

Also asked as:

- How do you remove duplicate values from an array?
- Set vs Array?
- What is Set used for?

Answer:

A Set is a collection that stores unique values. It is useful when we need to remove duplicates or efficiently check whether a value exists.

```
const numbers = [1, 2, 2, 3, 3];  
  
const unique = [...new Set(numbers)];  
  
console.log(unique);  
// [1, 2, 3]
```

Follow-up: Can a Set contain objects?

Yes. Objects can be stored in a Set, but uniqueness for objects is based on reference identity.

Q130. What is a Map?

Also asked as:

- Map vs Object?

- When would you use Map?
- What is JavaScript Map?

Answer:

A Map is a collection of key-value pairs where keys can be of any data type, including objects, functions and primitive values.

```
const map = new Map();
```

```
map.set("name", "Pranjal");
```

```
map.set(1, "One");
```

```
console.log(map.get("name"));
```

Q131. Map vs Object?

 **Important.**

Answer:

Objects are commonly used for structured records with named properties, while Map is designed specifically for key-value collections. A Map can use any value as a key and provides methods such as set(), get(), has(), and delete().

Follow-up: When would you choose Map?

I would choose Map when I need dynamic key-value data, especially when keys are not limited to strings or when Map's collection methods are useful.

Q132. Set vs Map?

Answer:

Set stores unique values, while Map stores key-value pairs.

Set → value

Map → key + value

Q133. What is an iterator?

Also asked as:

- What is an iterable?
- How does for...of work?
- What is the iterator protocol?

Answer:

An iterator is an object that provides a `next()` method to retrieve values one at a time. The `next()` method returns an object containing value and done.

```
const iterator = [10, 20][Symbol.iterator]();  
  
console.log(iterator.next());
```

Q134. What is an iterable?

Answer:

An iterable is an object that can provide an iterator through `Symbol.iterator`. Arrays, strings, Maps and Sets are common examples of iterables.

Follow-up: Is a normal object iterable?

Not by default.

```
for (const value of { a: 1 }) {  
  // TypeError  
}
```

But we can make objects iterable by implementing `Symbol.iterator`.

Q135. `for...in` VS `for...of`?

🔥 Frequently asked.

Answer:

`for...in` iterates over enumerable property keys, while `for...of` iterates over values of an iterable.

```
const arr = ["a", "b"];

for (const index in arr) {
  console.log(index);
}

for (const value of arr) {
  console.log(value);
}
```

Output conceptually:

```
for...in → 0, 1
for...of → a, b
```

Follow-up: Which one should you generally use for arrays?

`for...of` is generally more appropriate when we want the array values.

Q136. What is a generator function?

Also asked as:

- What is `function*`?
- What does `yield` do?
- What are generators?

Answer:

A generator is a special function that can pause its execution and resume it later. It is declared using `function*` and uses `yield` to produce values.


```
function* numbers() {  
  yield 1;  
  yield 2;  
  yield 3;  
}
```

```
const gen = numbers();
```

```
console.log(gen.next());  
console.log(gen.next());
```

Q137. What is `yield`?

Answer:

yield pauses a generator function and returns a value. When `next()` is called again, execution resumes from where it paused.

Fresher-level importance:

You should know the concept, but generators are *not among the highest-priority topics for a typical MERN fresher interview.*

Q138. What is Symbol in JavaScript?

Also asked as:

- What is the Symbol data type?
- Why are Symbols unique?
- Where is Symbol used?

Answer:

Symbol is a primitive data type that creates unique values. It is commonly used as unique object property keys and is also used by JavaScript for built-in protocols such as `Symbol.iterator`.

```
const id1 = Symbol("id");  
const id2 = Symbol("id");
```

```
console.log(id1 === id2); // false
```

Q139. What is `Symbol.iterator`?

Answer:

`Symbol.iterator` is a well-known `Symbol` that defines how an object can be iterated. Objects implementing it can be used with constructs such as `for...of`.

Q140. What is garbage collection?

Also asked as:

- How does JavaScript manage memory?
- Does JavaScript automatically free memory?

Answer:

JavaScript automatically manages memory using garbage collection. When an object is no longer reachable from the program, the JavaScript engine can identify it as unused and reclaim its memory.

Q141. What is a memory leak?

 **Practical interview question.**

Answer:

A memory leak occurs when an application unintentionally keeps references to objects that are no longer needed, preventing the garbage collector from reclaiming their memory.

Common causes:

- Unremoved event listeners
- Unnecessary timers

- Global references
 - Large objects kept unnecessarily
 - Improperly managed subscriptions
-

Q142. How can you prevent memory leaks in JavaScript?

Answer:

We should remove unnecessary event listeners, clear timers, release unused references and properly clean up subscriptions or resources when they are no longer needed.

Q143. What is optional chaining?

We've technically covered it, but interviewers can ask it differently.

Answer:

Optional chaining ?. safely accesses nested properties without throwing an error when an intermediate value is null or undefined.

```
user?.address?.city
```

Q144. What is nullish coalescing?

Answer:

The ?? operator provides a fallback only when the left side is null or undefined.

```
const username = user.name ?? "Guest";
```

Q145. What is the difference between `||` and `??`?

🔥 Very common follow-up.

```
0 || 10  
// 10
```

```
0 ?? 10  
// 0
```

Answer:

`||` considers all falsy values as false, while `??` only considers null and undefined as missing values.

Q146. What is short-circuit evaluation?

Also asked as:

- How does `&&` work in JavaScript?
- How can logical operators be used for conditional execution?

Answer:

Short-circuit evaluation means JavaScript stops evaluating a logical expression as soon as the final result is already known.

```
isLoggedIn && showDashboard();
```

If `isLoggedIn` is falsy, `showDashboard()` isn't evaluated.

Q147. What is optional chaining with function calls?

Answer:

We can use `?()` to call a function only if it exists.

```
user.greet?.();
```

If greet is undefined or null, it won't throw an error.

Q148. What is the difference between `delete` and setting a property to `undefined`?

Answer:

`delete` removes the property from the object, while assigning `undefined` keeps the property but changes its value.

```
const user = {  
  name: "A",  
  age: 20  
};
```

```
delete user.age;
```

Now age is no longer an own property of user.

Q149. What is `hasOwnProperty()`?

Also asked as:

- How do you check whether an object directly contains a property?
- How do you distinguish own properties from inherited properties?

Answer:

`hasOwnProperty()` checks whether a property belongs directly to an object rather than being inherited through its prototype chain.

```
user.hasOwnProperty("name");
```

Modern alternative:

```
Object.hasOwn(user, "name");
```

Q150. What is `Object.create()`?

Answer:

`Object.create()` creates a new object with the specified object as its prototype.

```
const person = {  
  greet() {  
    console.log("Hello");  
  }  
};
```

```
const user = Object.create(person);
```

```
user.greet();
```

Here user can access `greet()` through its prototype.

PART 5 — What is actually HIGH priority?

Don't give equal importance to everything.

MUST KNOW

These should be **instant answers**:

- Prototype
- Prototype chain
- `this`
- Arrow function `this`
- `call`/`apply`/`bind`
- Closure
- `map`/`filter`/`reduce`
- Spread/rest
- Shallow/deep copy
- Promise
- `async`/`await`
- Event Loop

- Microtask vs task
- Promise.all
- Event bubbling
- Event delegation
- localStorage/sessionStorage
- Fetch
- Error handling
- Debounce/throttle
- for...in vs for...of
- Set
- Map
- class / inheritance

Know the concept

- Generator
- Iterator
- Symbol
- Garbage collection
- Memory leaks
- Object.create
- Private fields

Don't spend excessive time memorizing

You don't need to become a JavaScript-engine expert for a **MERN fresher interview**.

Next: JavaScript Coding + Output Round

Ab theory ka major portion almost complete ho gaya. Next hum actual **interviewer mode mein JavaScript coding questions karenge**:

- Reverse string
- Palindrome
- Anagram
- Character frequency
- Remove duplicates

- Largest/second largest
- Missing number
- Two Sum
- Flatten array
- Sort without built-in
- Count vowels
- First non-repeating character
- Array/object manipulation
- map/filter/reduce problems
- Promise/output questions
- this output questions
- Closure output questions
- Hoisting output questions
- Event-loop output questions

Aur **har coding question ke alternate interview wordings + follow-up questions + interview-level answer** bhi rahenge.

Hogya bus ab practical questions mat dena uski ek alag series krenge ab react shuru kro btao kitne questions hai

Haan ❤ JavaScript ki **theory/interview-question series** yahin **complete** maan lo. Practical/coding questions ko bilkul separate series rakhenge.

Ab **React.js** start karte hain.

React Fresher Interview — Approx. 100–120 Questions

Main isko unnecessarily inflate nahi karunga. **Freshers ke MERN interviews mein genuinely relevant A→Z questions** cover karenge.

React ka roadmap:

Part 1 — React Fundamentals (~20)

- What is React?
- Why React?
- Features
- SPA

- Library vs framework
- Components
- Functional components
- JSX
- JSX vs HTML
- Props
- State
- Props vs state
- key
- Virtual DOM
- Real DOM
- Reconciliation
- Rendering
- Conditional rendering
- Lists
- Fragments

Part 2 — State & Hooks (~25)

- useState
- State updates
- Functional state updates
- Batching
- useEffect
- Dependency array
- Cleanup
- useRef
- useMemo
- useCallback
- useContext
- Custom hooks
- Rules of Hooks
- Hook-related follow-ups

Part 3 — Component Communication (~15)

- Parent → child
- Child → parent

- Prop drilling
- Lifting state up
- Composition
- Controlled components
- Uncontrolled components
- Forms
- Reusable components

Part 4 — React Advanced (~20)

- Re-rendering
- Reconciliation
- React Fiber
- Virtual DOM deeper questions
- memo
- Performance optimization
- Lazy loading
- Suspense
- Code splitting
- Error boundaries
- StrictMode
- Portal
- Higher-order components
- Render props

Part 5 — Routing + API (~15)

- React Router
- Routes
- Link
- useNavigate
- useParams
- useLocation
- Protected routes
- API calls
- Axios
- Loading/error states
- CRUD

- Authentication flow
- JWT handling from React side

Part 6 — React + MERN Practical Concepts (~15–20)

- Login/register flow
- Token handling
- Context for auth
- Global state
- Backend integration
- CORS
- Environment variables
- .env
- API interceptors
- Logout
- Protected frontend routes
- Role-based UI
- Deployment basics

Total

~110–120 carefully selected questions, including their follow-up questions and alternate interview wording.

Aur exactly JavaScript wali approach rahegi:

**Question → interviewer ka alternate wording → perfect easy-English
answer → follow-ups → follow-up answers**

Practical/coding questions nahi milayenge. Woh baad mein separate **JavaScript Practical Series** mein karenge.

React mein sabse pehle **Part 1: React Fundamentals** se start karenge.

Start



React Interview — Part 1: Fundamentals

Ab **React ke fresher-level A→Z interview questions** start karte hain.

Same rule: **easy English, interview-level answer, alternate wording + follow-ups**. Faltu advanced questions nahi.

Q1. What is React?

Interviewer is tarah bhi pooch sakta hai:

- What do you know about React?
- Why do you use React?
- What is React.js?
- Explain React in simple words.

Answer:

React is a JavaScript library for building user interfaces, especially interactive and component-based web applications. It was developed by Facebook, now Meta. React allows us to build reusable components and efficiently update the UI when data changes.

Follow-up: Is React a library or a framework?

React is a JavaScript library, mainly focused on building user interfaces. We can combine it with other libraries and tools to build a complete application.

Follow-up: Why do you use React?

I use React because it provides reusable components, efficient UI updates, a strong ecosystem, and makes large user interfaces easier to develop and maintain.

Q2. Why is React popular?

Also asked as:

- Why do companies use React?

- What are the advantages of React?
- Why did you choose React?

Answer:

React is popular because it supports reusable components, efficient UI updates, one-way data flow, a large ecosystem, and good developer support. It also makes it easier to build and maintain complex user interfaces.

Follow-up: What is the biggest advantage of React according to you?

Component reusability is one of the biggest advantages because we can create a component once and reuse it in multiple places.

Q3. What are the main features of React?

Answer:

The main features of React are **component-based architecture, JSX, virtual DOM, one-way data flow, reusable components, hooks, and declarative UI development.**

Follow-up: What does component-based architecture mean?

It means an application is divided into small, independent and reusable components, where each component manages a specific part of the UI.

Q4. What is a component in React?

 **Very important.**

Also asked as:

- What are React components?
- Why do we use components?
- Explain component-based architecture.
- What is a reusable component?

Answer:

A component is a reusable piece of UI that contains its own structure and behavior. We can combine multiple components to build a complete React application.

Example:

```
function Button() {  
  return <button>Login</button>;  
}
```

Follow-up: Why are components useful?

They improve **reusability, maintainability and organization** of the application.

Q5. What are the types of components in React?

Also asked as:

- Functional vs class components?
- How can you create components in React?

Answer:

React applications can use **functional components and class components**. Today, functional components are preferred because Hooks allow them to use state and other React features without needing classes.

Follow-up: Which one do you use?

I primarily use **functional components with Hooks** because they are simpler and are the modern approach to React development.

Q6. What is a functional component?

Answer:

A functional component is a JavaScript function that returns JSX representing the UI. It can use React Hooks to manage state, side effects and other React features.

```
function Welcome() {  
  return <h1>Hello</h1>;  
}
```

Q7. What is JSX?

 **Must know.**

Also asked as:

- What does JSX stand for?
- Why do we use JSX?
- Can React work without JSX?
- Is JSX HTML?

Answer:

JSX stands for JavaScript XML. It is a syntax extension for JavaScript that allows us to write HTML-like markup inside JavaScript. It makes React UI code easier to read and write.

```
const element = <h1>Hello World</h1>;
```

Follow-up: Is JSX HTML?

No. JSX looks similar to HTML, but it is not HTML. It is syntax that gets transformed into JavaScript expressions representing React elements.

Follow-up: Can React work without JSX?

Yes. JSX is optional. React can be written using JavaScript APIs such as createElement, but JSX is much more readable and convenient.

Q8. What is the difference between JSX and HTML?

🔥 Common.

Answer:

JSX looks similar to HTML, but there are some differences. JSX is written inside JavaScript and follows JavaScript-based rules. For example, we use `className` instead of `class`, `htmlFor` instead of `for`, and JavaScript expressions can be written inside `{}`.

```
<div className="container">
  {name}
</div>
```

Follow-up: Why do we use `className`?

Because `class` is a JavaScript keyword, React uses `className` for the CSS class attribute in JSX.

Q9. What are JavaScript expressions in JSX?

Answer:

We can use JavaScript expressions inside JSX using curly braces `{}`.

```
const name = "Pranjal";

return <h1>Hello {name}</h1>;
```

Follow-up: Can we write statements directly inside `{}`?

No. JSX expressions can contain expressions, but not statements such as `if` or `for` directly.

For conditions, we commonly use:

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

Q10. What is the Virtual DOM?

🔥🔥 Very important.

Also asked as:

- What is Virtual DOM?
- How does React update the UI?
- Why is React fast?
- Virtual DOM vs Real DOM?

Answer:

The Virtual DOM is an in-memory representation of the UI. When state or props change, React creates or updates the relevant virtual representation, compares it with the previous one, and then updates the necessary parts of the real DOM. This process helps React update the UI efficiently.

Follow-up: Is Virtual DOM faster than Real DOM?

The Virtual DOM itself is not simply “faster than the Real DOM.” Its benefit is that React can efficiently determine what DOM changes are needed instead of manually updating the entire UI.

Q11. What is the Real DOM?

Answer:

The Real DOM is the actual tree of elements maintained by the browser for the webpage. Changes to it can involve browser work such as layout and painting, so React tries to minimize unnecessary DOM updates.

Q12. Virtual DOM vs Real DOM?

Answer:

The Real DOM represents the actual webpage, while the Virtual DOM is an in-memory representation used by React to determine efficient updates. React compares the previous and next UI representations and applies the necessary changes to the Real DOM.

Q13. What is reconciliation in React?

🔥 Important follow-up to Virtual DOM.

Also asked as:

- How does React update the DOM?
- How does React know what to update?
- What happens when state changes?

Answer:

Reconciliation is the process React uses to compare the previous rendered UI with the new rendered UI and determine what changes need to be applied to the actual DOM.

Follow-up: What helps React identify list items?

Keys help React identify elements in lists and determine which items were added, removed or changed.

Q14. What is declarative UI?

Also asked as:

- What does declarative programming mean in React?
- Is React declarative or imperative?

Answer:

React is declarative. Instead of manually telling the browser how to update every DOM element, we describe what the UI should look like for a particular state, and React handles the necessary updates.

Easy example:

If `isLoggedIn` is true, I describe that the dashboard should be displayed.
React handles updating the UI when the state changes.

Q15. What is one-way data flow in React?

🔥 **Very common.**

Also asked as:

- How does data flow in React?
- Does data flow from child to parent?
- Explain unidirectional data flow.

Answer:

React follows one-way data flow, meaning data generally flows from parent components to child components through props. This makes the application easier to understand and debug.

Parent
↓ props
Child

Follow-up: Can a child send data to a parent?

Yes, but not by directly modifying the parent's state. The parent can pass a callback function as a prop, and the child can call that function to send information back.

Q16. What are props?

🔥🔥 **Must know.**

Also asked as:

- What is props in React?
- How do components communicate?
- How do you pass data from parent to child?

Answer:

Props, short for properties, are read-only values passed from a parent component to a child component. They are used to pass data or functions between components.

```
function User({ name }) {  
  return <h1>{name}</h1>;  
}
```

```
<User name="Pranjal" />
```

Follow-up: Can a child modify props?

No. Props are read-only from the child's perspective. If the data needs to change, the parent should update its state and pass the new value down.

Q17. Props vs State?

🔥🔥 Extremely important.

Answer:

Props are data passed from a parent to a child and are read-only for the receiving component. State is data managed by a component that can change over time and cause the component to re-render.

Props	State
Passed from parent	Managed by component
Read-only	Can be updated
Used for communication	Used for dynamic data
Changes come from parent	Updated through state mechanisms

Follow-up: Can props cause re-render?

Yes. When a component receives new props, React may re-render that component and its relevant descendants.

Q18. What is state in React?

 **Must know.**

Answer:

State is data managed by a React component that can change over time. When state changes through the appropriate React mechanism, React schedules a re-render so the UI can reflect the new state.

Examples:

- Login status
 - Counter value
 - Form input
 - Cart items
 - Loading state
-

Q19. Why should we not directly modify state?

Also asked as:

- Can we change state directly?
- Why should state be treated as immutable?

Answer:

We should not directly mutate React state because React relies on state updates to schedule rendering and determine changes. We should use the provided state update mechanism and create the required new value instead.

 **Avoid:**

```
user.name = "Rahul";
```

Instead, create an updated value through the state setter.

Q20. What is conditional rendering?

🔥 Common.

Also asked as:

- How do you conditionally display components?
- How do you show different UI based on state?

Answer:

Conditional rendering means displaying different UI based on a condition. React commonly uses JavaScript techniques such as if, ternary operators, and logical && expressions.

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

Q21. How do you render a list in React?

Answer:

We commonly use JavaScript's `map()` method to transform an array into React elements.

```
users.map(user => (  
  <User key={user.id} name={user.name} />  
))
```

Follow-up: Why do we need a key?

Keys help React identify individual elements in a list so it can efficiently determine which items changed, were added, or were removed.

Q22. What is a key in React?

🔥🔥 Very important.

Also asked as:

- Why do we use keys in React?
- Why does React show a key warning?
- Why shouldn't we use array index as key?

Answer:

A key is a unique identifier given to elements in a list. It helps React identify which items have changed, been added, or removed during reconciliation.

```
users.map(user => (  
  <User key={user.id} />  
))
```

Follow-up: Can we use array index as key?

We can, but it is generally not recommended when list items can be reordered, inserted, or deleted, because the index may change and can lead to incorrect component state or inefficient updates.

Follow-up: What should a good key be?

It should be **stable, unique among the siblings, and associated with the item itself**, such as a database ID.

Q23. What are React Fragments?

Also asked as:

- Why do we use <> </>?
- How can you return multiple elements without an extra div?

Answer:

Fragments allow us to group multiple elements without adding an unnecessary DOM element.

```
<>
<h1>Hello</h1>
<p>Welcome</p>
</>
```

Follow-up: Why use Fragment instead of a div?

When we don't need an extra DOM element, a Fragment keeps the DOM structure cleaner.

Q24. What is a Single Page Application (SPA)?

🔥 Important for React.

Also asked as:

- What type of applications can React build?
- What is SPA architecture?
- Why is React suitable for SPAs?

Answer:

A Single Page Application loads the main application shell once and dynamically updates the UI as the user navigates or interacts with the application, instead of performing a full page reload for every route.

Follow-up: Is React itself a routing library?

No. React handles UI development. For routing, we commonly use a routing library such as React Router.

Q25. What is the difference between SPA and traditional multi-page applications?

Answer:

In a traditional multi-page application, navigation generally requests a new HTML page from the server. In an SPA, the

application can update the displayed UI on the client side without requiring a full page reload for every route.

Q26. What is React StrictMode?

Also asked as:

- Why do we use StrictMode?
- Why does my function appear to run twice in development?

Answer:

StrictMode is a development-only feature that helps identify potential problems in a React application. It can intentionally invoke certain functions or lifecycle-related behavior more than once in development to help detect unsafe patterns. It does not mean the production application will behave the same way.

Follow-up: Does StrictMode run twice in production?

No. The extra development checks are not performed in the same way in production builds.

Q27. What is React rendering?

Also asked as:

- What happens when a component re-renders?
- When does React render a component?

Answer:

Rendering in React means React calls the component to determine what UI should be displayed for the current props and state. React then reconciles the result and applies the necessary DOM updates.

Follow-up: Does every render mean the entire DOM is recreated?

No. React reconciles the new result with the previous one and updates only the necessary DOM parts.

Q28. What causes a React component to re-render?

🔥 **Very common follow-up.**

Answer:

A component can re-render when its **state changes, its props change, or its parent re-renders**. Context changes can also cause consumers to re-render.

Follow-up: Does changing a normal JavaScript variable cause a React re-render?

No. Changing a normal variable does not tell React that the UI needs to update. State or other React mechanisms should be used for data that affects the UI.

Q29. What is component reusability?

Answer:

Component reusability means designing a component so it can be used in multiple places with different data or configurations. Props are commonly used to make components reusable.

Example:

```
<Button text="Login" />
<Button text="Register" />
```

Same component, different data.

Q30. What is component composition?

🔥 **Important React concept.**

Also asked as:

- How do you reuse components without inheritance?
- What is composition in React?

Answer:

Component composition means building larger components by combining smaller components. Instead of relying heavily on inheritance, React encourages composition to reuse and organize UI behavior.

Example:

```
<Card>  
  <Profile />  
</Card>
```

Here Card can display different child content.

Part 1 — Most Important Questions

Agar interview se just pehle React revise karna ho, **Q1–Q30 mein ye absolutely strong hone chahiye:**

1. **What is React?**
2. Library vs framework
3. Components
4. Functional components
5. JSX
6. JSX vs HTML
7. Virtual DOM
8. Real DOM
9. Reconciliation
10. Declarative UI
11. One-way data flow
12. Props
13. State
14. Props vs State

- 15. Keys
- 16. Conditional rendering
- 17. Fragments
- 18. SPA
- 19. Re-rendering
- 20. Component composition

Next Part 2: 🔥 React Hooks — useState, state updates, batching, useEffect, dependency array, cleanup, useRef, useMemo, useCallback, useContext, custom hooks + har ek ke important follow-ups.

Next

React Interview — Part 2: Hooks

Ab React ka **sabse important interview section — Hooks**. Freshers ke liye isme se questions bahut commonly aate hain, especially useState, useEffect, useRef, useMemo, useCallback, useContext.

Q31. What are React Hooks?

Also asked as:

- What are Hooks in React?
- Why were Hooks introduced?
- Why do we use Hooks?

Answer:

Hooks are functions provided by React that allow functional components to use features such as state, side effects, context and references. They were introduced in React 16.8.

Follow-up: Can Hooks be used inside class components?

No. Hooks are designed to be used in functional components or custom Hooks.

Q32. What are the Rules of Hooks?

🔥 **Very important.**

Answer:

There are two main rules: **Hooks should be called only at the top level of a component or custom Hook, and they should be called only from React function components or custom Hooks.**

❌ Don't call Hooks:

```
if (isLoggedIn) {  
  useState();  
}
```

❌ Don't call them inside loops or nested functions.

Follow-up: Why can't Hooks be called conditionally?

React relies on the consistent order of Hook calls between renders. Changing that order can cause React to associate state with the wrong Hook.

Q33. What is `useState()`?

🔥🔥 **Must know.**

Also asked as:

- How do you manage state in a functional component?
- How do you create state in React?
- Explain `useState`.

Answer:

`useState` is a React Hook that allows a functional component to create and manage state. It returns the current state value and a function used to update it.

```
const [count, setCount] = useState(0);
```

Here:

- count → current state
 - setCount → state update function
 - 0 → initial value
-

Q34. Does calling the state setter immediately change the state variable?

🔥 Tricky and important.

Answer:

No. A state setter schedules an update. The current render's state value does not change immediately inside that same execution. React will render the component again with the updated state.

Example:

```
setCount(count + 1);
```

```
console.log(count);
```

The console.log can still see the old value in that render.

Q35. What is a functional state update?

Also asked as:

- How do you update state based on previous state?
- Why do we use prevState?

Answer:

When the next state depends on the previous state, we should use the functional form of the state setter.

```
setCount(prevCount => prevCount + 1);
```

Follow-up: Why is this better?

It ensures React uses the latest available previous state, especially when multiple updates are scheduled.

Q36. What happens if you call `setCount(count + 1)` three times?

🔥 **Very common output/trick question.**

```
setCount(count + 1);  
setCount(count + 1);  
setCount(count + 1);
```

Answer:

All three updates use the same count value from the current render, so they don't necessarily produce three increments.

If we want three sequential updates:

```
setCount(prev => prev + 1);  
setCount(prev => prev + 1);  
setCount(prev => prev + 1);
```

Each update receives the latest previous state.

Q37. What is batching in React?

Answer:

Batching means React can group multiple state updates together and process them in a way that reduces unnecessary re-renders. Modern React performs automatic batching in many asynchronous situations as well as during event handling.

Follow-up: Why is batching useful?

It improves performance by reducing unnecessary rendering work.

Q38. What is `useEffect()`?

🔥🔥 One of the most asked React questions.

Also asked as:

- What is `useEffect`?
- When do you use `useEffect`?
- How do you handle side effects in React?

Answer:

`useEffect` is a Hook used to synchronize a component with external systems or perform side-effect-related work, such as API requests, subscriptions, timers, or interacting with browser APIs.

Example:

```
useEffect(() => {  
  console.log("Effect ran");  
}, []);
```

Q39. What is a side effect in React?

Answer:

A side effect is work that interacts with something outside the component's rendering process, such as fetching data, setting up a subscription, modifying a browser API, or starting a timer.

Examples:

- API request
 - `setInterval`
 - Event listener
 - WebSocket connection
 - Document title update
-

Q40. What does the dependency array in `useEffect` do?

🔥🔥 **Must know.**

Answer:

The dependency array tells React which values the effect depends on. React re-runs the effect when one of those dependencies changes.

```
useEffect(() => {  
  console.log(userId);  
}, [userId]);
```

The effect runs again when `userId` changes.

Q41. What happens if `useEffect` has no dependency array?

```
useEffect(() => {  
  console.log("Effect");  
});
```

Answer:

The effect runs after every completed render in which the component commits.

Q42. What happens if the dependency array is empty?

```
useEffect(() => {  
  console.log("Effect");  
}, []);
```

Answer:

The effect has no reactive dependencies, so it generally runs after the initial mount and does not re-run because of subsequent state or

prop changes. In development StrictMode, React may run the setup/cleanup cycle an extra time to help detect issues.

Q43. What is the cleanup function in `useEffect`?

🔥🔥 Very important.

Answer:

A cleanup function is returned from an effect to remove or stop whatever the effect set up, such as event listeners, timers, subscriptions, or connections.

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log("Running");  
  }, 1000);  
  
  return () => {  
    clearInterval(timer);  
  };  
}, []);
```

Follow-up: When does cleanup run?

Cleanup runs before the effect runs again when its dependencies change, and when the component is removed from the UI. In development StrictMode, React can also perform an extra setup-cleanup cycle.

Q44. Why is cleanup important?

Answer:

Cleanup prevents unwanted side effects, duplicate subscriptions, unnecessary timers, and potential memory leaks.

Q45. Can we make the `useEffect` callback directly `async`?

🔥 Common interview trap.

Answer:

We should not make the effect callback itself `async` because React expects the callback to return either nothing or a cleanup function, not a Promise.

Instead:

```
useEffect(() => {  
  async function fetchUsers() {  
    const response = await fetch("/api/users");  
    const data = await response.json();  
  }  
  
  fetchUsers();  
}, []);
```

Q46. What is `useRef()`?

🔥🔥 Very important.

Also asked as:

- Why do we use `useRef`?
- How do you access a DOM element in React?
- Does `useRef` cause re-render?

Answer:

`useRef` returns a mutable reference object whose `.current` value persists between renders. It is commonly used to access DOM elements or store values that should persist without causing a re-render when changed.

```
const inputRef = useRef(null);
```

```
<input ref={inputRef} />
```

Q47. Does changing `ref.current` cause a re-render?

🔥 Trick question.

Answer:

No. Changing `ref.current` does not trigger a React re-render.

That's one major difference between state and refs.

Q48. `useState` VS `useRef`?

🔥🔥 Very common.

Answer:

State is used for data that affects the UI and updating it causes a re-render. A ref is used for values that need to persist between renders without causing a re-render when they change.

<code>useState</code>	<code>useRef</code>
Used for UI state	Used for persistent mutable values
Update causes re-render	Changing <code>.current</code> doesn't
State value is part of rendering	Ref is not automatically part of rendering

Q49. What are common uses of `useRef`?

Answer:

Common uses include:

- Accessing DOM elements

- Storing timer IDs
 - Keeping a previous value
 - Storing mutable values that don't need to trigger rendering
-

Q50. What is `useContext()`?

🔥 Important.

Also asked as:

- How do you avoid prop drilling?
- How do you share data between components?
- What is React Context?

Answer:

`useContext` allows a functional component to read a value from a React Context without manually passing that value through every intermediate component.

Example use cases:

- Theme
 - Authentication user
 - Language
 - App-level settings
-

Q51. What is Context API?

Answer:

Context provides a way to share values with components in a part of the component tree without passing props through every intermediate component.

Basic structure:

```
const AuthContext = createContext();
```

```
<AuthContext.Provider value={user}>
```

```
<App />  
</AuthContext.Provider>
```

A child can consume it with:

```
const user = useContext(AuthContext);
```

Q52. What is prop drilling?

🔥 Very common.

Answer:

Prop drilling means passing data through multiple intermediate components using props even when those components do not need the data themselves, just so a deeply nested component can receive it.

```
App  
↓  
Parent  
↓  
Child  
↓  
GrandChild
```

If only GrandChild needs the data, passing it through every level is prop drilling.

Q53. How can you avoid prop drilling?

Answer:

For shared data, we can use Context API or a state management solution when appropriate. We can also restructure components or use composition to avoid unnecessary prop passing.

Q54. What is `useMemo()`?

🔥 Important.

Also asked as:

- How do you optimize expensive calculations?
- What is memoization in React?
- When should you use `useMemo`?

Answer:

`useMemo` memoizes the result of a calculation and recomputes it when its dependencies change. It can be useful for expensive calculations when avoiding unnecessary recalculation provides a real performance benefit.

```
const total = useMemo(() => {  
  return calculateTotal(items);  
}, [items]);
```

Follow-up: Does `useMemo` prevent component re-rendering?

No. It memoizes a calculated value. It does not prevent the component itself from rendering.

Q55. What is `useCallback()`?

🔥 Important.

Also asked as:

- How do you memoize a function?
- Why do we use `useCallback`?
- `useMemo` vs `useCallback`?

Answer:

`useCallback` memoizes a function reference and returns the same function between renders until its dependencies change. It can be useful when passing callbacks to memoized child components or when a stable function reference is needed.

```
const handleClick = useCallback(() => {  
  console.log("Clicked");  
}, []);
```

Q56. useMemo VS useCallback?

🔥🔥 Very common.

Answer:

useMemo memoizes a calculated value, while useCallback memoizes a function reference.

Easy trick:

```
useMemo    → remembers a VALUE  
useCallback → remembers a FUNCTION
```

Q57. What is React.memo?

Also asked as:

- How do you prevent unnecessary child renders?
- What is component memoization?

Answer:

React.memo memoizes a component so React can skip re-rendering it when its props have not changed, subject to the usual React rendering rules.

```
const User = React.memo(function User({ name }) {  
  return <h1>{name}</h1>;  
});
```

Follow-up: Does React.memo guarantee no re-render?

No. It is an optimization, not a guarantee. A component can still render because of its own state, context changes, or other factors.

Q58. React.memo VS useMemo VS useCallback?

🔥🔥 Excellent interview question.

Answer:

React.memo memoizes a component based on its props, useMemo memoizes a calculated value, and useCallback memoizes a function reference.

```
React.memo → Component  
useMemo → Value  
useCallback → Function
```

Q59. What is a custom Hook?

🔥 Important.

Also asked as:

- How do you create reusable Hook logic?
- What is custom Hook?
- Why create custom Hooks?

Answer:

A custom Hook is a JavaScript function whose name starts with use and which can use other React Hooks to encapsulate and reuse stateful logic between components.

Example:

```
function useAuth() {  
  const [user, setUser] = useState(null);  
  
  return { user, setUser };  
}
```

Q60. Why do we use custom Hooks?

Answer:

Custom Hooks help us reuse related stateful logic across multiple components without duplicating the same logic.

Follow-up: Does a custom Hook share the same state between components?

No. Calling the same custom Hook in two components gives each component its own Hook state. The logic is shared, not automatically the state.

Q61. What is the difference between `useEffect` and `useMemo`?

🔥 **Very common.**

Answer:

`useEffect` is used to synchronize with external systems and perform side effects, while `useMemo` is used to memoize the result of a calculation.

`useEffect` → side effects / synchronization
`useMemo` → calculated value

Q62. What is the difference between `useEffect` and `useCallback`?

Answer:

`useEffect` is for running effect logic based on dependencies, while `useCallback` is for keeping a stable function reference between renders until its dependencies change.

Q63. Does `useEffect` run before or after rendering?

Answer:

The standard `useEffect` runs after React has committed the updated UI. It is not intended for work that must happen before the browser paints.

Q64. What is `useLayoutEffect`?

Fresher-relevant, but lower priority.

Answer:

`useLayoutEffect` is similar to `useEffect`, but it runs after DOM mutations and before the browser paints. It is mainly useful when we need to measure or synchronously adjust the DOM before the user sees the result.

Follow-up: Which one should you normally use?

`useEffect` should be preferred in most cases. `useLayoutEffect` should be used when the effect specifically needs to run before the browser paints.

Q65. Can Hooks be called inside loops or conditions?

Answer:

No. Hooks should be called at the top level of a React component or custom Hook, not inside loops, conditions, or nested functions.

Part 2 — Most Important for Interview

Agar interviewer React Hooks par focus kare, ye questions **100% strong** hone chahiye:

1. What are Hooks?
2. Rules of Hooks
3. useState
4. Functional state update
5. Batching
6. useEffect
7. Dependency array
8. Empty dependency array
9. Cleanup
10. Side effects
11. useRef
12. useState vs useRef
13. useContext
14. Context API
15. Prop drilling
16. useMemo
17. useCallback
18. React.memo
19. useMemo vs useCallback
20. Custom Hooks
21. useEffect vs useMemo

Next Part 3: React Forms + Events + Component Communication + Controlled/Uncontrolled Components + Lifting State Up + Composition + Props patterns.

Next

React Interview — Part 3: Forms, Events & Component Communication

Ab **React Forms + Events + Parent/Child communication** karte hain. Ye topics MERN fresher interviews mein kaafi important hain.

Q66. How are events handled in React?

Also asked as:

- How do you handle events in React?
- What is event handling in React?
- How is React event handling different from HTML?

Answer:

React handles events using event handler functions passed as props, such as `onClick`, `onChange`, and `onSubmit`.

```
<button onClick={handleClick}>Click</button>
```

Follow-up: Why don't we write `onClick="handleClick()"` like HTML?

React uses JavaScript functions directly as event handlers, so we pass the function reference rather than a string.

Q67. What is SyntheticEvent in React?

Also asked as:

- What is React's event system?
- Does React use normal browser events?

Answer:

React provides a normalized event object to event handlers, historically called `SyntheticEvent`, which gives a consistent interface across browsers. Modern React uses the browser's event system underneath while providing React's event handling interface.

Follow-up: Can we access the original browser event?

Yes. The underlying browser event can be accessed through `event.nativeEvent`.

Q68. What is a controlled component?

🔥🔥 Very important

Also asked as:

- What is a controlled form?
- How do you control form inputs in React?
- How do you handle input values using state?

Answer:

A controlled component is a form element whose value is controlled by React state. The input value comes from state, and an event handler updates that state when the user types.

```
const [name, setName] = useState("");
```

```
<input  
  value={name}  
  onChange={(e) => setName(e.target.value)}  
/>
```

Follow-up: Why use controlled components?

They give React a single source of truth for the form data and make validation and conditional UI easier to manage.

Q69. What is an uncontrolled component?

Answer:

An uncontrolled component stores its current form value in the DOM rather than React state. We can commonly access its value using a ref.

```
const inputRef = useRef();
```

```
<input ref={inputRef} />
```

Q70. Controlled vs uncontrolled components?

🔥🔥 Very common

Answer:

In a controlled component, React state is the source of truth for the input value. In an uncontrolled component, the DOM maintains the value and React accesses it when needed, usually through a ref.

Controlled	Uncontrolled
React state controls value	DOM controls value
Uses value + onChange	Commonly uses ref
Easier validation	Less state management
More control	Simpler for some forms

Follow-up: Which do you generally prefer?

For most application forms, I prefer **controlled components** because they provide better control over validation and form behavior.

Q71. How do you handle form submission in React?

Answer:

We attach an onSubmit handler to the form and usually call `event.preventDefault()` to prevent the browser's default page reload. Then we can validate the data and send it to the backend.

```
const handleSubmit = (e) => {  
  e.preventDefault();
```

```
  // validate and submit data  
};
```

```
<form onSubmit={handleSubmit}>
```

Q72. How do you handle multiple inputs using one handler?

🔥 Practical interview concept.

Answer:

We can give each input a name attribute and update the corresponding property in state dynamically.

```
const [form, setForm] = useState({
  name: '',
  email: ''
});

const handleChange = (e) => {
  setForm({
    ...form,
    [e.target.name]: e.target.value
  });
};
```

Follow-up: Why use `[e.target.name]`?

It is a computed property name, so the property being updated is determined dynamically from the input's name.

Q73. How do you validate forms in React?

Answer:

We can validate form values before submission using JavaScript conditions or a validation library. We can check required fields, formats, length, and other business rules, then display appropriate error messages.

Follow-up: Client-side validation is enough?

No. Client-side validation improves user experience, but the backend must also validate the data because client-side checks can be bypassed.

🔥 This is a very good MERN interview answer.

Q74. How do you pass data from parent to child?

🔥🔥 Must know

Answer:

We pass data from a parent to a child through props.

```
function Parent() {  
  return <Child name="Pranjal" />;  
}
```

```
function Child({ name }) {  
  return <h1>{name}</h1>;  
}
```

Q75. How do you pass data from child to parent?

🔥🔥 Very common

Answer:

The parent passes a callback function to the child through props. The child calls that function with the required data.

```
function Parent() {  
  const handleData = (data) => {  
    console.log(data);  
  };  
  
  return <Child onSend={handleData} />;  
}  
  
function Child({ onSend }) {  
  return (  
    <button onClick={() => onSend("Hello")}>  
      Send  
    </button>  
  );  
}
```

Follow-up: Does the child directly change the parent's state?

No. The child calls the callback provided by the parent, and the parent decides how to update its state.

Q76. How do sibling components communicate?

Answer:

Sibling components generally communicate through their common parent. We lift the shared state to the parent and pass the required data or callbacks to each sibling.

```
Parent
 /  \
Child A Child B
```

The parent manages the shared data.

Q77. What is lifting state up?

🔥🔥 Very important

Also asked as:

- How do you share state between components?
- Where should shared state be stored?
- How do sibling components share data?

Answer:

Lifting state up means moving shared state to the closest common parent of the components that need it. The parent then passes the data and update functions through props.

Follow-up: Why do we lift state up?

To maintain a **single source of truth** for shared data and keep components synchronized.

Q78. What is the difference between state lifting and Context API?

Answer:

State lifting passes data through props from a common parent, while Context allows deeply nested components to access shared values without passing props through every intermediate component.

Follow-up: Should we always use Context instead of props?

No. For simple parent-child communication, props are usually clearer. Context is useful when data needs to be accessed by many components at different levels.

Q79. What is component composition?

We've touched this already, but interviewers often ask it here.

Answer:

Composition means building a component by combining smaller components or passing content through props such as children. It promotes reusable and flexible components.

Example:

```
function Card({ children }) {  
  return <div className="card">{children}</div>;  
}
```

Usage:

```
<Card>  
  <h2>Profile</h2>  
  <p>Hello</p>  
</Card>
```

Q80. What is the `children` prop?

🔥 Common.

Also asked as:

- How do you pass content between component tags?
- What is `props.children`?

Answer:

`children` is a special prop that contains the content placed between the opening and closing tags of a component.

```
<Card>
  <h1>Hello</h1>
</Card>
```

Inside Card:

```
function Card({ children }) {
  return <div>{children}</div>;
}
```

Q81. What is prop drilling?

We've already covered it, but here's the **interview-ready version**:

Prop drilling is passing data through multiple components that do not actually need the data, only to deliver it to a deeply nested component.

Follow-up: How can you solve it?

Depending on the situation, we can use Context, a state-management solution, component composition, or restructure the component tree.

Q82. What is a reusable component?

Answer:

A reusable component is designed to work in multiple places by accepting different data or configuration through props instead of being tightly coupled to one specific use case.

Example:

```
<Button text="Login" />
<Button text="Register" />
```

Q83. What is a presentational component?

Also asked as:

- What is a UI component?
- What is a dumb component?

Answer:

A presentational component mainly focuses on displaying UI based on the props it receives, with minimal application logic.

Example:

```
function UserCard({ name, email }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>{email}</p>
    </div>
  );
}
```

Q84. What is a container component?

Answer:

A container component traditionally focuses more on application logic, data fetching, and state management, and passes the required data to presentational components.

Follow-up: Is this pattern mandatory in modern React?

No. It is a useful design pattern, but modern React often combines logic and UI differently using Hooks and custom Hooks.

Q85. What are controlled inputs useful for?

Answer:

Controlled inputs are useful when we need **real-time validation, conditional UI, dynamic form behavior, or direct access to the current form state.**

Q86. How do you reset a React form?

Answer:

For a controlled form, we can reset the state holding the form values to their initial values.

```
setForm({  
  name: '',  
  email: ''  
});
```

Follow-up: Can we use the HTML form reset?

Yes, but when inputs are controlled by React state, we should also reset the corresponding state so React and the DOM remain synchronized.

Q87. How do you handle checkbox and radio inputs?

Answer:

For checkboxes, we commonly use the checked property, while text inputs use value.

```
<input
  type="checkbox"
  checked={isChecked}
  onChange={(e) => setIsChecked(e.target.checked)}
/>
```

Follow-up: Why checked instead of value?

Because the meaningful state of a checkbox is whether it is checked or unchecked.

Q88. How do you handle a select dropdown in React?

Answer:

We can control it using value and onChange, just like other controlled inputs.

```
<select
  value={country}
  onChange={(e) => setCountry(e.target.value)}
>
  <option value="india">India</option>
  <option value="usa">USA</option>
</select>
```

Q89. What is event propagation in React?

Answer:

Event propagation is the process through which an event moves through the component/DOM hierarchy. React event handling supports the usual capture and bubbling phases.

Follow-up: How do you stop bubbling?

```
event.stopPropagation();
```

It stops the event from continuing to propagate.

Q90. What is `preventDefault()` in React?

Answer:

It prevents the browser's default behavior for an event.

Most common example:

```
function handleSubmit(e) {  
  e.preventDefault();  
}
```

This prevents the browser from performing the normal form submission.

Follow-up: `preventDefault()` VS `stopPropagation()`?

`preventDefault()` stops the browser's default action, while `stopPropagation()` stops event propagation.

Q91. How does a child component trigger a parent action?

Answer:

The parent passes a callback as a prop, and the child calls that callback when the required event occurs.

This is the standard **child → parent communication pattern** in React.

Q92. Can props be functions?

🔥 Common follow-up.

Answer:

Yes. Functions can be passed as props. This is commonly used for child-to-parent communication and reusable components.

```
<Child onDelete={handleDelete} />
```

Q93. Can a component modify its parent's props?

Answer:

No. Props are read-only. A child can call a callback received through props, but the parent remains responsible for changing its own state.

Q94. What is the single source of truth?

Answer:

The single source of truth means keeping a particular piece of shared state in one authoritative location rather than maintaining multiple independent copies that can become inconsistent.

In React, shared state is often kept in the closest common parent.

🔥 PART 3 — Most Important

Interview ke liye inko **especially strong** karo:

■ MUST KNOW

- Controlled vs uncontrolled components
- Form handling
- onChange
- onSubmit
- preventDefault
- Parent → child
- Child → parent
- Sibling communication
- Lifting state up
- Props as functions
- Prop drilling
- Context vs props
- children
- Component composition
- Event propagation
- Reusable components

■ Secondary

- Presentational component
 - Container component
 - Checkbox/radio handling
 - Select handling
-

Next Part 4 🔥

Ab hum **React Router + API Integration + Axios + Authentication + Protected Routes** karenge — ye MERN fresher ke liye **bahut high-value section** hai.

Next



React Interview — Part 4: React Router + API Integration

Ab React ka **MERN-specific high-value section**. Isme interviewer tumhare React ko backend ke saath connect karne ki understanding check karta hai.

Q95. What is React Router?

Also asked as:

- How do you implement routing in React?
- What is client-side routing?
- How do you navigate between pages in React?

Answer:

React Router is a routing library used to manage navigation and different URLs in React applications. It allows us to render different components based on the current URL without requiring a full page reload.

Follow-up: Is React Router part of React itself?

No. It is a separate routing library used with React.

Q96. What is client-side routing?

Answer:

Client-side routing means the browser's URL can change and React can display the corresponding component without requesting and loading a completely new HTML document for every route.

Q97. What is the difference between `<Link>` and `<a>`?



Very common

Answer:

Link from React Router performs client-side navigation, while a normal `<a>` element typically causes the browser to navigate by requesting the target URL, which can result in a full page reload.

```
jsx id="4l6o6n" <Link to="/about">About</Link>
```

Q98. What is `useNavigate()`?

Answer:

`useNavigate` is a React Router Hook that allows us to navigate programmatically from JavaScript code.

```
``jsx id="8vlqv6" const navigate = useNavigate();  
navigate("/dashboard");
```

Follow-up: When would you use it?

> For actions such as navigating after a successful login, logout, form submission, or button click.

Q99. What is `useParams()`?

Answer:

> **`useParams` is used to access dynamic parameters from the current URL.**

For example:

```
``text id="1xik5d"  
/cars/123
```

Route:

```
jsx id="d7v4zh" /cars/:id
```

Then:

```
jsx id="s6i2k1" const { id } = useParams();
```

id will be "123".

Q100. What is `useLocation()`?

Answer:

`useLocation` gives information about the current URL location, such as the pathname, search parameters, and hash.

```
``jsx id="f5r5fp" const location = useLocation();
```

```
console.log(location.pathname);
```

Q101. What is a dynamic route?

Answer:

> ****A dynamic route contains a variable URL segment that can represent different resources.****

Example:

```
``text id="d9ol48"
```

```
/users/:id
```

These URLs can represent different users:

```
text id="y9m3xk" /users/10 /users/20 /users/30
```

Q102. What is a nested route?

Answer:

A nested route is a route defined inside another route, allowing a parent layout to contain different child pages.

Example:

```
text id="o5rvsp" /dashboard /dashboard/profile /dashboard/settings
```

Q103. What is a protected route?

 **Very important for MERN**

Also asked as:

- How do you protect a route?
- How do you restrict unauthenticated users?
- How do you implement authentication-based routing?

Answer:

A protected route is a route that can only be accessed when certain conditions are satisfied, commonly when the user is authenticated. If the user is not authenticated, we redirect them to a login page.

Example concept:

```
text id="x4v7jd" User logged in?    ↓    Yes → Dashboard    No → Login
```

Follow-up: Is frontend route protection enough for security?

No. Frontend protection only controls the user interface. The backend must also authenticate and authorize every protected API request.

 This answer makes a good impression in a MERN interview.

Q104. Authentication vs Authorization?

 **Must know**

Answer:

Authentication verifies who the user is, while authorization determines what that authenticated user is allowed to access.

Example:

``text id="12i9zv" Authentication: "Who are you?"

Authorization: "What are you allowed to do?"

Q105. How does a typical login flow work in a MERN application?

🔥🔥🔥 ****Very important****

Answer:

> ****The user submits login credentials to the backend. The backend verifies the credentials and returns an authentication mechanism, commonly a session or token. The client then uses that authentication state when accessing protected resources. The backend validates the authentication on protected requests.****

Follow-up: What happens after successful login?

> The frontend stores or receives the authentication state according to the application's security design, updates the UI, and can navigate the user to a protected page.

Q106. What is an API?

Answer:

> ****An API, or Application Programming Interface, is a defined way for different software components to communicate with each other. In a MERN application, the React frontend commonly communicates with the Node/Express backend through HTTP APIs.****

Q107. How does React communicate with a backend?



Answer:

> **React communicates with the backend by sending HTTP requests to API endpoints using tools such as Fetch or Axios. The backend processes the request and returns a response, commonly in JSON format.**

``text id="7bry9n"

React

↓ HTTP request

Express API

↓

MongoDB

↓

Express

↓ response

React

Q108. What is Axios?

Answer:

Axios is a Promise-based HTTP client used to make requests from frontend applications to APIs. It supports features such as request and response interceptors, custom configuration, and convenient response handling.

Q109. Axios vs Fetch?

 **Common**

Answer:

Fetch is a browser-provided Web API, while Axios is an external HTTP client library. Both support asynchronous HTTP requests, but Axios provides additional features such as interceptors and convenient configuration.

Follow-up: Which one do you prefer?

Good answer:

Both are valid. In a MERN project, I prefer Axios when I need features like centralized configuration or interceptors; for simple requests, Fetch is also sufficient.

Q110. What is an API endpoint?

Answer:

An API endpoint is a specific URL through which a client can access a particular backend resource or operation.

Examples:

text id="jpjf1q" /api/users /api/users/:id /api/auth/login /api/cars

Q111. What are GET, POST, PUT, PATCH and DELETE?

 **Must know**

Answer:

They are HTTP methods used for different operations:

GET → retrieve data

POST → create data

PUT → replace/update a resource

PATCH → partially update a resource

DELETE → remove a resource

Q112. How do you handle API loading and error states in React?

🔥 Practical theory question.

Answer:

I usually maintain separate loading, data, and error states. Before the request I set loading to true, after success I store the data, and if the request fails I store an appropriate error message. Finally, I stop the loading state.

Conceptually:

text id="6n1ev7" loading → true ↓ API request ↓ success → data OR failure → error
↓ loading → false

Q113. What is CORS?

🔥🔥 MERN interview favourite

Also asked as:

- Why am I getting a CORS error?
- What is Cross-Origin Resource Sharing?
- Why can't my React app access my Express API?

Answer:

CORS stands for Cross-Origin Resource Sharing. It is a browser security mechanism that controls whether a web page from one origin can access resources from another origin. The server can specify which origins, methods, headers, and credentials are allowed.

Example:

``text id="7pblzw" Frontend: http://localhost:5173

Backend: http://localhost:5000

These are different origins because their ports differ.

Q114. How do you enable CORS in Express?

Answer:

> We can use the `cors` middleware and configure the allowed origins and other options according to the application's requirements.

```
``js id="o6y1o3"
import cors from "cors";
```

```
app.use(cors());
```

Follow-up: Is `cors()` itself authentication?

No. CORS controls browser cross-origin access. It does not authenticate users or protect an API by itself.

🔥 Important distinction.

Q115. What is an interceptor in Axios?

🔥 Important for MERN.

Answer:

An Axios interceptor allows us to run logic before a request is sent or before a response is handled. It is commonly used for adding authentication headers, logging, or handling common errors centrally.

Example concept:

text id="j8d6nr" Request ↓ Interceptor ↓ API

Q116. Why are Axios interceptors useful for authentication?

Answer:

They can automatically attach authentication information to outgoing requests instead of manually adding it in every API call. They can also handle common authentication-related responses centrally.

Q117. What is an environment variable in a React application?

Answer:

An environment variable stores configuration values outside the source code, such as API URLs or environment-specific settings. It allows us to use different configuration values for development and production.

Example concept:

text id="x4t9yc" Development → local API Production → deployed API

Follow-up: Should we store secret API keys in frontend environment variables?

 **Important**

No. Anything included in a frontend build can potentially be exposed to users. Frontend environment variables should only contain values that are safe to make public. Sensitive secrets should stay on the backend.

Q118. How do you handle authentication state in React?

Answer:

Authentication state can be managed using React state, Context, or a dedicated state-management solution. The application typically keeps information such as the current user and authentication status, while protected API requests are authenticated through the application's chosen authentication mechanism.

Q119. Where should authentication tokens be stored?

🔥🔥 Tricky security question

Answer:

It depends on the authentication architecture. For web applications, an HttpOnly, Secure cookie is often preferred for sensitive session or token storage because JavaScript cannot directly read an HttpOnly cookie. If tokens are stored in browser storage such as localStorage, they are accessible to JavaScript and therefore require careful XSS protection.

Follow-up: Is localStorage completely secure?

No. Data in localStorage is accessible to JavaScript running on the page, so an XSS vulnerability could expose it.

Q120. What is JWT?

🔥🔥 MERN favourite

Also asked as:

- What is JSON Web Token?
- How does JWT authentication work?

- How do you authenticate APIs using JWT?

Answer:

JWT stands for JSON Web Token. It is a compact token format that can carry claims and can be digitally signed. In a typical authentication flow, the server issues a JWT after successful authentication, and the client sends it with later requests so the server can verify the token and identify the user.

Follow-up: Is JWT encrypted?

Not by default. A normal signed JWT is encoded and signed, not encrypted. Its payload should not contain sensitive information that must remain secret.

🔥 Excellent interview point.

Q121. What is the difference between authentication token and authorization?

Answer:

A token can be used as part of an authentication mechanism to prove or represent a user's authenticated session. Authorization is the decision about what that authenticated user is allowed to access or perform.

Q122. What happens when a JWT expires?

Answer:

The server should reject the expired token. The application can then require the user to authenticate again or, if the system uses refresh tokens, obtain a new access token through the refresh flow.

Q123. What is a refresh token?

Answer:

A refresh token is a longer-lived credential used to obtain a new access token without requiring the user to log in again, depending on the authentication architecture.

Follow-up: Why use refresh tokens?

They allow short-lived access tokens while maintaining a smoother user session.

Q124. What is CRUD?

🔥 Easy but commonly asked.

Answer:

CRUD stands for Create, Read, Update and Delete. These represent the basic operations performed on application data.

text id="x9gr5g" Create → POST Read → GET Update → PUT/PATCH Delete → DELETE

Q125. How would you fetch data when a React component loads?

Answer:

I would use an effect to synchronize the component with the API request, maintain loading and error states, and store the returned data in state.

Conceptually:

```
jsx id="xw6t0a" useEffect(() => { fetchUsers(); }, []);
```

Follow-up: Why not fetch directly during rendering?

Because rendering should remain free of side effects. API requests belong in an appropriate effect or data-fetching mechanism.

Q126. What happens if an API request fails?

Answer:

I would catch the error, update the error state, stop the loading state, and show a meaningful message or fallback UI to the user.

Part 4 — MUST KNOW

For a **MERN fresher**, these should be absolutely strong:

1. React Router
2. <Link> vs <a>
3. useNavigate
4. useParams
5. Dynamic routes
6. Protected routes
7. Authentication vs authorization
8. React  backend communication
9. Fetch vs Axios
10. HTTP methods
11. API endpoint
12. Loading/error handling
13. CORS
14. Axios interceptors
15. Environment variables
16. JWT
17. Access token vs refresh token
18. Token storage/security
19. CRUD

20. API calls with useEffect

Next Part 5: 🔥 React Performance + Advanced Concepts — re-rendering, React.memo, reconciliation deeper questions, lazy loading, Suspense, code splitting, portals, error boundaries, StrictMode, and the kinds of **tricky React questions interviewers use to differentiate candidates**.

Next

React Interview — Part 5: Performance + Advanced Concepts

Ab React ke **advanced-but-fresher-relevant** questions. Ye woh section hai jahan interviewer thoda depth check karta hai. **10-year experience wale unnecessary topics nahi laa raha.**

Q127. What is re-rendering in React?

Also asked as:

- What does re-render mean?
- When does React re-render a component?
- What causes a component to render again?

Answer:

A re-render means React runs a component again to determine what its UI should look like with the current props and state. React then reconciles the result and applies the necessary DOM changes.

Follow-up: Does re-render mean the DOM is completely recreated?

No. React compares the new result with the previous one and updates only the necessary DOM changes.

Q128. What can cause a component to re-render?

🔥🔥 Very common

Answer:

A component can re-render when its **state changes, its props change, its parent re-renders, or a consumed context value changes.**

Follow-up: Does changing a normal variable cause a re-render?

No. A normal JavaScript variable changing does not tell React to update the UI.

Q129. What is unnecessary re-rendering?

Answer:

An unnecessary re-render happens when a component renders even though its output does not need to change because of the update. Too many unnecessary renders can affect performance in large applications.

Follow-up: How can you reduce unnecessary re-renders?

We can use appropriate component structure, `React.memo`, `useMemo`, `useCallback`, and avoid unnecessary state or context updates. These should be used only when they provide an actual benefit.

Q130. What is `React.memo()`?

🔥 Already introduced, but interviewer may ask independently.

Answer:

`React.memo` is a performance optimization that allows React to skip re-rendering a component when its props have not changed.

Follow-up: Does `React.memo` compare objects deeply?

No. By default, it uses a shallow comparison of props.

Q131. What is shallow comparison?

Answer:

Shallow comparison checks whether values are the same at the top level. For objects, arrays, and functions, it generally compares their references rather than their contents.

Example:

```
{ } === { } // false
```

because they are different object references.

Q132. Why can an object prop cause a memoized component to re-render?

🔥 Good follow-up

```
<Child user={{ name: "Pranjal" }} />
```

Answer:

A new object is created on every parent render, so its reference is different. React.memo sees the prop as changed even if the object's contents are the same.

Follow-up: How can we avoid this when necessary?

We can keep the object reference stable, for example with `useMemo`, when that optimization is actually needed.

Q133. Why can a function prop cause unnecessary re-renders?

Answer:

A function created inside a component gets a new reference on each render. If it is passed to a memoized child, the child may see the function prop as changed.

Follow-up: What can help?

`useCallback` can provide a stable function reference when appropriate.

Q134. What is memoization?

Answer:

Memoization means storing a previously calculated result or reference so it can be reused when the relevant inputs have not changed.

In React:

`React.memo` → memoizes component rendering

`useMemo` → memoizes a value

`useCallback` → memoizes a function reference

Q135. Should we use `useMemo` and `useCallback` everywhere?

🔥🔥 **Trick question**

Answer:

No. They are performance optimizations and also have their own cost and complexity. We should use them when they solve an actual performance or referential-equality problem rather than using them everywhere.

🔥 This is a much better answer than saying “they make React faster.”

Q136. What is lazy loading in React?

Also asked as:

- How do you load components only when needed?
- How do you optimize bundle size?

Answer:

Lazy loading means loading a component or module only when it is needed instead of including everything in the initial JavaScript bundle. React supports this using lazy().

```
const About = lazy(() => import("./About"));
```

Q137. What is code splitting?

Answer:

Code splitting means dividing the application’s JavaScript bundle into smaller chunks that can be loaded when required instead of downloading the entire application at once.

Follow-up: Why is it useful?

It can reduce the initial JavaScript that the browser needs to download and process, improving initial load performance.

Q138. What is `Suspense`?

🔥 Important

Answer:

Suspense allows React to display fallback UI while some child content is not ready, such as a component being loaded with `React.lazy`.

```
<Suspense fallback={<p>Loading...</p>}>>  
<About />  
</Suspense>
```

Follow-up: What is the fallback?

The fallback is the temporary UI displayed while the suspended content is not ready.

Q139. How do `lazy()` and `Suspense` work together?

Answer:

`lazy()` defines a component that is loaded dynamically, while `Suspense` provides fallback UI while that component is loading.

```
lazy()  
↓  
Component loading  
↓  
Suspense fallback  
↓  
Component loaded  
↓  
Actual UI
```

Q140. What is an Error Boundary?

 Important

Also asked as:

- How do you handle rendering errors in React?
- What happens if a component crashes?
- How do you show fallback UI for component errors?

Answer:

An Error Boundary is a React component that catches certain errors during rendering and lifecycle processing of its child component

tree and displays fallback UI instead of allowing the entire UI to crash.

Follow-up: Can an Error Boundary catch API errors?

Not automatically. Error boundaries are mainly for errors during rendering and related React lifecycle processing. API errors should normally be handled using application error-handling logic.

Q141. Can a functional component itself be an Error Boundary?

Answer:

Traditionally, Error Boundaries are implemented using class components because the boundary API relies on lifecycle methods such as componentDidCatch.

Q142. What is a Portal in React?

Also asked as:

- How do you render a modal outside the parent DOM hierarchy?
- What is createPortal?

Answer:

A Portal allows React to render a component's DOM into a different DOM node while keeping it part of the same React tree. It is commonly used for modals, dialogs, and tooltips.

Example concept:

```
createPortal(  
  <Modal />,  
  document.getElementById("modal-root")  
);
```

Q143. Does a Portal break the React component relationship?

Answer:

No. The DOM node is rendered somewhere else, but the component remains part of the same React tree. Events and context can still behave according to the React tree.

Q144. What is React StrictMode?

🔥 Common follow-up

Answer:

StrictMode is a development feature that helps detect potential problems and unsafe patterns in a React application. It can intentionally perform additional development checks, including extra setup/cleanup behavior for some effects.

Follow-up: Does StrictMode affect production?

The extra development checks are not performed the same way in production builds.

Q145. Why does `useEffect` sometimes appear to run twice?

🔥🔥 Very common fresher trap

Answer:

In React development with StrictMode, React may intentionally run an effect's setup and cleanup cycle an extra time to help identify side-effect bugs. This is a development behavior and should not be confused with normal production behavior.

Follow-up: Should we remove StrictMode because of this?

No. We should make the effect properly handle setup and cleanup rather than removing StrictMode just to hide the issue.

Q146. What is reconciliation?

🔥 Interviewer can ask this separately.

Answer:

Reconciliation is React's process of comparing the previous and new rendered element trees to determine what changes need to be applied to the actual DOM.

Follow-up: What role do keys play?

Keys help React identify list items consistently across renders, especially when items are inserted, removed, or reordered.

Q147. Why are keys important in React lists?

Answer:

Keys provide stable identity to list elements so React can correctly determine which items changed, were added, or were removed during reconciliation.

Follow-up: Why is index sometimes a bad key?

If items are inserted, removed, or reordered, their indexes can change. This can cause React to associate the wrong identity with a component.

Q148. What is React Fiber?

🟡 **Know the concept, don't over-study it.**

Answer:

React Fiber is React's internal architecture for managing rendering work. It allows React to break rendering work into smaller units and better prioritize and schedule updates.

Follow-up: Do you use Fiber directly?

No. It is an internal React implementation detail that developers normally do not interact with directly.

Q149. What is the difference between rendering and committing in React?

🟡 Good advanced fresher question.

Answer:

During the render phase, React calculates what the UI should look like. During the commit phase, React applies the necessary changes to the actual DOM and performs related commit work.

Q150. What is hydration?

🔥 Important if interviewer asks React + SSR.

Answer:

Hydration is the process of attaching React's behavior and event handling to HTML that was already rendered on the server.

Follow-up: Where is hydration commonly used?

It is commonly used in server-rendered React applications and frameworks such as Next.js.

Q151. CSR vs SSR?

Answer:

CSR, or Client-Side Rendering, generates the UI mainly in the browser using JavaScript. SSR, or Server-Side Rendering, generates HTML on the server and sends it to the browser before React takes over the interactive parts.

Follow-up: One advantage of SSR?

SSR can improve initial content delivery and can be beneficial for SEO, depending on the application.

Q152. What is the difference between state and derived data?

Answer:

State is data that the application needs to store and update over time. Derived data can be calculated from existing props or state and usually does not need to be stored separately.

Follow-up: Why avoid unnecessary derived state?

Storing the same information in multiple places can create synchronization problems. If something can be reliably calculated from existing state, it is often better to calculate it.

Q153. Should every value used by a component be stored in state?

 **Good interviewer question**

Answer:

No. Only values that need to persist between renders and whose changes should update the UI generally need state. Constants, derived values, and temporary values may not need state.

Q154. What is the difference between state and props in terms of ownership?

Answer:

The component receiving props does not own those values; they are provided by its parent. State is owned and managed by the component that declares it.

Q155. What is the best way to optimize a React application?

🔥 Broad interview question.

Answer:

First, I would identify the actual performance bottleneck using profiling tools. Then I would optimize based on the problem—for example, reducing unnecessary renders, splitting large bundles, lazy-loading routes or components, keeping state at an appropriate level, and memoizing expensive work when necessary.

🔥 Don't say:

“I use useMemo everywhere.”

That sounds like memorized knowledge rather than understanding.

🔥 Part 5 — Interview Priority

■ MUST KNOW

These are the ones I'd make **very strong for a fresher**:

1. Re-rendering
2. Causes of re-render

3. Unnecessary re-render
4. React.memo
5. Shallow comparison
6. useMemo
7. useCallback
8. memo vs useMemo vs useCallback
9. Lazy loading
10. Code splitting
11. Suspense
12. Error Boundary
13. StrictMode
14. Why useEffect runs twice in development
15. Reconciliation
16. Keys
17. Portal
18. State vs derived data

Know the concept

- Fiber
- Render vs commit
- Hydration
- CSR vs SSR

Next Part 6: 🔥 React + MERN project/interview questions — “Tumne React project mein kya kiya?”, API integration explain karo, authentication kaise ki, folder structure, state management, errors, deployment, Git/GitHub, aur project-based follow-ups. Ye section tumhare **actual project ko defend karne** ke liye bahut important hoga.

Now start node kitne questions honge ab question repeat mat Krna kyuki react me tumne ek same questions ko 2 3 baar show Kiya tha

Haan, bilkul. **Node.js ab fresh series hogi** aur React wali mistake repeat nahi hogi. Ek concept ko ek hi baar main question ke रूप में karenge; agar interviewer usi concept ko doosre wording mein pooch sakta hai, woh **usi question ke andar** de dunga.

Node.js — Fresher MERN Interview

Main approximately **90–100 genuinely important questions** rakhunga.

Isme:

Section	Approx. Questions
Node.js Fundamentals	15
Modules & npm	12
Core Node.js	15
Async JavaScript in Node	12
File System & Streams	10
HTTP & APIs	10
Express.js	15
Authentication & Middleware	10
Error Handling & Security	8
MERN/Project-based Node Questions	10
Total	~100

Important rule

-  Same question repeat nahi hoga.
-  10-year experienced developer wale unnecessary questions nahi.
-  Practical/coding questions abhi nahi.
-  Fresher + MERN interview relevant.
-  Basic → intermediate → advanced progression.
-  **Alternate wording + follow-up + follow-up answer** har relevant question ke saath.
-  Easy English, but **interview-level wording**.
-  Jahan interviewer tricky follow-up pooch sakta hai, woh bhi cover karenge.

Aur **Express ko Node ke andar mix karke confuse nahi karunga** — Node fundamentals pehle complete, phir Express ka dedicated section.

Total target: ~100 questions. Exact count content ke hisaab se 1–2 upar-niche ho sakta hai, lekin filler questions add karke number nahi badhaunga.

Node.js — Part 1: Fundamentals

Q1 se start karenge.

Start

Node.js Interview — Part 1: Fundamentals

Target: ~100 questions

Aaj Q1–Q20. Same question repeat nahi hoga.

Q1. What is Node.js?

Also asked as:

- What do you know about Node.js?
- Explain Node.js in simple words.
- Why is Node.js used?
- Is Node.js a programming language?

Answer:

Node.js is a JavaScript runtime environment that allows us to run JavaScript outside the browser. It is built on Chrome's V8 JavaScript engine and is commonly used to build server-side applications and APIs.

Follow-up: Is Node.js a programming language?

No. JavaScript is the programming language, while Node.js is a runtime environment that executes JavaScript outside the browser.

Follow-up: Is Node.js a framework?

No. Node.js is a runtime environment. Frameworks such as Express.js can be built and used on top of Node.js.

Q2. Why do we use Node.js?

Also asked as:

- Why did you choose Node.js?
- What are the advantages of Node.js?
- Why is Node.js popular for backend development?

Answer:

Node.js allows developers to use JavaScript on the server side, has an efficient asynchronous I/O model, and has a large npm ecosystem. It is well suited for APIs, real-time applications, and applications that handle many I/O operations.

Follow-up: What is the biggest advantage of Node.js for a MERN developer?

We can use JavaScript across both frontend and backend, which makes development and code sharing easier.

Q3. What is V8 Engine?

 Very common.

Answer:

V8 is Google's open-source JavaScript engine, originally developed for Chrome. Node.js uses V8 to execute JavaScript code outside the browser.

Follow-up: Does Node.js contain only V8?

No. Node.js uses V8 for JavaScript execution and also provides additional runtime APIs and components for things like networking, file systems, and asynchronous operations.

Q4. How is Node.js different from JavaScript in the browser?

Answer:

JavaScript is the language, while Node.js provides a runtime for executing JavaScript outside the browser. Browser JavaScript has browser APIs such as the DOM, while Node.js provides server-side APIs such as file-system and networking APIs.

Follow-up: Can Node.js directly access the browser DOM?

No. Node.js does not provide the browser DOM APIs.

Q5. Is Node.js single-threaded?

 **Very important**

Answer:

JavaScript execution in Node.js normally runs on a single main thread, but Node.js can use the operating system and its underlying runtime mechanisms, including a worker pool, to handle certain asynchronous operations.

Follow-up: If Node.js is single-threaded, how can it handle multiple requests?

Node.js uses asynchronous, non-blocking I/O. Instead of waiting for an I/O operation to finish, it can continue handling other work and process the result when the operation completes.

Q6. What is non-blocking I/O?

 **Must know.**

Also asked as:

- What does non-blocking mean in Node.js?

- How does Node.js handle I/O?
- Explain asynchronous I/O.

Answer:

Non-blocking I/O means Node.js does not have to stop JavaScript execution while waiting for an I/O operation such as a database query or file operation to complete. It can continue doing other work and handle the result later.

Follow-up: Why is this useful?

It allows Node.js to efficiently handle many I/O-bound operations without creating a separate JavaScript thread for every request.

Q7. What is blocking I/O?

Answer:

Blocking I/O means the execution waits for an I/O operation to complete before continuing with the next operation.

Follow-up: Why can excessive blocking be a problem in Node.js?

Because Node.js relies heavily on its main JavaScript thread for executing application code. Long blocking operations can prevent it from processing other work efficiently.

Q8. Blocking vs non-blocking I/O?

Answer:

Blocking I/O waits for the operation to finish before continuing, while non-blocking I/O allows other work to continue while the operation is in progress.

Easy line:

Blocking waits; non-blocking continues.

Q9. What is the Event Loop in Node.js?

🔥🔥🔥 Extremely important

Also asked as:

- How does Node.js handle asynchronous operations?
- How does the Node.js event loop work?
- Explain Node.js asynchronous architecture.

Answer:

The Event Loop is a mechanism that allows Node.js to handle asynchronous operations without blocking the main JavaScript execution. When asynchronous work completes, its callbacks or continuations can be processed by the Event Loop when the JavaScript thread is ready.

Follow-up: Why is the Event Loop important?

It enables Node.js to handle many I/O operations efficiently while keeping JavaScript execution on its main thread.

Q10. What is asynchronous programming?

Answer:

Asynchronous programming allows a program to start an operation without waiting for it to finish before doing other work. Once the operation completes, its result can be handled through a callback, Promise, or async/await.

Example situations:

- Database request
 - API request
 - File operation
 - Network request
-

Q11. What is synchronous programming?

Answer:

Synchronous programming executes operations sequentially, where the current operation generally completes before the next operation proceeds.

Q12. Why is Node.js suitable for I/O-intensive applications?

🔥 Important.

Answer:

Node.js uses an asynchronous, non-blocking I/O model, so it can handle many operations such as network and database requests without blocking the main JavaScript execution while waiting for those operations to complete.

Follow-up: Is Node.js ideal for every type of application?

No. Node.js is particularly strong for I/O-bound workloads. CPU-heavy work can block the main JavaScript thread if it is not handled appropriately.

Q13. Is Node.js good for CPU-intensive tasks?

🔥 Tricky.

Answer:

Node.js can handle CPU-intensive tasks, but long CPU-bound work on the main JavaScript thread can block the Event Loop and reduce responsiveness. For such workloads, techniques such as worker threads or separate services can be used.

Follow-up: What happens if a CPU-heavy operation blocks the Event Loop?

Other JavaScript callbacks and requests that need the main thread may have to wait, which can make the application slow or unresponsive.

Q14. What is the difference between Node.js and traditional server-side technologies?

Answer:

Node.js uses JavaScript and an asynchronous, event-driven model, while many traditional server-side environments commonly use a thread-per-request or similar blocking model. Node.js can handle I/O-heavy workloads efficiently without creating a separate JavaScript thread for every request.

⚠ Don't say:

“Node.js is always faster than every backend technology.”

That's too absolute.

Q15. What is event-driven architecture in Node.js?

🔥 Important.

Answer:

In an event-driven architecture, the application responds to events or completed operations by executing the corresponding handlers. Node.js heavily uses this model for asynchronous operations and event-based communication.

Example:

Request arrives

↓

Start async operation



Other work continues



Operation completes



Handler executes

Q16. What is an EventEmitter in Node.js?

Answer:

EventEmitter is a Node.js mechanism that allows objects to emit named events and register listener functions that respond to those events.

Example:

```
import { EventEmitter } from "events";
```

```
const emitter = new EventEmitter();
```

```
emitter.on("message", () => {  
  console.log("Message received");  
});
```

```
emitter.emit("message");
```

Follow-up: What do `on()` and `emit()` do?

`on()` registers a listener for an event, while `emit()` triggers that event.

Q17. What is a callback in Node.js?

Answer:

A callback is a function passed to another function so that it can be executed later, commonly after an asynchronous operation completes.

Example:

```
fs.readFile("data.txt", "utf8", (err, data) => {  
  console.log(data);  
});
```

Follow-up: What is callback hell?

Callback hell is deeply nested callback-based code that becomes difficult to read, maintain, and handle errors in. Promises and async/await help make asynchronous code easier to manage.

Q18. What is a Promise?

🔥🔥 Must know.

Answer:

A Promise is an object that represents the eventual completion or failure of an asynchronous operation. It can be in a pending, fulfilled, or rejected state.

Follow-up: What are the states of a Promise?

Pending, fulfilled, and rejected.

Follow-up: How do you handle a Promise?

We can use `.then()` and `.catch()`, or use `async/await` with `try/catch`.

Q19. What is `async/await`?

Answer:

`async/await` is syntax built on top of Promises that makes asynchronous code easier to read and write in a synchronous-looking style.

Example:

```
async function getUser() {  
  try {
```

```
const user = await fetchUser();
console.log(user);
} catch (error) {
  console.log(error);
}
}
```

Follow-up: Does `await` make the entire Node.js application stop?

No. It pauses the execution of that particular async function until the Promise settles; it does not block the entire Node.js Event Loop by itself.

🔥 Very important distinction.

Q20. Callback vs Promise vs async/await?

🔥🔥 Common comparison.

Answer:

Callbacks are functions executed after an operation completes. Promises provide a structured way to represent asynchronous results, and async/await provides cleaner syntax for working with Promises.

Callback
↓
Promise
↓
async/await

Follow-up: Which do you prefer?

For modern Node.js applications, I generally prefer async/await with proper error handling because it makes asynchronous code easier to read and maintain.

🔥 Part 1 — Most Important

Agar Node.js interview se pehle **sirf fundamentals revise** karne ho, ye strongest rakho:

■ MUST KNOW

1. What is Node.js?
2. Node.js vs JavaScript
3. V8
4. Single-threaded nature
5. Non-blocking I/O
6. Blocking vs non-blocking
7. Event Loop
8. Asynchronous programming
9. I/O-intensive applications
10. CPU-intensive tasks
11. Event-driven architecture
12. Callback
13. Promise
14. async/await

■ Important

15. EventEmitter
16. Traditional server-side model comparison

Next Part 2: npm, package.json, package-lock.json, modules, CommonJS vs ES Modules, require vs import, built-in modules, process, environment variables, __dirname, __filename — **without repeating today's questions.**

Hey kuch questions to same h react me bhi and node me bhi to tum koshish kro mujhe vo questions dubara do hi mat jo already hogye h jaise async await promise ye to hogye yrr callback hell vgera bhi

Haan yaar, **bilkul sahi pakda tumne.** 👍

Meri mistake thi — React aur Node ko alag subjects maan kar maine kuch **JavaScript fundamentals** dobara include kar diye.

Ab se ek **strict rule:**

No-Repetition Rule

Jo concept already JavaScript series mein ho chuka hai, **Node/Express mein uska question dobara nahi aayega**, unless Node mein uska **genuinely different Node-specific angle** ho.

Already covered, so **Node series mein dobara nahi karenge**:

- Promise
- async/await
- Callback
- Callback Hell
- Synchronous vs asynchronous
- Event Loop basics
- Hoisting
- Closures
- Scope
- this
- Arrow functions
- Array methods
- Destructuring
- Spread/Rest
- Error handling fundamentals
- ES6 concepts etc.

Aur React mein jo generic JS concepts aa gaye the, unko bhi future Node/Express mein repeat nahi karunga.

Ek aur correction

Node ke Q1–Q20 mein jo generic JS/async questions the, unhe ab final Node list se hata do.

Unko count bhi nahi karenge.

Node ka actual syllabus ab roughly:

Node.js Fundamentals → Modules → npm → Core Modules → process → Environment → File System → Streams → Buffers → HTTP → Express → Middleware → REST API → Authentication → Security → MERN project questions

Aur har question ke saath:

**Main Question → Alternate wording → Interview answer → Follow-up
→ Follow-up answer**

No filler. No duplicate. No 10-year-experience stuff.

Ab next se Node Q1 fresh start hoga, aur sirf **Node-specific questions** honge.

Yes start authentication vgera bhi dekh kr hi dena because cors vgera bhi hogye h already now start jo important questions h vo dedo means mha important and less important bhi mention kr dena bus fresher se jo jo bhi puch skte h sb kuch sirf node not express aur agar sirf 10 hi important questions h to sirf 10 hi do 20 h to 20 do 100 h to 100 do bus unnecessary bilkul mat dena jo tumhe lge ki bilkul important nhi h

Bilkul. Ab **Node.js ko zero se fresh, deduplicated list** mein karenge.

Express, CORS, JWT/authentication implementation abhi Node mein mix nahi karunga; authentication ka Node-specific runtime angle agar genuinely relevant hua tabhi aayega. Generic JS/React/CORS questions repeat nahi honge.

Node.js — Part 1: Core Fundamentals

Priority:

- **Must Know** = interview mein genuinely high chance
 - **Important** = fresher se poocha ja sakta hai
 - **Low Priority** = sirf awareness; abhi skip kar sakti ho
-

Q1. What is Node.js? ●

Also asked as:

- What is Node.js used for?
- Explain Node.js in simple words.
- Is Node.js a language?
- Is Node.js a framework?

Answer:

Node.js is a JavaScript runtime environment that allows us to execute JavaScript outside the browser. It is built on Chrome's V8 JavaScript engine and is commonly used to build backend applications, APIs, and real-time applications.

Follow-up: Is Node.js a programming language?

No. JavaScript is the programming language, and Node.js is the runtime environment that executes JavaScript outside the browser.

Follow-up: Is Node.js a framework?

No. Node.js is a runtime environment. Express.js is a framework commonly used with Node.js.

Q2. Why is Node.js used for backend development?

Also asked as:

- Why did you choose Node.js?
- What are the advantages of Node.js?
- Why is Node.js popular?

Answer:

Node.js allows us to use JavaScript on the backend, has a large npm ecosystem, and is well suited for I/O-heavy applications such as APIs, web applications, and real-time services.

Follow-up: What is the advantage of using JavaScript on both frontend and backend?

It allows developers to use the same programming language across the full stack, which can simplify development and code sharing.

Q3. What is the V8 JavaScript engine? ●

Also asked as:

- Which engine does Node.js use?
- How does Node.js execute JavaScript?

Answer:

V8 is Google's JavaScript engine used by Chrome. Node.js uses V8 to execute JavaScript code outside the browser.

Follow-up: Does V8 provide Node.js APIs like `fs` and `http`?

No. V8 mainly executes JavaScript. Node.js provides additional runtime APIs such as file-system and networking functionality.

Q4. How is Node.js different from JavaScript in the browser? ●

Answer:

JavaScript is the programming language. Node.js provides a runtime for executing JavaScript outside the browser. Browser JavaScript has APIs such as the DOM, while Node.js provides server-side APIs such as file-system and networking APIs.

Follow-up: Can Node.js access `document` directly?

No. `document` is a browser DOM API and is not provided by Node.js.

Q5. Is Node.js single-threaded? ●

🔥 Very commonly asked

Answer:

Node.js executes JavaScript on a main thread, but the Node.js runtime can use the operating system and a worker pool for certain operations. Its architecture allows many I/O operations to be

handled without creating a separate JavaScript thread for every request.

Follow-up: Then how can Node.js handle many requests?

Through its asynchronous I/O architecture. While an I/O operation is waiting, Node.js can continue processing other work instead of blocking the JavaScript thread.

Q6. Why is Node.js called event-driven?

Also asked as:

- What is event-driven architecture in Node?
- Explain Node's event-based model.

Answer:

Node.js uses an event-driven architecture where operations can emit events or complete asynchronously, and registered handlers respond to those events or results. This model is heavily used for handling I/O and network operations.

Follow-up: Where do you see this in Node.js?

EventEmitter, HTTP servers, streams, and many asynchronous APIs use event-based patterns in Node.js.

Q7. What is the Event Loop in Node.js?

 JavaScript mein basic Event Loop already kar chuki ho, so Node interview mein isko sirf Node-specific angle se yaad karo.

Also asked as:

- How does Node handle asynchronous operations?
- How does Node avoid blocking?
- Explain Node's Event Loop.

Answer:

The Event Loop is part of Node's runtime mechanism that allows JavaScript to continue handling work while asynchronous operations are in progress. When an operation is ready to be handled, its callback or continuation can be processed when the JavaScript thread is available.

Follow-up: What happens if we run a long CPU-heavy operation?

It can block the JavaScript thread and delay other callbacks and requests.

Q8. What is the difference between Node.js and a traditional thread-per-request server? 🟡

Answer:

Node.js generally uses an event-driven, non-blocking I/O model rather than creating a separate JavaScript thread for every request. This makes it particularly suitable for applications with many concurrent I/O operations.

Follow-up: Does this mean Node always performs better?

No. Performance depends on the workload and application design. Node.js is particularly suitable for I/O-bound applications.

Q9. Why is Node.js good for I/O-intensive applications? 🔴

Also asked as:

- Why is Node.js suitable for APIs?
- Why is Node.js good for real-time applications?

Answer:

Because Node.js can perform I/O operations asynchronously without blocking the main JavaScript execution. This allows it to efficiently handle many concurrent network, database, and file-related operations.

Follow-up: Give examples of I/O operations.

Database queries, HTTP requests, file operations, network communication, and reading from external services.

Q10. Is Node.js suitable for CPU-intensive applications?

Answer:

Node.js can handle CPU-intensive work, but long CPU-heavy operations on the main JavaScript thread can block the Event Loop. For such workloads, worker threads or separate services can be used.

Follow-up: What is the problem with blocking the Event Loop?

Other requests and callbacks may have to wait, which can make the application slow or unresponsive.

Q11. What is EventEmitter in Node.js?

Also asked as:

- How does Node handle custom events?
- What are emit() and on()?

Answer:

EventEmitter is a Node.js class that allows objects to emit named events and register listener functions that respond to those events.

```
import { EventEmitter } from "events";
```



```
const emitter = new EventEmitter();
```

```
emitter.on("login", () => {  
  console.log("User logged in");  
});
```

```
emitter.emit("login");
```

Follow-up: What does `on()` do?

It registers a listener for an event.

Follow-up: What does `emit()` do?

It triggers the event.

Q12. What are Node.js Core Modules?

Also asked as:

- What built-in modules does Node provide?
- Do we need to install Node's built-in modules?

Answer:

Core modules are modules provided by Node.js itself, so they do not need to be installed separately. They provide functionality such as file handling, networking, paths, events, and operating-system information.

Examples:

```
fs  
path  
http  
events  
os  
crypto  
url
```

Follow-up: Do we install `fs` using npm?

No. It is a built-in Node.js module.

Q13. What is the `fs` module? 📌

Also asked as:

- How do you work with files in Node.js?
- What is File System module?

Answer:

The `fs` module provides APIs for working with the file system, such as reading, writing, updating, and deleting files and directories.

Follow-up: Does `fs` provide synchronous and asynchronous APIs?

Yes. Node provides both asynchronous APIs and synchronous versions for many file-system operations.

Q14. What is the `path` module? 🟡

Answer:

The `path` module provides utilities for working with file and directory paths in a platform-independent way.

Example:

```
import path from "path";  
  
const filePath = path.join("uploads", "image.jpg");
```

Follow-up: Why use `path.join()` instead of manually writing `/`?

It handles path separators appropriately for the operating system.

Q15. What is the `http` module?

Answer:

The `http` module is a built-in Node.js module used to create HTTP servers and make HTTP requests.

Basic idea:

```
import http from "http";

const server = http.createServer((req, res) => {
  res.end("Hello");
});

server.listen(5000);
```

Follow-up: Do we need Express to create an HTTP server?

No. Node's built-in `http` module can create a server directly. Express provides a more convenient framework on top of Node's HTTP capabilities.

Q16. What is `process` in Node.js?

Also asked as:

- What is `process.env`?
- How do you access environment variables in Node?
- How do you get command-line arguments?

Answer:

`process` is a global Node.js object that provides information and control over the current Node.js process. It can be used to access environment variables, command-line arguments, exit codes, and other process-related information.

Example:

```
console.log(process.env.PORT);
```

Q17. What are environment variables? ●

Answer:

Environment variables are configuration values provided to an application through its environment instead of hard-coding them into the source code. They are commonly used for values such as port numbers, database connection strings, and API configuration.

Follow-up: Why shouldn't secrets be hard-coded?

Because source code can be exposed through Git repositories, logs, or deployments. Sensitive credentials should be managed securely through environment or secret-management systems.

Follow-up: Should `.env` be committed to GitHub?

Normally no, especially when it contains secrets. It should be added to `.gitignore`, and the required variable names can be documented in a safe example file.

Q18. What is `npm`? ●

Also asked as:

- What is `npm` in Node.js?
- Why do we use `npm`?
- What is Node Package Manager?

Answer:

`npm` is the standard package manager for Node.js. It is used to install, manage, update, and publish JavaScript packages and also provides scripts for running project commands.

Follow-up: Is `npm` a programming language?

No. It is a package manager and command-line tool.

Q19. What is `package.json`?

 **Extremely common**

Answer:

`package.json` is the main configuration file of a Node.js project. It contains information such as the project name, version, dependencies, scripts, and other project metadata.

Example:

```
{
  "scripts": {
    "start": "node server.js"
  }
}
```

Follow-up: What are scripts?

Scripts are commands defined in `package.json` that can be executed using `npm`, such as `npm start` or `npm run dev`.

Q20. What is `package-lock.json`?

Answer:

`package-lock.json` records the exact dependency versions and dependency tree installed for a project. It helps ensure that the same dependency versions can be installed consistently across different environments.

Follow-up: Should we commit `package-lock.json`?

Yes, for applications it is generally recommended to commit it so installations remain reproducible.

Q21. What is the difference between `dependencies` and `devDependencies`?

Answer:

`dependencies` contain packages required by the application at runtime, while `devDependencies` contain packages mainly needed during development, testing, or building.

Example:

`dependencies`

→ `express`

→ `mongoose`

`devDependencies`

→ `nodemon`

→ testing tools

Follow-up: How do you install a development dependency?

`npm install nodemon --save-dev`

Q22. What is `node_modules`?

Answer:

`node_modules` is the directory where npm installs a project's dependencies and their required dependency packages.

Follow-up: Should `node_modules` be pushed to GitHub?

No. It is normally excluded through `.gitignore` because it can be recreated from `package.json` and the lock file using `npm install` commands.

Q23. What is the difference between `npm install` and `npm ci`? 🟡

Answer:

`npm install` can update the lock file when necessary and is commonly used during development. `npm ci` performs a clean installation based strictly on the lock file and is commonly used in automated or CI environments.

Follow-up: Which is useful for deployment/CI?

`npm ci` is commonly preferred when we want a clean, reproducible installation based on the lock file.

Q24. What is `npx`? 🟡

Answer:

`npx` is a command-line tool that can execute npm packages, including packages that don't need to be installed globally.

Example:

```
npx create-vite my-app
```

Follow-up: `npm` vs `npx`?

`npm` is primarily used to manage/install packages and run scripts, while `npx` is commonly used to execute package binaries.

Q25. What is the difference between local and global npm packages? 🟡

Answer:

A local package is installed inside a particular project and is available to that project. A global package is installed for broader system-level command-line use.

Follow-up: Which should you generally prefer for project dependencies?

Local installation, because the project can control the exact version it depends on.

Part 1 Final Priority

Must Know — 17

Q1, Q2, Q3, Q4, Q5, Q6, Q7, Q9, Q12, Q13, Q15, Q16, Q17, Q18, Q19, Q20, Q21, Q22

Actually **18** — and that's intentional; I'm not cutting a genuinely important question just to make the number look neat.

Important — 7

Q8, Q10, Q11, Q14, Q23, Q24, Q25

Unnecessary for now

None added. Agar koi topic fresher Node interview ke liye genuinely low-value hai, main usko list mein launga hi nahi.

Next: Node Modules — CommonJS vs ES Modules, require, import/export, module caching, built-in/user-defined/third-party modules, exports vs module.exports, aur **sirf Node-specific important questions.**

Aur kitne h questions

Abhi **25 questions complete** hue hain.

Node.js ka pura **fresher-only, non-repeated** set approximately **70–80 questions** ka banega.

-  **25 done**
-  **~45–55 remaining**
-  Express included nahi
-  CORS repeat nahi
-  Promise/async-await/callback repeat nahi

- ❌ Authentication ke generic questions repeat nahi honge
- ❌ Unnecessary 5–10 year experience topics nahi

Main **exact number artificially fix nahi karunga** — agar 70 mein genuinely sab cover ho gaya, to 70 par hi stop. Agar 80 tak genuinely important topics bante hain, tab 80.

Next: Modules → npm ecosystem → Core Node APIs → HTTP → Streams/Buffers → process/runtime → Node-specific security & production basics.

Next

Node.js Interview — Part 2: Modules

Q26–Q38

🔴 = Must Know | 🟡 = Important

No repeat: Promise, async/await, callbacks, CORS, JWT, React concepts etc. nahi laa raha.

Q26. What is a module in Node.js? 🔴

Also asked as:

- What are modules in Node.js?
- Why do we use modules?
- How do you organize code in Node.js?

Answer:

A module is a reusable and self-contained piece of code that can be imported and used in another file. Modules help us organize a Node.js application into smaller and maintainable parts.

Follow-up: Why are modules useful?

They improve code organization, reusability, maintainability, and separation of responsibilities.

Q27. What types of modules are available in Node.js?

Answer:

Node.js modules are commonly categorized as built-in modules, user-defined modules, and third-party modules.

Examples:

Built-in → fs, path, http

User-defined → ./utils.js

Third-party → express, mongoose

Follow-up: Do we install built-in modules?

No. They are provided by Node.js.

Q28. What are CommonJS and ES Modules?

 Very common.

Answer:

CommonJS and ES Modules are two module systems used in Node.js. CommonJS commonly uses require() and module.exports, while ES Modules use import and export.

CommonJS:

```
const fs = require("fs");
```

```
module.exports = myFunction;
```

ES Modules:

```
import fs from "fs";
```

```
export default myFunction;
```

Q29. What is the difference between `require()` and `import`? 

Answer:

`require()` is associated with the CommonJS module system, while `import` belongs to the ES Module system. ES Modules use static `import` and `export` syntax, while CommonJS uses `require()` and `module.exports`.

Follow-up: Which one are you using in your MERN projects?

I use ES Modules when the project is configured for them, for example by using `"type": "module"` in `package.json`.

Q30. What does `"type": "module"` do in `package.json`? 

 Especially relevant because this is a common Node project configuration.

Answer:

When `"type": "module"` is specified in `package.json`, Node treats `.js` files in that package as ES Modules, allowing syntax such as `import` and `export`.

Example:

```
{  
  "type": "module"  
}
```

Then:

```
import express from "express";
```

can be used.

Follow-up: What happens if you use `require()` in an ES Module?

It is not available by default because `require()` belongs to CommonJS. We should use ES Module syntax or explicitly use an appropriate CommonJS setup.

Q31. What is `module.exports`?

Answer:

`module.exports` defines the value that a CommonJS module makes available to other files when they use `require()`.

Example:

```
function add(a, b) {  
  return a + b;  
}
```

```
module.exports = add;
```

Then:

```
const add = require("./add");
```

Q32. What is the difference between `exports` and `module.exports`?

 Tricky but important.

Answer:

`exports` initially references `module.exports`, so properties can be added through `exports`. However, assigning a completely new value to `exports` does not replace `module.exports`.

Example:

```
exports.add = add;
```

works.

But:

```
exports = add;
```

does **not** change what the module exports.

Follow-up: What is safer to use when replacing the entire export?

`module.exports`.

Q33. What is the purpose of `import` and `export`?

Answer:

export makes code available from a module, while import allows another module to use that exported code.

Example:

```
// math.js
export const add = (a, b) => a + b;

// app.js
import { add } from './math.js';
```

Q34. Named export vs default export?

Answer:

A named export exports a specific named value and is imported using the same exported name, while a default export provides the module's primary exported value and can be imported with any local name.

Named:

```
export const add = () => {};
```

```
import { add } from './math.js';
```

Default:

```
export default add;
```

```
import calculate from './math.js';
```

Follow-up: Can one file have multiple named exports?

Yes.

Follow-up: Can one ES Module have multiple default exports?

No. It can have only one default export.

Q35. What is module caching in Node.js? 🟡

Answer:

When a CommonJS module is loaded, Node.js caches the loaded module. If the same module is required again, Node can reuse the cached module instead of executing the module code again from scratch.

Follow-up: Why is caching useful?

It avoids repeatedly loading and executing the same module and helps improve efficiency.

Q36. What is a third-party module? 🔴

Answer:

A third-party module is a package created and published by someone outside the Node.js core project and installed through a package manager such as npm.

Examples:

```
express  
mongoose  
dotenv  
jsonwebtoken
```

Follow-up: Are these modules included with Node.js?

No. They normally need to be installed as project dependencies.

Q37. What is the difference between built-in and third-party modules? ●

Answer:

Built-in modules are provided by Node.js itself and don't require separate installation. Third-party modules are external packages that we install, usually through npm.

Example:

Built-in:

fs
path
http

Third-party:

express
mongoose
dotenv

Q38. What is a user-defined module? ●

Answer:

A user-defined module is a module created by the developer to organize application-specific functionality into reusable files.

Example:

```
// utils.js
export function add(a, b) {
  return a + b;
}
```

Then:

```
import { add } from './utils.js';
```

Follow-up: Why create user-defined modules?

To separate responsibilities and keep the application organized and maintainable.

Part 2 — Priority

Must Know

- Q26 — What is a module?
- Q27 — Types of modules
- Q28 — CommonJS vs ES Modules
- Q29 — require vs import
- Q30 — "type": "module"
- Q31 — module.exports
- Q32 — exports vs module.exports
- Q33 — import / export
- Q36 — Third-party modules
- Q37 — Built-in vs third-party

Important

- Q34 — Named vs default export
- Q35 — Module caching
- Q38 — User-defined modules

38 total done.

Next part mein **Node Core APIs + process + environment/configuration + path/url/os/crypto** ke genuinely interview-relevant questions karenge.

Next

Node.js Interview — Part 3: Core APIs & Runtime

Q39–Q51

 Must Know |  Important

No repeats from JavaScript/React/CORS/previous Node parts.

Q39. What is the `process` object in Node.js?

Also asked as:

- What is `process` in Node?
- How can you access information about the running Node application?

Answer:

`process` is a global Node.js object that provides information and control over the currently running Node.js process. It can be used to access environment variables, command-line arguments, the current working directory, and process-related information.

Follow-up: Do we need to import `process`?

No. It is available globally in Node.js.

Q40. What is `process.env`?

Answer:

`process.env` is an object containing the environment variables available to the Node.js process. It is commonly used for configuration such as ports, database URLs, and API credentials.

Example:

```
const port = process.env.PORT;
```

Follow-up: Are environment variable values automatically numbers?

No. Environment variables are generally received as strings, so numeric values may need to be converted.

Q41. What is `process.argv`? 🟡

Answer:

`process.argv` contains the command-line arguments passed to a Node.js process.

For example:

```
node app.js hello
```

The argument can be accessed through `process.argv`.

Follow-up: Where is this useful?

It can be used for command-line tools and scripts that accept input from the terminal.

Q42. What is `process.cwd()`? 🟡

Answer:

`process.cwd()` returns the current working directory from which the Node.js process was started.

Example:

```
console.log(process.cwd());
```

Follow-up: Is `process.cwd()` always the same as the directory containing the current JavaScript file?

No. The working directory depends on where the process was started, while the file's directory is based on the module's location.

Q43. What is `__dirname`? 🟡

Answer:

In CommonJS, `__dirname` contains the absolute path of the directory containing the current module.

Example:

```
console.log(__dirname);
```

Follow-up: Is `__dirname` available directly in ES Modules?

No. It is a CommonJS-specific variable. In ES Modules, the module URL can be used to derive the current file's location.

Q44. What is the difference between `process.cwd()` and `__dirname`? 🔴

🔥 Good interviewer follow-up.

Answer:

`process.cwd()` gives the current working directory of the running process, while `__dirname` in CommonJS refers to the directory of the current module file.

Simple example:

```
process.cwd()
```

→ Where the application was started

```
__dirname
```

→ Where the current file is located

Q45. What is the `os` module? 🟡

Answer:

The `os` module provides operating-system-related information such as CPU information, memory information, platform, hostname, and home directory.

Example:

```
import os from "os";

console.log(os.platform());
console.log(os.cpus());
```

Follow-up: Is `os` a third-party package?

No. It is a built-in Node.js module.

Q46. What is the `url` module? 🟡

Answer:

The `url` module provides utilities for parsing, constructing, and working with URLs.

It can help work with things such as:

```
protocol
hostname
pathname
query parameters
```

Follow-up: Is URL handling also available through a global API?

Yes. Modern Node.js provides the standard URL API, which is commonly used for URL manipulation.

Q47. What is the `crypto` module? 🔴

🔥 Important for MERN backend interviews.

Answer:

The crypto module provides cryptographic functionality in Node.js, including hashing, encryption-related operations, random data generation, and digital signatures.

Follow-up: Is hashing the same as encryption?

No. Hashing is designed as a one-way transformation, while encryption is designed so that authorized users can decrypt the data back into its original form.

Follow-up: Should passwords be encrypted before storing them?

Passwords should generally be hashed using an appropriate password-hashing algorithm rather than stored using reversible encryption.

Q48. What is hashing?

Answer:

Hashing converts data into a fixed-length representation using a hash function. A properly designed password-hashing system is one-way and is used to store passwords without storing the original password.

Follow-up: Can you get the original password directly from a hash?

No. A secure password hash is designed to be one-way. During login, the submitted password is checked against the stored hash using the password-hashing algorithm.

Q49. Why should passwords never be stored as plain text?

 Very common security question.

Answer:

If plain-text passwords are stored and the database is compromised, attackers can immediately obtain users' passwords. Passwords should instead be stored as secure password hashes.

Follow-up: Do we hash the password again during login?

The entered password is processed using the same password-hashing mechanism and compared securely with the stored hash.

Q50. What is a Buffer in Node.js? 🟡

Answer:

A Buffer is a Node.js data structure used to work with raw binary data. It is useful when handling things such as files, network data, and streams.

Example:

```
const buffer = Buffer.from("Hello");
```

```
console.log(buffer);
```

Follow-up: Why does Node need Buffers?

Because many operations involve binary data rather than JavaScript strings, such as reading files or receiving network data.

Q51. What are Streams in Node.js? 🔴

🔥 Very important.

Also asked as:

- Why are streams used in Node?
- What is streaming data?
- How can Node handle large files efficiently?

Answer:

Streams allow Node.js to process data gradually instead of loading the entire data into memory at once. They are useful for large files, network communication, and other continuous data sources.

Example:

```
Large file
  ↓
small chunks
  ↓
process gradually
```

Follow-up: Why are streams useful for large files?

They can reduce memory usage because the entire file does not have to be loaded into memory at once.

Q52. What are the types of streams in Node.js?

Answer:

Node.js has four main types of streams:

```
Readable → read data
Writable → write data
Duplex   → read + write
Transform → read + modify + write
```

Follow-up: Give an example of a Transform stream.

Compression or decompression is a common example because data can be transformed while it is being read and written.

Q53. What is a Pipe in Node.js? 🟡

Answer:

pipe() connects a readable stream to a writable stream so that data can flow from one to the other efficiently.

Example:

```
readStream.pipe(writeStream);
```

Follow-up: Why is piping useful?

It allows data to be processed as a stream rather than loading the entire source into memory first.

🎯 Part 3 Priority

🔴 MUST KNOW

Q39 process
Q40 process.env
Q44 cwd() vs __dirname
Q47 crypto
Q48 hashing
Q49 password storage
Q51 streams
Q52 stream types

🟡 IMPORTANT

Q41 process.argv
Q42 process.cwd()
Q43 __dirname
Q45 os
Q46 url
Q50 Buffer
Q53 pipe

53 questions completed.

Next: **Node HTTP + server fundamentals** — request/response, headers, status codes, HTTP server, routing at the Node level, JSON handling, request body, MIME/content type, etc. **Express abhi nahi.**

Next

Node.js Interview — Part 4: HTTP & Server Fundamentals

Q54–Q68

 Must Know |  Important

Express abhi nahi. Sirf Node.js ke HTTP concepts.

Q54. How do you create an HTTP server in Node.js?

Also asked as:

- How can Node.js create a web server?
- Can Node create a server without Express?

Answer:

Node.js provides the built-in http module, which can be used to create an HTTP server. The server receives a request and sends a response through a callback.

```
```js id="3wq7lm" import http from "http";  

const server = http.createServer((req, res) => { res.end("Hello World"); });

server.listen(5000);
```

### Follow-up: Do you need Express to create a Node server?

> **\*\*No.\*\*** Node's built-in `http` module can create a server without Express.

---

# Q55. What are `req` and `res` in a Node HTTP server? ●

### Answer:

> **\*\*`req` represents the incoming HTTP request, while `res` represents the response that the server sends back to the client.\*\***

```
``js id="u6v8zz"
http.createServer((req, res) => {
 console.log(req.method);
 console.log(req.url);

 res.end("Response");
});
```

Follow-up: What information can you get from `req`?

**Request method, URL, headers, and other request information.**

---

Q56. What is an HTTP request? ●

Answer:

**An HTTP request is a message sent by a client to a server to request a resource or perform an operation. It contains information such as the HTTP method, URL, headers, and sometimes a request body.**

Example:

text id="o9a7ll" POST /api/users Headers Body

---

Q57. What is an HTTP response? ●

Answer:

**An HTTP response is the message sent by the server back to the client after processing a request. It contains a status code, headers, and optionally a response body.**

Example:

```
``text id="e7g1xw" 200 OK Content-Type: application/json
{ "name": "John" }
```

---

# Q58. What are HTTP headers? ●

**\*\*Also asked as:\*\***

- What is the purpose of request headers?
- What are response headers?

### Answer:

> **\*\*HTTP headers carry additional information about a request or response, such as content type, authorization information, caching instructions, and client or server details.\*\***

Example:

```
``text id="f9v6zj"
Content-Type: application/json
Authorization: Bearer token
```

**Follow-up: Are headers part of the request body?**

**No.** Headers and body are separate parts of an HTTP message.

---

**Q59. What is the HTTP request body? ●**

**Answer:**

**The request body contains data sent by the client to the server, commonly with methods such as POST, PUT, or PATCH.**

Example:

```
json id="p0o5x2" { "name": "Pranjal", "email": "user@example.com" }
```

Follow-up: Is every HTTP request required to have a body?

**No.** A request body is optional and depends on the request.

---

## Q60. What is Content-Type?

Answer:

**Content-Type is an HTTP header that tells the receiver what type of data is contained in the request or response body.**

Examples:

text id="h8q9nf" application/json text/html text/plain multipart/form-data

Follow-up: What Content-Type is commonly used for JSON APIs?

**application/json.**

---

## Q61. What are HTTP status codes?

Answer:

**HTTP status codes indicate the result of an HTTP request. They are grouped into categories based on the first digit.**

text id="2j4i7u" 1xx → Informational 2xx → Success 3xx → Redirection 4xx → Client error  
5xx → Server error

---

## Q62. What is the difference between 2xx, 4xx and 5xx?

Answer:

**2xx indicates successful processing, 4xx generally indicates a problem with the client request, and 5xx indicates that the server encountered a problem while processing the request.**

---

## Q63. What is the difference between 401 and 403? 📌

🔥🔥 Very commonly asked in MERN

Answer:

**401 means the request does not have valid authentication credentials, while 403 means the server understands the request but refuses to authorize access to the requested resource.**

Easy way:

``text id="yrn8fl" 401 → "You are not properly authenticated."

403 → "You are authenticated, but not allowed."

### Follow-up: Give an example.

> A user without valid authentication may receive **\*\*401\*\***. An authenticated normal user trying to access an admin-only resource may receive **\*\*403\*\***.

---

# Q64. What is the difference between 400 and 500? 📌

### Answer:

> **\*\*400** indicates that the server cannot process the request because the client sent an invalid request. **500** indicates an unexpected server-side error occurred while processing a valid request.\*\*

Example:

``text id="i4ks4r"

400 → Invalid request data

500 → Unexpected server error

---

## Q65. What are some commonly used HTTP status codes?

Answer:

You should know these:

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
500	Internal Server Error

Follow-up: When would you use 201?

**When a request successfully creates a new resource.**

---

## Q66. What is routing in a Node HTTP server?

Answer:

**Routing means determining how the server should respond based on the request's URL and HTTP method.**

For example:

text id="h0v6jc" /users + GET /users + POST /products + GET

The server can handle each combination differently.

Follow-up: Can basic routing be implemented without Express?

**Yes.** With Node's `http` module, we can inspect `req.url` and `req.method` and decide how to respond.

---

## Q67. What is JSON and why is it commonly used in APIs? 📌

Answer:

**JSON stands for JavaScript Object Notation. It is a lightweight text-based data format commonly used for exchanging structured data between clients and servers.**

Example:

```
json id="2e7y2n" { "name": "Pranjal", "role": "developer" }
```

Follow-up: Is JSON the same as a JavaScript object?

**No.** A JavaScript object is an in-memory data structure, while JSON is a text-based data format.

---

## Q68. How do you send a JSON response using Node's HTTP module? 🟡

Answer:

**We set the appropriate Content-Type header and serialize the JavaScript data into JSON using `JSON.stringify()` before sending it in the response.**

```
``js id="v5b2u8" const user = { name: "Pranjal" };
res.writeHead(200, { "Content-Type": "application/json" });
res.end(JSON.stringify(user));
```

### Follow-up: Why do we use `JSON.stringify()`?

> **\*\*Because HTTP sends the response body as data, and `JSON.stringify()` converts the JavaScript object into a JSON string.\*\***

---

## # 🎯 Part 4 Priority

### ### 🔴 MUST KNOW

- Q54 — Creating HTTP server
- Q55 — `req` and `res`
- Q56 — HTTP request
- Q57 — HTTP response
- Q58 — HTTP headers
- Q59 — Request body
- Q60 — Content-Type
- Q61 — Status codes
- Q62 — 2xx/4xx/5xx
- Q63 — 401 vs 403
- Q64 — 400 vs 500
- Q65 — Common status codes
- Q67 — JSON

### ### 🟡 IMPORTANT

- Q66 — Node-level routing
- Q68 — Sending JSON response

**\*\*68 questions done.\*\***

Next is the **\*\*last major Node-only area before Express: File System + Streams + performance/concurrency + child processes/worker threads\*\***. I'll keep only the fresher-relevant ones; no obscure Node internals.

User Prompt:
Next

## # 🚀 Node.js Interview — Part 5: Performance, Concurrency & Advanced Basics



**\*\*Q69–Q80\*\***

🔴 Must Know | 🟡 Important

Ab hum **\*\*Node-only\*\*** concepts kar rahe hain. JS/React ke already-done concepts repeat nahi honge.

---

**## Q69. What is concurrency in Node.js?** 🔴

**\*\*Also asked as:\*\***

- How does Node.js handle multiple tasks?
- How can Node handle many requests with one main thread?

**### Answer:**

> **\*\*Concurrency means Node.js can make progress on multiple tasks during overlapping periods. It achieves this mainly through asynchronous I/O and the Event Loop, allowing it to handle other work while I/O operations are waiting.\*\***

**### Follow-up: Is concurrency the same as parallelism?**

> **\*\*No.\*\*** Concurrency means managing multiple tasks in overlapping time, while parallelism means executing multiple tasks at the same time, typically using multiple CPU cores or threads.

---

**# Q70. What is the difference between concurrency and parallelism?** 🔴

**### Answer:**

> **\*\*Concurrency is about handling multiple tasks by switching or progressing between them, while parallelism means actually executing multiple tasks simultaneously.\*\***

Simple:

``text id="8rq3os"

Concurrency → Multiple tasks in progress  
Parallelism → Multiple tasks executing simultaneously

---

## Q71. What are Worker Threads in Node.js? 🟡

**Also asked as:**

- How can Node.js perform CPU-intensive work?
- How can you use multiple threads in Node?

**Answer:**

**Worker Threads allow JavaScript code to run in separate threads. They can be used for CPU-intensive tasks so that heavy computation does not block the main JavaScript thread.**

**Follow-up: When would you use Worker Threads?**

**For CPU-intensive operations such as heavy calculations, image processing, or other computational work that could block the main thread.**

---

## Q72. What is the difference between the main thread and Worker Threads? 🟡

**Answer:**

**The main thread handles normal JavaScript execution and Event Loop activity, while Worker Threads provide separate threads for executing CPU-intensive JavaScript work.**

**Follow-up: Should every task use Worker Threads?**

**No. They add complexity and are mainly useful when CPU-heavy work could block the main thread.**

---

## Q73. What is the `cluster` module in Node.js? 🟡

Answer:

**The Cluster module allows a Node.js application to create multiple processes that can run across CPU cores and share a server port, helping applications make better use of multi-core systems.**

Follow-up: Are cluster workers threads?

**No.** Cluster workers are separate Node.js processes, unlike Worker Threads, which use threads.

---

## Q74. Worker Threads vs Cluster? 🟡

🔥 Good comparison question.

Answer:

**Worker Threads use separate threads within a Node.js process and are useful for CPU-intensive JavaScript work. Cluster creates separate Node.js processes and is commonly used to scale server workloads across CPU cores.**

Easy:

Worker Threads → threads → CPU-heavy tasks

Cluster → processes → scale server across CPU cores

---

# Q75. What is a child process in Node.js? 🟡

### Answer:

> **\*\*A child process is a separate process created by a Node.js application to run another program or command independently. Node.js provides the `child_process` module for this purpose.\*\***

### Follow-up: Why would you create a child process?

> **To run external commands or programs separately from the main Node.js process.**

---

# Q76. What is the `child\_process` module? 🟡

### Answer:

> **The `child\_process` module provides APIs for creating and interacting with child processes from a Node.js application.**

Common methods include:

```
```text id="w1y6xv"
exec()
spawn()
fork()
execFile()
```

Follow-up: Do you need to memorize every method for a fresher interview?

No. You should understand what the module is used for and know the basic purpose of the commonly used methods.

Q77. What is memory leak in Node.js? 🟡

Answer:

A memory leak occurs when an application unintentionally keeps references to objects that are no longer needed, preventing the memory from being reclaimed. Over time, this can cause memory usage to grow and potentially crash the application.

Follow-up: How can memory leaks affect a Node application?

They can increase memory consumption, reduce performance, and eventually cause the process to run out of memory.

Q78. What is garbage collection in Node.js? 🟡

Answer:

Garbage collection automatically identifies objects that are no longer reachable and reclaims the memory they occupy. Node.js relies on the V8 engine's garbage collector for JavaScript memory management.

Follow-up: Can developers manually delete JavaScript objects from memory?

We generally don't manually free JavaScript memory. We remove unnecessary references and let the garbage collector reclaim unreachable objects.

Q79. What is a graceful shutdown in Node.js? 🟡

🔥 Practical interview question.

Also asked as:

- How should a Node server shut down?
- What happens when you stop a production Node server?

Answer:

A graceful shutdown means allowing the application to stop safely by handling termination signals, stopping new work, completing or cleaning up ongoing operations where appropriate, closing resources such as server connections, and then exiting.

Follow-up: Why is graceful shutdown important?

It helps prevent incomplete requests, lost data, and improperly closed resources.

Q80. How can you handle uncaught errors in a Node.js application?

Answer:

Errors should primarily be handled close to where they occur using appropriate error-handling mechanisms. Node.js also provides process-level events such as `uncaughtException` and `unhandledRejection` for detecting serious unhandled failures, but these should not replace proper error handling.

Follow-up: Should you simply continue running after an `uncaughtException`?

Generally, an uncaught exception indicates an unexpected application state. In production, the process is often restarted after performing necessary cleanup rather than assuming the application is still completely safe.

Part 5 Priority

MUST KNOW

- Q69 — Concurrency
- Q70 — Concurrency vs parallelism
- Q80 — Uncaught errors

IMPORTANT

- Q71 — Worker Threads
- Q72 — Main thread vs Worker Threads
- Q73 — Cluster
- Q74 — Worker Threads vs Cluster
- Q75 — Child process
- Q76 — `child_process`
- Q77 — Memory leak
- Q78 — Garbage collection
- Q79 — Graceful shutdown

80 questions completed.

Ab **Node.js** ka **genuinely important fresher portion almost complete** hai. Next mein main **Node-specific security + production/configuration questions** karunga, aur uske baad agar koi remaining Node topic genuinely interview-worthy nahi hua, **wahin Node stop kar denge** — sirf number badhane ke liye questions nahi banaunga.

Tum kya krre ho itne saare questions nhi krane bus jo important hai vo krane the itne to practically possible bhi nhi h yaad krne you fool 50 krado genuinely important

😂 Haan yaar, **meri galti**. Tumne clearly bola tha *sirf genuinely important*, aur maine unnecessary depth mein le gaya.

Ab **Node.js** = **exactly 50 genuinely important fresher interview questions**. Jo 80 mein extra the, **delete/ignore**. Unhe yaad karne ki zarurat nahi.

Final Node.js Target — 50

Tumhare already covered JS/React concepts ko hata kar:

1. What is Node.js?
2. Node.js vs JavaScript
3. Node.js vs framework
4. V8 Engine
5. Why Node.js for backend?
6. Is Node.js single-threaded?
7. Event-driven architecture
8. Node.js Event Loop — Node-specific
9. Non-blocking I/O
10. I/O-bound vs CPU-bound
11. Core modules
12. Built-in vs third-party modules
13. User-defined modules
14. CommonJS vs ES Modules
15. require vs import
16. module.exports vs exports
17. "type": "module"

18. npm
19. package.json
20. package-lock.json
21. dependencies vs devDependencies
22. node_modules
23. npm install vs npm ci
24. npx
25. process
26. process.env
27. Environment variables
28. process.argv
29. process.cwd() vs __dirname
30. fs module
31. path module
32. crypto module
33. Hashing & password storage
34. Buffer
35. Streams
36. Types of streams
37. HTTP module
38. Creating HTTP server
39. Request vs response
40. HTTP headers
41. HTTP status codes
42. 401 vs 403
43. 400 vs 500
44. JSON in Node APIs
45. Node routing without Express
46. Concurrency vs parallelism
47. Worker Threads — basic understanding
48. Memory leak — basic understanding
49. Graceful shutdown
50. Error handling / process-level failures

✗ Ab kya nahi karenge

- Child process ke 5 alag questions ✗

- Cluster ke unnecessary details ✗
- Deep V8 internals ✗
- Advanced streams internals ✗
- obscure Node APIs ✗
- 10-year-experience architecture ✗
- Same Promise/async/await/callback questions ✗
- CORS repeat ✗
- Express ko Node mein ghusana ✗

50 = FINAL.

Aur tumhe **50 ke 50 ratne ki bhi zarurat nahi** — ● Must Know aur ● Important priority ke saath karenge.

Ab hum **Q1 se clean final 50-series** karenge, taaki koi confusion na rahe.

Start 25 25 krke dedo

Bilkul. **Final Node.js = 50 questions**, aur ab **25 + 25** mein karenge.
Neeche **Part 1 ke Q1–Q25** hain. Jo JS/React mein already ho chuka hai, usko intentionally repeat nahi kar raha.

NODE.JS — PART 1/2

Q1–Q25 | ● Must Know | ● Important

Q1. What is Node.js? ●

Also asked as: What is Node.js used for? / Is Node.js a language?

Node.js is a JavaScript runtime environment that allows us to run JavaScript outside the browser. It is built on Chrome's V8 engine and is commonly used to build backend applications and APIs.

Follow-up: Is Node.js a programming language?

No. JavaScript is the programming language, while Node.js is the runtime environment.

Q2. Why is Node.js used for backend development? ●

Node.js allows us to use JavaScript on the backend, has a large npm ecosystem, and its asynchronous I/O model makes it suitable for applications that handle many I/O operations.

Follow-up: Why is it useful for a MERN developer?

Because JavaScript can be used on both the frontend and backend.

Q3. What is the V8 Engine? ●

Also asked as: Which JavaScript engine does Node.js use?

V8 is Google's JavaScript engine used by Chrome. Node.js uses V8 to execute JavaScript code outside the browser.

Follow-up: Does V8 provide Node's fs and http modules?

No. V8 executes JavaScript, while Node.js provides additional APIs such as file-system and networking APIs.

Q4. Is Node.js a framework? ●

No. Node.js is a runtime environment. Express.js is a framework commonly used with Node.js.

Q5. How is Node.js different from JavaScript in the browser? ●

JavaScript is the language, while Node.js provides a runtime for executing JavaScript outside the browser. Browser JavaScript provides APIs such as the DOM, while Node.js provides server-side APIs such as file-system and networking APIs.

Follow-up: Can Node.js directly access the DOM?

No. The DOM is a browser API.

Q6. Is Node.js single-threaded? ●

JavaScript execution in Node.js runs on a main thread, while the runtime can use the operating system and worker mechanisms for certain operations. This allows Node.js to handle many I/O operations efficiently without creating a separate JavaScript thread for every request.

Follow-up: Then how does Node handle multiple requests?

It uses asynchronous, non-blocking I/O so the main JavaScript thread does not have to wait for every I/O operation to finish.

Q7. What is non-blocking I/O in Node.js? ●

Non-blocking I/O means Node.js can continue processing other work while an I/O operation, such as a database or file operation, is in progress.

Follow-up: Why is this useful?

It allows Node.js to handle many I/O operations efficiently without blocking the main JavaScript execution.

Q8. Why is Node.js called event-driven? ●

Node.js uses an event-driven architecture where events or completed operations trigger corresponding handlers. This model is widely used for asynchronous I/O and network operations.

Follow-up: Name a Node feature based on events.

EventEmitter is a common example.

Q9. What is the Event Loop in Node.js? ●

The Event Loop is a mechanism that allows Node.js to process asynchronous operations and their callbacks while continuing to handle other work on the main JavaScript thread.

Follow-up: What happens if we run a very long CPU-heavy task?

It can block the main thread and delay other callbacks and requests.

Q10. Why is Node.js suitable for I/O-intensive applications? ●

Also asked as: Why is Node.js good for APIs?

Because Node.js uses non-blocking I/O, so operations such as database queries and network requests can be handled without blocking the main JavaScript execution.

Follow-up: Give examples of I/O operations.

Database queries, file operations, HTTP requests, and network communication.

Q11. What are Node.js Core Modules? ●

Core modules are modules provided by Node.js itself. They provide functionality such as file handling, networking, paths, events, and cryptography.

Examples:

```
fs
path
http
events
crypto
os
```

Follow-up: Do we install them using npm?

No. They are built into Node.js.

Q12. What types of modules are used in Node.js? ●

The main categories are built-in modules, user-defined modules, and third-party modules.

Example:

Built-in → fs
User-defined → ./utils.js
Third-party → express

Q13. What is the difference between built-in and third-party modules? ●

Built-in modules are provided by Node.js and don't need separate installation. Third-party modules are external packages that we normally install using npm.

Follow-up: Give examples.

fs and http are built-in, while Express and Mongoose are third-party packages.

Q14. What are CommonJS and ES Modules? ●

They are two module systems used in Node.js. CommonJS commonly uses `require()` and `module.exports`, while ES Modules use `import` and `export`.

CommonJS:

```
const fs = require("fs");
```

ES Module:

```
import fs from "fs";
```

Q15. What is the difference between `require()` and `import`? ●

`require()` is used with CommonJS, while `import` is used with ES Modules. Modern Node.js supports both module systems depending on the project configuration.

Follow-up: Which one do you use in an ES Module project?

I use `import` and `export`.

Q16. What is `module.exports`?

`module.exports` defines what a CommonJS module makes available to other files when it is imported using `require()`.

Example:

```
function add(a, b) {  
  return a + b;  
}
```

```
module.exports = add;
```

Q17. What is the difference between `exports` and `module.exports`?

Initially, `exports` references `module.exports`, so we can add properties through `exports`. But assigning a completely new value to `exports` does not replace `module.exports`.

Follow-up: Which should you use when replacing the complete export?

`module.exports`.

Q18. What does `"type": "module"` mean in `package.json`?

It tells Node.js to treat `.js` files in that package as ES Modules, allowing us to use `import` and `export` syntax.

Example:

```
{  
  "type": "module"  
}
```

Follow-up: What happens if you use `require()` in an ES Module?

`require()` is not available by default because it belongs to CommonJS.

Q19. What is npm? ●

Also asked as: What is Node Package Manager?

npm is the standard package manager for Node.js. It is used to install, manage, update, and publish packages and to run project scripts.

Follow-up: Is npm a programming language?

No. It is a package manager and command-line tool.

Q20. What is package.json? ●

🔥 Very common.

package.json is the main configuration file of a Node.js project. It contains information such as project metadata, dependencies, scripts, and project configuration.

Follow-up: What are npm scripts?

They are commands defined inside package.json that we can execute using npm.

Example:

```
"scripts": {  
  "start": "node server.js"  
}
```

Q21. What is package-lock.json? ●

package-lock.json records the exact versions and dependency tree installed for the project. It helps ensure consistent dependency installation across different environments.

Follow-up: Should it be pushed to GitHub?

Yes. For applications, it is generally recommended to commit the lock file.

Q22. What is the difference between `dependencies` and `devDependencies`?

dependencies contain packages required by the application at runtime, while **devDependencies** contain packages mainly required during development, testing, or building.

Example:

`dependencies`

→ `express`

→ `mongoose`

`devDependencies`

→ `nodemon`

Follow-up: How do you install a development dependency?

```
npm install nodemon --save-dev
```

Q23. What is `node_modules`?

`node_modules` is the directory where npm installs the project's dependencies and their required packages.

Follow-up: Should we push `node_modules` to GitHub?

No. It is normally added to `.gitignore` because it can be recreated from the project's dependency files.

Q24. What is the difference between `npm install` and `npm ci`?

`npm install` installs dependencies and can update the lock file when necessary, while **`npm ci`** performs a clean installation based on the existing lock file and is commonly used in CI or deployment environments.

Follow-up: Which is preferred for reproducible CI installations?

`npm ci`.

Q25. What is `npx`? 🟡

`npx` is used to execute package binaries, including packages that don't need to be installed globally.

Example:

```
npx create-vite my-app
```

Follow-up: `npm` vs `npx`?

`npm` is mainly used to manage and install packages, while `npx` is commonly used to execute package commands.

🔥 PART 1 — Priority Revision

🔴 Must absolutely prepare

Q1–Q23

🟡 Prepare after that

Q24–Q25

याद रखने का तरीका

Node basics → Modules → `npm` → Project files

बस इतना Part 1 में hai.

Next 25 mein Q26–Q50: process, .env, fs, path, crypto, HTTP server, requests/responses, status codes, streams, Buffer, concurrency, Worker Threads, memory leaks, graceful shutdown etc. **Sirf final 25 — uske baad Node complete.**

Next

Node.js — Part 2/2

Q26–Q50 | Final 25 Questions

Ab Node.js ke exactly 50 questions complete karenge. No extra questions after this.

🔴 Must Know | 🟡 Important

Q26. What is the `process` object in Node.js? 🔴

Also asked as: What is process in Node.js?

process is a global Node.js object that provides information and control over the currently running Node.js process.

Follow-up: Do we need to import it?

No. It is available globally.

Q27. What is `process.env`? 🔴

Also asked as: How do you access environment variables in Node.js?

process.env provides access to environment variables available to the Node.js process. It is commonly used for configuration such as ports, database URLs, and API keys.

Follow-up: Are environment variables always numbers?

No. They are generally received as strings.

Q28. Why do we use environment variables? 🔴

Environment variables keep configuration and sensitive values outside the source code. They are commonly used for database credentials, API keys, ports, and other environment-specific settings.

Follow-up: Should `.env` containing secrets be pushed to GitHub?

No. Sensitive `.env` files should normally be excluded using `.gitignore`.

Q29. What is `process.argv`? 🟡

`process.argv` contains the command-line arguments passed to a Node.js process.

Example:

```
node app.js hello
```

We can access command-line arguments through `process.argv`.

Follow-up: Where is it useful?

It is useful when creating command-line tools or scripts that accept terminal input.

Q30. What is the difference between `process.cwd()` and `__dirname`? 🔴

🔥 Very good follow-up question.

`process.cwd()` returns the current working directory from which the Node process was started, while `__dirname` in CommonJS refers to the directory containing the current module.

Simple:

```
process.cwd()
```

→ Where the process was started

```
__dirname
```

→ Where the current file is located

Q31. What is the `fs` module? 🔴

Also asked as: How do you work with files in Node.js?

The `fs` or File System module provides APIs for reading, writing, updating, deleting, and working with files and directories.

Follow-up: Does fs support synchronous and asynchronous operations?

Yes, it provides both.

Q32. What is the `path` module? 🟡

The path module provides utilities for working with file and directory paths in a platform-independent way.

Example:

```
import path from "path";  
  
const filePath = path.join("uploads", "image.jpg");
```

Follow-up: Why use path.join()?

It creates paths using the correct path separator for the operating system.

Q33. What is the `crypto` module? 🔴

🔥 Important for MERN backend.

The crypto module provides cryptographic functionality in Node.js, including hashing, random data generation, encryption-related operations, and digital signatures.

Follow-up: Is hashing the same as encryption?

No. Hashing is generally one-way, while encryption is designed to be reversible using the appropriate key.

Q34. How should passwords be stored in a Node.js application? 🔴

🔥🔥 Very common.

Passwords should never be stored as plain text. They should be processed using a secure password-hashing algorithm and only the resulting hash should be stored in the database.

Follow-up: Why not store plain passwords?

If the database is compromised, attackers could immediately obtain users' passwords.

Q35. What is a Buffer in Node.js? 🟡

A Buffer is a Node.js object used to work with raw binary data. It is commonly useful when handling files, streams, and network data.

Example:

```
const buffer = Buffer.from("Hello");
```

Follow-up: Why does Node need Buffers?

Because many operations involve binary data rather than normal JavaScript strings.

Q36. What are Streams in Node.js? 🔴

🔥 Important.

Streams allow Node.js to process data gradually in chunks instead of loading the entire data into memory at once.

Follow-up: Why are streams useful for large files?

They can reduce memory usage because the complete file does not need to be loaded into memory at once.

Q37. What are the four types of Streams? 🟡

Node.js has four main types of streams:

Readable → reads data
Writable → writes data
Duplex → reads and writes
Transform → reads, transforms, and writes

Follow-up: Give an example of Transform stream.

Compression and decompression are common examples.

Q38. What is the HTTP module in Node.js?

The http module is a built-in Node.js module used to create HTTP servers and make HTTP requests.

Follow-up: Can we create a server without Express?

Yes. Node's built-in http module can create a server directly.

Q39. How do you create an HTTP server in Node.js?

We can use `http.createServer()` and provide a function that handles incoming requests and sends responses.

```
import http from "http";

const server = http.createServer((req, res) => {
  res.end("Hello");
});

server.listen(5000);
```

Follow-up: What are req and res?

req represents the incoming request and res represents the response sent by the server.

Q40. What is the difference between an HTTP request and response? ●

A request is sent by the client to the server, while a response is sent by the server back to the client after processing the request.

Request can contain:

Method
URL
Headers
Body

Response can contain:

Status code
Headers
Body

Q41. What are HTTP headers? ●

HTTP headers contain additional information about a request or response, such as content type, authorization information, caching instructions, and other metadata.

Example:

Content-Type: application/json
Authorization: Bearer token

Follow-up: Are headers part of the body?

No. Headers and body are separate parts of an HTTP message.

Q42. What are HTTP status codes? ●

HTTP status codes indicate the result of an HTTP request.

2xx → Success
3xx → Redirection

4xx → Client error

5xx → Server error

Follow-up: Give common examples.

200, 201, 400, 401, 403, 404, 409, and 500.

Q43. What is the difference between 401 and 403?

  Must know for MERN.

401 means the request does not have valid authentication credentials, while 403 means the user is authenticated but is not allowed to access the requested resource.

Easy:

401 → Not properly authenticated

403 → Authenticated but not authorized

Q44. What is the difference between 400 and 500?

400 indicates that the client sent an invalid request, while 500 indicates an unexpected error occurred on the server.

Example:

400 → Invalid input/request

500 → Server-side failure

Q45. What is JSON and why is it used in APIs?

JSON stands for JavaScript Object Notation. It is a lightweight text-based format commonly used to exchange structured data between clients and servers.

Example:

```
{  
  "name": "Pranjal",  
  "role": "developer"  
}
```

Follow-up: Is JSON exactly the same as a JavaScript object?

No. A JavaScript object is an in-memory data structure, while JSON is a text-based data format.

Q46. What is concurrency in Node.js?

Also asked as: How does Node.js handle many requests?

Concurrency means handling multiple tasks whose execution overlaps in time. Node.js achieves this mainly through asynchronous I/O and its Event Loop, allowing it to handle other work while I/O operations are waiting.

Follow-up: Is concurrency the same as parallelism?

No. Concurrency means managing multiple tasks in overlapping periods, while parallelism means executing multiple tasks at the same time.

Q47. What are Worker Threads in Node.js?

Worker Threads allow JavaScript code to run in separate threads. They can be used for CPU-intensive tasks so that heavy computation does not block the main JavaScript thread.

Follow-up: When would you use them?

For CPU-heavy operations such as complex calculations or intensive data processing.

Q48. What is a memory leak in Node.js?

A memory leak occurs when an application unintentionally keeps references to objects that are no longer needed, preventing their memory from being reclaimed.

Follow-up: What can happen because of a memory leak?

Memory usage can continuously increase, causing performance problems and potentially crashing the application.

Q49. What is graceful shutdown in Node.js?

Also asked as: How should you safely shut down a Node server?

Graceful shutdown means safely stopping the application by handling termination signals, finishing or cleaning up necessary work, closing resources, and then exiting the process.

Follow-up: Why is it important?

It helps prevent incomplete requests, lost data, and improperly closed resources.

Q50. How do you handle errors in a Node.js application?

Errors should be handled appropriately where they occur, and asynchronous operations should have proper error handling. At the process level, Node also provides mechanisms such as `uncaughtException` and `unhandledRejection` for detecting serious unhandled failures.

Follow-up: Should we ignore `uncaughtException` and continue normally?

No. An uncaught exception can leave the application in an unsafe state, so production applications should perform necessary cleanup and use a reliable restart strategy.

FINAL NODE.JS REVISION

MUST KNOW

Q1–23 except Q24–25, plus:

Q26, Q27, Q28, Q30, Q31, Q33, Q34, Q36, Q38, Q39, Q40, Q41, Q42, Q43, Q44, Q45, Q46, Q50

IMPORTANT

Q24, Q25, Q29, Q32, Q35, Q37, Q47, Q48, Q49

NODE.JS COMPLETE — 50 QUESTIONS

Ab Node ko aur nahi kheenchna hai.

Next **Express.js** ki separate series karenge, aur wahan bhi **sirf genuinely important fresher questions** — authentication, middleware, routing, REST API, error handling, CORS etc. ko **duplicate nahi karenge** agar pehle cover ho chuka hai.

Tumhe kya lgta h node ke aur bhi important questions h kya h to dedo vo bhi

Haan, **50 ke baad bhi 5–7 Node-specific questions hain jo genuinely useful hain**, especially agar interviewer thoda deeper Node.js pooch le. Lekin **100 tak kheenchna bilkul unnecessary** hai.

Main inhe **bonus questions** maanungi — 50 ke baad priority mein.

Bonus — genuinely worth knowing

51. What is EventEmitter in Node.js?

> **EventEmitter** allows objects to emit named events and lets us register listeners that respond to those events.

Follow-up: What do `on()` and `emit()` do?

> `on()` registers a listener, while `emit()` triggers the event.

52. What is the difference between `setImmediate()` and `setTimeout()`? 🟡

`setImmediate()` schedules a callback for the check phase of the Event Loop, while `setTimeout()` schedules it after a specified minimum delay. Their execution order can depend on the context in which they are called.

👉 Bas itna jaana enough hai; Event Loop internals ratne ki zarurat nahi.

53. What is `readFile()` vs `readFileSync()`? 🔴

`readFile()` performs the file operation asynchronously, while `readFileSync()` blocks execution until the file operation completes.

Follow-up: Which is generally preferred in a server application?

> The asynchronous version, because synchronous file operations can block the main thread.

54. What is middleware in Node.js? 🔴

⚠️ **Important:** Iska detailed version Express series mein karenge, so Node ke liye sirf basic understanding:

Middleware is a function that can process a request or perform some operation between receiving a request and sending a response.

👉 Express mein iska proper question hoga, yahan dobara detail nahi karenge.

55. What is backpressure in Node.js Streams? 🟡

Backpressure occurs when a data source produces data faster than the destination can process it. Streams provide mechanisms to control the flow so the consumer is not overwhelmed.

👉 Fresher ke liye basic definition enough.

56. What is AbortController in Node.js? 🟡

AbortController provides a standard way to signal that an asynchronous operation should be cancelled, when that operation supports cancellation.

Example use case: cancelling a request that is no longer needed.

57. What is the difference between spawn(), exec(), and fork()? 🟡

They are different APIs for working with child processes. spawn() is useful for streaming process input/output, exec() is convenient for executing a command and collecting its output, and fork() is designed for starting another Node.js process with built-in IPC support.

👉 **Don't memorize implementation details.** Just understand the basic difference.

🛑 **Bas. Iske baad Node.js STOP.**

So final:

50 Core + 7 Bonus = 57 questions

But preparation priority:

First 50 → properly prepare

51-57 → only if you have extra time

Aur **EventEmitter, middleware, streams/backpressure, child process** jaise topics ko Express series mein unnecessarily repeat nahi karunga; jahan Node aur Express ka angle genuinely different hai, **sirf difference/follow-up** dunga.

Ye itne chote answers viva thodi h interview h bhai khene ka matlab h na jyada bde na jyada chote 6-8 lines lekar chlo agar difference ka question h to 8 lines otherwise 6 ya 7

Haan yaar 🤔 **ab samajh gaya exactly tumhe kis level ka answer chahiye.** Ye viva-type one-liners nahi honge.

🔒 **Ab se fixed answer format**

Normal question:

➡ **6–7 lines** ka natural interview answer.

Difference/comparison:

➡ **7–8 lines**, preferably clear comparison ke saath.

Follow-up:

➡ Usually **2–4 lines**, but enough to sound confident.

Language:

➡ Easy English, but professional.

➡ Aisa nahi lagega ki tumne definition rat rakhi hai.

➡ Aisa lagega ki **concept actually samajh ke explain kar rahi ho.**

Example:

❌ **Too short — ab nahi:**

Node.js is a runtime environment for JavaScript.

❌ **Too long — ye bhi nahi:**

Node.js was initially created by Ryan Dahl in 2009 and internally uses...

✅ **Correct interview level:**

Node.js is a JavaScript runtime environment that allows us to execute JavaScript outside the browser. It is built on Google's V8 JavaScript engine. Node.js provides APIs for server-side operations such as file handling, networking, and working with processes. It follows an event-driven and non-blocking I/O model, which makes it suitable for handling many concurrent I/O operations. It is commonly used to build backend applications, REST APIs, and real-time applications.

Follow-up: Is Node.js a programming language?

No, Node.js is not a programming language. JavaScript is the programming language, while Node.js provides the runtime environment that allows JavaScript to run outside the browser.

Aur haan, **jo 50 Node questions humne kar liye hain unke answers bhi isi standard ke hone chahiye.** Unhe chhote answers ki wajah se dobara padhne ki zarurat nahi — **main jab revision karwaunga tab proper 6–8 line interview answers mein karwa dunga.**

Aage se यही format locked. 🔒

Nhi ye to bhot bda h tameez se krade

😄 Haan, ab **perfect middle ground** rakhenge. Na viva one-liner, na essay.

🔒 **Final format**

Normal question → 3–4 natural lines

Difference → 4–5 lines

Follow-up → 1–2 lines

Answer aisa hoga jo tum **interview mein comfortably bol sako**, yaad bhi rahe, aur professional bhi lage.

Example — “What is Node.js?”

Node.js is a JavaScript runtime environment that allows us to run JavaScript outside the browser. It is built on Chrome’s V8 engine. It is mainly used for backend development and APIs. Its non-blocking I/O model makes it suitable for applications that handle many requests.

Follow-up: Is Node.js a programming language?

No. JavaScript is the programming language, and Node.js is the runtime environment used to execute it outside the browser.

Bas **isi length aur level** mein ab saare answers rahenge. ❤️

Yes now start first 25 questions aur ye bhi btana ki same question aur kaise pucha ja skta h like scenario/straightforward

Bilkul. Ab **final Node.js 50** ko fresh format mein karte hain — **first 25.**
Har question mein:

-  **Must Know** /  **Important**
 - **Same question aur kis wording/scenario mein aa sakta hai**
 - **Interview-level answer** — neither too short nor too long
 - **Follow-up + answer**
-

NODE.JS INTERVIEW — Q1–Q25

Q1. What is Node.js?

Same question can be asked as:

- What is Node.js?
- Explain Node.js.
- Why do we use Node.js?
- What is Node.js used for?

Answer:

Node.js is a JavaScript runtime environment that allows us to run JavaScript outside the browser. It is built on Google's V8 engine. It is mainly used for backend development, APIs, and server-side applications. Its non-blocking I/O model makes it suitable for handling multiple I/O operations efficiently.

Follow-up: Is Node.js a programming language?

No. JavaScript is the programming language, while Node.js is the runtime environment that executes JavaScript outside the browser.

Q2. Why is Node.js used for backend development?

Same question:

- Why did you choose Node.js for backend?
- What are the advantages of Node.js?
- Why is Node.js popular for server-side development?

Answer:

Node.js allows developers to use JavaScript on the server side, so the same language can be used across the MERN stack. It has a large npm ecosystem with many reusable packages. Its non-blocking I/O model is also useful for applications handling many network and database operations.

Follow-up: Why is it useful for MERN?

Because MERN uses JavaScript across the frontend and backend, which makes development more consistent and easier to manage.

Q3. What is the V8 Engine?

Same question:

- Which engine does Node.js use?
- How does Node.js execute JavaScript?
- What is Google's V8?

Answer:

V8 is Google's open-source JavaScript engine used by Chrome and Node.js. It converts JavaScript code into machine-level instructions so that it can be executed efficiently. Node.js uses V8 as its JavaScript execution engine.

Follow-up: Does V8 provide Node's fs module?

No. V8 executes JavaScript, while Node.js provides additional APIs such as fs, http, and path.

Q4. Is Node.js a framework?

Same question:

- Is Node.js a language?
- Is Node.js a library or framework?
- Where does Express fit in?

Answer:

No, Node.js is not a framework. It is a JavaScript runtime environment that provides the environment and APIs needed to run JavaScript outside the browser. Express.js is a web framework that runs on top of Node.js and makes backend development easier.

Follow-up: So what is Express?

Express is a Node.js web framework used for routing, middleware, and building APIs more easily.

Q5. How is Node.js different from JavaScript? ●

Same question:

- Node.js vs JavaScript?
- Can JavaScript run without a browser?
- What is the difference between JavaScript and Node?

Answer:

JavaScript is a programming language, while Node.js is a runtime environment for executing JavaScript outside the browser. Browser JavaScript provides browser-specific APIs such as the DOM. Node.js provides server-side APIs for things like file handling, networking, and processes.

Follow-up: Can Node.js access the DOM?

No. The DOM is provided by browsers, not by Node.js.

Q6. Is Node.js single-threaded? ●

Same question:

- Does Node.js use one thread?
- How does Node handle multiple requests if it's single-threaded?
- Is Node.js multi-threaded?

Answer:

Node.js runs JavaScript execution on a main thread, but the Node.js runtime can use the operating system and other mechanisms for certain operations. Its asynchronous and non-blocking architecture allows it to handle many I/O operations without creating a separate JavaScript thread for every request.

Follow-up: Does single-threaded mean Node can handle only one request?

No. Node can handle many concurrent requests because I/O operations can proceed asynchronously while the main thread continues processing other work.

Q7. What is non-blocking I/O?

Same question:

- What does non-blocking mean in Node?
- Why is Node.js called non-blocking?
- How does Node handle I/O operations?

Answer:

Non-blocking I/O means Node.js does not have to wait for an I/O operation to finish before continuing other work. For example, while waiting for a database or file operation, Node can continue processing other tasks. This helps the server handle many I/O operations efficiently.

Follow-up: Give examples of I/O operations.

Database queries, file operations, network requests, and reading data from external services are common examples.

Q8. Why is Node.js called event-driven?

Same question:

- Explain Node's event-driven architecture.

- What does event-driven mean?
- How does Node respond to events?

Answer:

Node.js follows an event-driven architecture where certain actions or completed operations generate events. The application can listen for these events and execute the appropriate handler when they occur. This model works well with Node's asynchronous I/O operations.

Follow-up: Which Node feature is based on events?

EventEmitter is a common Node.js feature used to create and handle events.

Q9. What is the Event Loop in Node.js?

Same question:

- How does Node.js handle asynchronous operations?
- Explain the Node.js Event Loop.
- How does Node manage multiple tasks?

Answer:

The Event Loop is a mechanism that allows Node.js to handle asynchronous operations without blocking the main JavaScript thread. When an I/O operation is waiting, Node can continue working on other tasks. Once the operation is ready, its corresponding callback or continuation can be processed by the Event Loop.

Follow-up: What happens if the main thread is blocked?

Other requests and callbacks can be delayed, which can reduce the application's performance.

Q10. Why is Node.js suitable for I/O-intensive applications? ●

Same question:

- Why is Node good for APIs?
- What type of applications are best suited for Node?
- Why choose Node for a backend?

Answer:

Node.js is well suited for I/O-intensive applications because it uses asynchronous and non-blocking I/O. Operations such as database queries and network requests can happen without blocking the main JavaScript thread. This allows Node to handle many concurrent I/O operations efficiently.

Follow-up: Is Node.js ideal for every type of application?

No. CPU-heavy tasks can block the main thread, so they may require techniques such as Worker Threads or separate services.

Q11. What are Node.js Core Modules? ●

Same question:

- What are built-in modules?
- Name some Node.js modules.
- Do you need to install fs?

Answer:

Core modules are modules provided directly by Node.js. They provide commonly required functionality such as file handling, networking, paths, operating-system information, and cryptography. Examples include fs, http, path, os, and crypto.

Follow-up: Do we install core modules using npm?

No. They are already included with Node.js.

Q12. What types of modules are there in Node.js?



Same question:

- What are the different types of Node modules?
- Explain module types in Node.
- What is a user-defined module?

Answer:

Node.js modules can broadly be categorized as built-in, user-defined, and third-party modules. Built-in modules are provided by Node.js, user-defined modules are created by developers, and third-party modules are installed from package managers such as npm.

Follow-up: Give one example of each.

fs is built-in, ./utils.js can be user-defined, and Express is a third-party package.

Q13. Built-in module vs third-party module?

Same question:

- What is the difference between core and external modules?
- Do we install all Node modules?
- fs vs Express?

Answer:

Built-in modules are provided by Node.js and don't require separate installation. Third-party modules are created and published externally and are normally installed using npm. For example, fs is built-in, while Express and Mongoose are third-party packages.

Follow-up: Can we use built-in modules without npm?

Yes. They are already available as part of Node.js.

Q14. What are CommonJS and ES Modules?

Same question:

- What module systems does Node support?
- CommonJS vs ES Modules?
- Explain Node's module system.

Answer:

CommonJS and ES Modules are two module systems supported by Node.js. CommonJS commonly uses `require()` and `module.exports`, while ES Modules use `import` and `export`. The module system used depends on the project's configuration and file type.

Follow-up: Which module system do you prefer in a modern MERN project?

I generally use ES Modules because `import` and `export` provide a modern and standardized module syntax.

Q15. What is the difference between `require()` and `import`?

Same question:

- `require` vs `import`?
- How do you import modules in Node?
- Which syntax is used in CommonJS?

Answer:

`require()` is the CommonJS syntax for loading modules, while `import` is used with ES Modules. CommonJS commonly works with `module.exports`, whereas ES Modules use `export`. The choice depends on the module system configured in the project.

Follow-up: What do you use when "type": "module" is present?

I use `import` and `export` syntax.

Q16. What is `module.exports`?

Same question:

- How do you export a module in CommonJS?
- How do you share code between files?
- What does `module.exports` do?

Answer:

`module.exports` defines the value that a CommonJS module makes available to other files. Another file can access that exported value using `require()`. It can export a function, object, class, or other value.

Example:

```
function add(a, b) {  
  return a + b;  
}
```

```
module.exports = add;
```

Follow-up: How do you import it?

```
const add = require("./add");
```

Q17. What is the difference between `exports` and `module.exports`?

Same question:

- Is `exports` the same as `module.exports`?
- How does `exports` work in CommonJS?
- Why does `exports = something` sometimes not work?

Answer:

Initially, `exports` is a reference to `module.exports`, so we can use it to add properties to the exported object. However, assigning a new value directly to `exports` does not replace `module.exports`. Therefore,

module.exports should be used when replacing the complete exported value.

Follow-up: Which is safer when exporting one complete value?

module.exports is clearer and safer for replacing the complete export.

Q18. What does "type": "module" do? ●

Same question:

- Why are you using import instead of require?
- How do you enable ES Modules in Node?
- Why is require not working?

Answer:

When "type": "module" is added to package.json, Node treats .js files in that package as ES Modules. This allows us to use import and export syntax. It also means CommonJS-specific features such as require() are not directly available in the normal way.

Follow-up: What would you do if you want CommonJS instead?

I would use the CommonJS module system, for example by removing the module configuration or using the appropriate .cjs setup.

Q19. What is npm? ●

Same question:

- What is npm used for?
- What is Node Package Manager?
- How do you install packages in Node?

Answer:

npm is the package manager commonly used with Node.js. It allows us to install, update, remove, and manage project dependencies. It also provides commands for running scripts defined in package.json.

Follow-up: Is npm a programming language?

No. It is a package manager and command-line tool.

Q20. What is `package.json`?

Same question:

- Why is `package.json` important?
- What information does `package.json` contain?
- What is the main configuration file of a Node project?

Answer:

`package.json` is the main configuration and metadata file of a Node.js project. It contains information such as the project name, version, dependencies, `devDependencies`, and npm scripts. It can also contain configuration such as `"type": "module"`.

Follow-up: What are npm scripts?

They are custom commands defined in `package.json`, such as `start` or `dev`, which can be executed using npm.

Q21. What is `package-lock.json`?

Same question:

- Why is `package-lock.json` created?
- Why do we commit `package-lock.json`?
- What is the purpose of the lock file?

Answer:

package-lock.json records the exact versions of the installed dependencies and their dependency tree. It helps different environments install consistent versions of packages. For application projects, it is generally committed to version control.

Follow-up: Should node_modules also be committed?

No. node_modules is normally excluded because dependencies can be recreated from the package files and lock file.

Q22. dependencies VS devDependencies?

Same question:

- Where do you put Express?
- Where do you put Nodemon?
- What is --save-dev?

Answer:

dependencies contain packages required by the application at runtime, while devDependencies contain packages mainly needed during development, testing, or local tooling. For example, Express is normally a dependency, while Nodemon is commonly a devDependency.

Follow-up: How do you install a devDependency?

```
npm install nodemon --save-dev
```

Q23. What is node_modules?

Same question:

- Where are npm packages stored?
- Why don't we push node_modules to GitHub?
- Can node_modules be deleted?

Answer:

node_modules is the directory where npm installs the project's dependencies and their required packages. It can become very large, so it is normally added to `.gitignore` and not pushed to GitHub. If deleted, it can generally be recreated by installing the dependencies again.

Follow-up: How do you recreate it?

By running `npm install` or, when using the lock file for a clean installation, `npm ci`.

Q24. What is the difference between `npm install` and `npm ci`? 

Same question:

- Which command is used in CI/CD?
- How do you install exact locked dependencies?
- `npm install` vs `npm ci`?

Answer:

`npm install` installs project dependencies and may update the lock file when required. `npm ci` performs a clean installation based on the existing lock file and is designed for consistent installations. Therefore, `npm ci` is commonly preferred in CI/CD environments.

Follow-up: Can `npm ci` update `package-lock.json`?

No. It expects the lock file to already be in a valid state.

Q25. What is `npx`? 

Same question:

- `npm` vs `npx`?
- How do you run a package command without installing it globally?

- What is npx create-vite?

Answer:

npx is a command-line tool used to execute package binaries. It is especially useful for running CLI tools without needing to install them globally. For example, npx create-vite can run the Vite project creation tool.

Follow-up: npm vs npx?

npm is mainly used to install and manage packages, while npx is mainly used to execute package commands.

Part 1 — Priority

Questions	Priority
Q1–Q23	 Must Know
Q24–Q25	 Important

Part 1 complete: 25/50.

Next **Q26–Q50** mein Node ke remaining genuinely interview-worthy topics honge — process.env, fs, path, crypto, Buffer, Streams, HTTP, status codes, concurrency, Worker Threads, memory leak, graceful shutdown, etc. **Uske baad Node officially complete.**

Next

Node.js Interview — Part 2/2

Q26–Q50 | Final 25

Format locked:

 **Must Know** |  **Important**

Normal → 3–4 lines | Difference → 4–5 lines | Follow-up → 1–2 lines

Aur har question ke **alternative wording/scenario** bhi diye hain.

Q26. What is the `process` object in Node.js? ●

Same question can be asked as:

- What is process in Node.js?
- How can you access information about the running Node application?
- Which global object represents the current Node process?

Answer:

process is a global object provided by Node.js that gives information about the currently running application. It can be used to access environment variables, command-line arguments, process information, and control certain process operations. It is available without importing any module.

Follow-up: Do we need to import process?

No. process is available globally in Node.js.

Q27. What is `process.env`? ●

Same question:

- How do you access environment variables in Node.js?
- How do you read `.env` values?
- Where do you store configuration values?

Answer:

process.env is an object that provides access to environment variables available to the Node.js process. It is commonly used for values such as database URLs, API keys, ports, and other configuration. These values can change between development and production environments without changing the source code.

Follow-up: Are environment variable values strings?

Yes, they are generally received as strings, so values like numbers may need conversion when required.

Q28. Why should we use environment variables?



Same question:

- Why do we use .env?
- Should API keys be written directly in code?
- How do you keep secrets out of source code?

Answer:

Environment variables keep configuration and sensitive information separate from the application source code. They are useful for database credentials, API keys, JWT secrets, and environment-specific settings. This also makes it easier to use different configurations for development and production.

Follow-up: Should .env be pushed to GitHub?

No, sensitive .env files should normally be added to .gitignore.

Q29. What is `process.argv`?

Same question:

- How do you get command-line arguments in Node?
- How can a Node script accept terminal input?
- What is `process.argv` used for?

Answer:

`process.argv` is an array containing the command-line arguments passed when starting a Node.js application. It allows a Node script to receive input directly from the terminal. It is mainly useful when building command-line tools or scripts.

Follow-up: Give an example.

```
node app.js hello
```

hello can be accessed through the values stored in `process.argv`.

Q30. What is the difference between `process.cwd()` and `__dirname`?

Same question:

- `cwd()` vs `__dirname`
- How do you get the current working directory?
- What is the difference between process location and file location?

Answer:

`process.cwd()` returns the directory from which the Node.js process was started. `__dirname`, in CommonJS, refers to the directory containing the current JavaScript file. So `cwd()` depends on where the application was started, while `__dirname` depends on the location of the current file.

Follow-up: Is `__dirname` directly available in ES Modules?

No. It is a CommonJS feature; ES Modules use the module URL to work with the current file's location.

Q31. What is the `fs` module in Node.js?

Same question:

- How do you work with files in Node.js?
- What is the File System module?
- How can Node read or write files?

Answer:

The `fs` module is a built-in Node.js module used to work with files and directories. It provides operations for reading, writing, updating, deleting, and managing file-system data. It provides both asynchronous and synchronous APIs.

Follow-up: Which should generally be preferred in a server?

Asynchronous operations are generally preferred because synchronous operations can block the main thread.

Q32. What is the `path` module?

Same question:

- How do you work with file paths in Node?
- Why do we use `path.join()`?
- What is Node's `path` module?

Answer:

The `path` module is a built-in Node.js module used for working with file and directory paths. It provides methods such as `join()`, `resolve()`, and `basename()`. It also handles platform-specific path separators, making path operations more reliable across operating systems.

Follow-up: Why use `path.join()`?

It safely combines path segments using the correct separator for the operating system.

Q33. What is the `crypto` module?

Same question:

- How does Node provide cryptographic functionality?
- What is Node's `crypto` module used for?
- How can Node generate secure random values?

Answer:

The `crypto` module is a built-in Node.js module that provides cryptographic functionality. It supports operations such as hashing, generating secure random values, encryption-related operations, and digital signatures. It is commonly used for security-related functionality in backend applications.

Follow-up: Is hashing the same as encryption?

No. Hashing is generally one-way, while encryption is designed to be reversible using the correct key.

Q34. How should passwords be stored in a Node.js application?

Same question:

- Should passwords be encrypted?
- How do you securely store user passwords?
- Why shouldn't passwords be stored directly?

Answer:

Passwords should never be stored as plain text in a database. They should be processed using a secure password-hashing algorithm and the resulting hash should be stored. During login, the entered password is securely compared with the stored hash instead of recovering the original password.

Follow-up: Why is plain-text storage dangerous?

If the database is compromised, attackers could immediately obtain users' actual passwords.

Q35. What is a Buffer in Node.js?

Same question:

- How does Node handle binary data?
- What is Buffer used for?
- Why do we need Buffers in Node?

Answer:

A Buffer is a Node.js data structure used to work with raw binary data. It is useful when handling files, network data, streams, and other data that is not naturally represented as text. Buffers allow Node.js to work efficiently with binary information.

Follow-up: Give a simple example.

```
const buffer = Buffer.from("Hello");
```

This creates a Buffer containing the binary representation of the string.

Q36. What are Streams in Node.js?

Same question:

- How does Node handle large files?
- Why shouldn't we load a huge file completely into memory?
- What is streaming data?

Answer:

Streams allow Node.js to process data gradually in smaller chunks instead of loading the entire data into memory at once. They are useful for large files, network communication, and continuous data processing. This can improve memory efficiency when dealing with large amounts of data.

Follow-up: Why are streams useful for large files?

Because the complete file does not need to be stored in memory before processing begins.

Q37. What are the four types of Streams in Node.js?

Same question:

- What types of streams are available?
- Explain Readable, Writable, Duplex and Transform streams.
- What is a Transform stream?

Answer:

Readable streams are used to read data, while Writable streams are used to write data. Duplex streams can both read and write data. Transform streams are a type of Duplex stream that can modify or transform data while it passes through the stream.

Follow-up: Give an example of a Transform stream.

Compression and decompression are common examples.

Q38. What is the HTTP module in Node.js?

Same question:

- How can Node create an HTTP server?
- Can Node work without Express?
- What is the built-in module for HTTP?

Answer:

The http module is a built-in Node.js module used to create HTTP servers and handle HTTP requests and responses. It provides low-level functionality for working with HTTP. Express is commonly used on top of Node's HTTP capabilities to make backend development easier.

Follow-up: Can we create a server without Express?

Yes. We can create an HTTP server directly using Node's http module.

Q39. How do you create an HTTP server in Node.js?

Same question:

- Create a server using Node.js.
- How does Node handle an incoming request?
- What does http.createServer() do?

Answer:

We can use the built-in `http` module and its `createServer()` method. It receives a callback containing the request and response objects. We can inspect the request and then send a response using the response object. Finally, `listen()` starts the server on a specific port.

```
import http from "http";

const server = http.createServer((req, res) => {
  res.end("Hello World");
});

server.listen(5000);
```

Follow-up: What are req and res?

`req` represents the incoming request, while `res` represents the response sent back to the client.

Q40. What is the difference between an HTTP request and response?

Same question:

- Explain request and response.
- What does the client send to the server?
- What does the server return?

Answer:

An HTTP request is sent by the client to the server to request data or perform an operation. An HTTP response is sent by the server after processing that request. A request can contain a method, URL, headers, and body, while a response contains a status code, headers, and usually a response body.

Follow-up: Give an example.

A browser can send a GET request for user data, and the server can respond with status 200 and JSON data.

Q41. What are HTTP headers? ●

Same question:

- What is the purpose of headers?
- What information is sent through HTTP headers?
- Explain request and response headers.

Answer:

HTTP headers carry additional information about an HTTP request or response. They can describe the data being sent, authentication information, caching rules, and other metadata. Common examples include Content-Type, Authorization, and Accept. Headers are separate from the request or response body.

Follow-up: Give an example of a header.

Content-Type: application/json tells the receiver that the body contains JSON data.

Q42. What are HTTP status codes? ●

Same question:

- Explain HTTP status codes.
- What does 200/404/500 mean?
- What are 2xx, 4xx and 5xx responses?

Answer:

HTTP status codes tell the client the result of its request. 2xx codes generally indicate success, 3xx indicate redirection, 4xx indicate client-side problems, and 5xx indicate server-side errors. They help the client understand whether the request succeeded or failed and why.

Follow-up: What does 201 mean?

201 Created means the request successfully created a new resource.

Q43. What is the difference between 401 and 403?

Same question:

- 401 vs 403?
- Authentication vs authorization status codes?
- When would you return 401 or 403?

Answer:

401 means the request does not contain valid authentication credentials or the authentication is missing. 403 means the server understands who the user is but the user is not allowed to access that resource. In simple terms, 401 is mainly an authentication problem, while 403 is an authorization problem.

Follow-up: Give a real example.

An unauthenticated request to a protected API may get 401, while a normal user trying to access an admin-only resource may get 403.

Q44. What is the difference between 400 and 500?

Same question:

- Client error vs server error?
- When do you return 400?
- What does Internal Server Error mean?

Answer:

400 Bad Request generally means the client sent invalid or unacceptable request data. 500 Internal Server Error means something unexpected went wrong on the server while processing

the request. So 400 usually points to a problem with the request, while 500 points to a server-side failure.

Follow-up: Give an example of 400.

Sending invalid data or an incorrectly formatted request to an API can result in a 400 response.

Q45. What is JSON and why is it used in APIs?

Same question:

- Why do REST APIs use JSON?
- What format does the backend return?
- Is JSON a JavaScript object?

Answer:

JSON stands for JavaScript Object Notation and is a lightweight text-based format for exchanging structured data. It is widely used in APIs because it is easy for both humans and applications to read and process. Although JSON looks similar to a JavaScript object, JSON itself is a text-based data format.

Follow-up: How do you convert a JavaScript object into JSON?

We use `JSON.stringify()` to convert a JavaScript value into a JSON string.

Q46. What is concurrency in Node.js?

Same question:

- How does Node handle many requests?
- How can Node serve multiple users?
- What does concurrency mean in Node?

Answer:

Concurrency means Node.js can manage multiple tasks whose execution overlaps in time. Its Event Loop and asynchronous I/O allow it to continue handling other work while an I/O operation is waiting. This is why Node can efficiently handle many concurrent I/O-based requests without creating a separate JavaScript thread for every request.

Follow-up: Is concurrency the same as parallelism?

No. Concurrency means managing multiple tasks in overlapping periods, while parallelism means executing tasks simultaneously.

Q47. What are Worker Threads in Node.js? 🟡

Same question:

- How does Node handle CPU-intensive tasks?
- Can Node use multiple threads?
- What are Worker Threads used for?

Answer:

Worker Threads allow JavaScript code to run in separate threads from the main JavaScript thread. They are useful for CPU-intensive operations that could otherwise block the main thread. They are not normally needed for simple I/O operations because Node already handles those asynchronously.

Follow-up: Give an example use case.

Heavy calculations or CPU-intensive data processing can be moved to a Worker Thread.

Q48. What is a memory leak in Node.js? 🟡

Same question:

- What causes high memory usage in Node?

- What happens when an application keeps consuming memory?
- Explain memory leaks.

Answer:

A memory leak occurs when an application unintentionally keeps references to objects that are no longer needed. Because those objects are still referenced, garbage collection cannot reclaim their memory. Over time, memory usage can increase and eventually affect performance or crash the application.

Follow-up: How can we reduce memory leaks?

We should remove unnecessary references, clean up resources such as timers and listeners, and monitor memory usage when debugging.

Q49. What is graceful shutdown in Node.js? 🟡

Same question:

- How do you safely stop a Node server?
- What happens when a production server is shutting down?
- Why should a server shut down gracefully?

Answer:

Graceful shutdown means safely stopping a Node.js application instead of terminating it abruptly. The application can stop accepting new work, finish or clean up necessary operations, close database or server connections, and then exit. This helps prevent incomplete requests and resource-related problems.

Follow-up: Why is it important?

It reduces the chance of losing data or leaving connections and other resources improperly open.

Q50. How do you handle errors in a Node.js application? ●

Same question:

- How do you handle errors in Node?
- What happens with an unhandled error?
- What are `uncaughtException` and `unhandledRejection`?

Answer:

Errors should be handled properly where they occur, especially around file operations, database operations, and other asynchronous work. Node also provides process-level events such as `uncaughtException` and `unhandledRejection` to detect serious unhandled failures. These should not replace normal error handling, and production applications should have a proper recovery or restart strategy.

Follow-up: Should we simply ignore `uncaughtException`?

No. It can indicate an unsafe application state, so it should be handled carefully with appropriate cleanup and a reliable restart strategy.

🎯 FINAL NODE.JS STATUS

● **Must Know**

Q1–23 + Q26–28 + Q30–31 + Q33–34 + Q36 + Q38–46 + Q50

● **Important**

Q24–25 + Q29 + Q32 + Q35 + Q37 + Q47–49

✅ **Node.js = COMPLETE — 50/50**

Ab Node ke liye aur questions add nahi karne hain. Is level par fresher MERN interview ke liye preparation practical rahegi.

What does uncaught exception and unhandled rejection do?

What do `uncaughtException` and `unhandledRejection` do?

Both are process-level events used to detect errors that were not properly handled in a Node.js application. `uncaughtException` occurs when a synchronous exception reaches the process without being caught. `unhandledRejection` occurs when a Promise is rejected and there is no rejection handler for it. They are mainly useful for logging and detecting serious application failures.

Follow-up: Should we use them instead of normal error handling?

No. They should not replace normal error handling. They are mainly a last-resort mechanism, and serious failures may require cleanup and restarting the application.

Aur koi important bcha h to vo bhi bta do

Haan, 50 ke baad genuinely important Node.js fresher level par bas kuch hi topics bachte hain. Main unnecessary questions add nahi karunga.

● 4 genuinely useful bonus questions

51. What is EventEmitter in Node.js?

> `EventEmitter` is a Node.js class used to create and handle custom events. We can register a listener using `on()` and trigger an event using `emit()`. It is useful when different parts of an application need to communicate through events.

Same question: What is `emit()`? / How do you create custom events in Node?

52. What is the difference between `readFile()` and `readFileSync()`?

> `readFile()` performs the file operation asynchronously, so Node can continue other work while the file is being read. `readFileSync()` blocks execution until the operation is completed. For server applications, the asynchronous version is generally preferred because blocking can affect performance.

Same question: Synchronous vs asynchronous file reading? / Which fs method is better for a server?

53. What is pipe() in Node.js?

> pipe() connects a readable stream to a writable stream and allows data to flow between them. It is useful for processing large amounts of data without loading everything into memory at once. For example, a file can be read through a stream and directly written to another file.

Same question: How do streams transfer data? / How can you copy a large file efficiently?

54. What is backpressure in Node.js? 🟡

> Backpressure happens when a stream produces data faster than the destination can consume it. Node.js streams provide mechanisms to control the flow of data so the receiving side is not overwhelmed. It is mainly important when working with large or continuous data streams.

Same question: What happens when a readable stream is faster than a writable stream?

🛑 Ab genuinely stop.

Final Node.js preparation = 54 questions.

But realistically, Q1–50 are your main preparation, and 51–54 are bonus. Iske baad jo topics aayenge woh mostly **advanced/rare Node.js internals** hain, jo fresher MERN interview ke liye tumhara preparation unnecessarily huge kar denge.

Os module bhi btade

Haan, os module bhi important hai, but bas 1 proper interview question enough hai. Isko 55 maan lo.

Q55. What is the os module in Node.js? 🟡

Same question can be asked as:

- What is the use of the os module?

- How can you get system information in Node.js?
- How do you get CPU or memory information in Node.js?

Answer:

The os module is a built-in Node.js module used to get information about the operating system and the computer on which the application is running. It can provide information such as CPU details, memory, operating system, hostname, and network interfaces. It can be useful when an application needs system-level information.

Example:

```
import os from "os";

console.log(os.platform());
console.log(os.cpus());
console.log(os.totalmem());
console.log(os.freemem());
```

Follow-up: Do we need to install the os module?

No. It is a built-in Node.js module, so it doesn't need to be installed separately.

Remember these methods:

os.platform() → operating system
os.cpus() → CPU information
os.totalmem() → total memory
os.freemem() → available memory
os.hostname() → computer hostname

Node.js final count → 55 questions.

What is clustering in node.js

Q56. What is Clustering in Node.js?

Same question can be asked as:

- How can Node.js utilize multiple CPU cores?

- How do you scale a Node.js application across CPU cores?
- What is the Node.js Cluster module?

Answer:

Clustering allows a Node.js application to create multiple worker processes that can run on different CPU cores. Normally, Node.js runs JavaScript on a single main thread, so clustering helps utilize multiple CPU cores. A primary process can create and manage multiple worker processes, and these workers can handle incoming requests. This can improve the application's ability to handle higher traffic.

Follow-up: Is clustering the same as Worker Threads?

No. Clustering uses multiple processes, while Worker Threads use multiple threads within a Node.js process.

Follow-up: Why would you use clustering?

I would use it when I need to utilize multiple CPU cores and handle higher server traffic with multiple Node.js worker processes.

How do you optimize the performance of Node.js applications for production?

Q57. How do you optimize the performance of Node.js applications for production? ●

Same question can be asked as:

- How do you improve Node.js application performance?
- How would you make a Node.js API faster?
- What are the best practices for production performance?

Answer:

I would first identify bottlenecks using proper monitoring and profiling instead of optimizing blindly. I would avoid blocking the Event Loop and use asynchronous operations wherever appropriate. Database queries should be optimized with proper indexes and unnecessary data should not be fetched. Caching can be used for frequently requested data, and compression and efficient streaming

can reduce response and memory usage. For high traffic, I can use clustering or multiple application instances behind a load balancer.

Follow-up: What can block Node.js performance?

CPU-intensive operations, synchronous I/O, inefficient database queries, memory leaks, and unnecessarily large responses can block or slow down the application.

Follow-up: How would you handle CPU-heavy operations?

I can move them to Worker Threads or separate services so that the main Event Loop remains responsive.

How can you ensure scalability and maintainability in Node.js APIs?

Q58. How can you ensure scalability and maintainability in Node.js APIs?

Same question can be asked as:

- How do you build scalable Node.js APIs?
- How do you make a backend easy to maintain?
- What are the best practices for scalable APIs?

Answer:

I would keep the application modular by separating routes, controllers, services, models, and middleware based on their responsibilities. I would follow consistent API and error-handling patterns so the code remains predictable. Database queries should be optimized and caching can be used where appropriate. Environment variables should be used for configuration, and proper validation, logging, and security practices should be implemented. For scalability, the API should be stateless so multiple server instances can handle requests behind a load balancer.

Follow-up: Why should an API be stateless?

A stateless API does not depend on data stored in a particular server instance, so requests can be handled by different instances easily.

Follow-up: How does modular code improve maintainability?

Each part has a clear responsibility, so changes and debugging become easier without affecting unrelated parts of the application.

How do you secure Node.js applications from common vulnerabilities?

Q59. How do you secure Node.js applications from common vulnerabilities? ●

Same question can be asked as:

- How do you secure a Node.js API?
- What security practices do you follow in Node.js?
- How do you protect a backend application from attacks?

Answer:

I would validate and sanitize user input to prevent attacks such as injection. Authentication and authorization should be implemented properly, and passwords should always be securely hashed. Sensitive information such as API keys and database credentials should be kept in environment variables. I would also use HTTPS, secure HTTP headers, rate limiting, and properly configured CORS. Finally, dependencies should be regularly updated and checked for known vulnerabilities.

Follow-up: How do you prevent SQL/NoSQL injection?

I validate user input and use safe database queries or APIs instead of directly constructing queries from untrusted user input.

Follow-up: How do you protect sensitive data?

I avoid hardcoding secrets, keep them in environment variables, and never expose them in API responses or frontend code.

How do you connect node with mongodb

Q60. How do you connect Node.js with MongoDB? ●

Same question can be asked as:

- How do you connect MongoDB with a Node.js application?
- How do you establish a MongoDB connection in Node.js?
- Which library do you use to connect Node.js with MongoDB?
- How did you connect MongoDB in your MERN project?

Answer:

In a Node.js application, I can connect MongoDB using the MongoDB driver or Mongoose. In a MERN project, I generally prefer Mongoose because it provides schemas, models, validation, and easier database operations. I keep the MongoDB connection string in an environment variable instead of hardcoding it. Then I create the database connection when the server starts and handle connection errors properly.

Follow-up: Why do you prefer Mongoose?

Mongoose provides schema-based modeling, validation, and useful methods that make working with MongoDB easier and more structured.

Follow-up: Where should the MongoDB connection string be stored?

It should be stored in an environment variable, such as MONGO_URI, rather than directly in the source code.

Follow-up: What happens if MongoDB connection fails?

The application should handle the error properly, log the issue, and avoid pretending that the database is available when the connection has failed.

What is Node.js Web Application Architecture?

Q61. What is Node.js Web Application Architecture?

Same question can be asked as:

- Explain the architecture of a Node.js web application.
- How is a Node.js backend structured?
- Explain the flow of a request in a Node.js application.
- How do different layers of a Node.js API work together?

Answer:

Node.js web application architecture describes how different parts of a backend application work together. A typical application has routes that receive requests, controllers that handle the request logic, services for business logic, and models for database operations. Middleware can handle tasks such as authentication, validation, and logging. The request usually flows from the client to the route, through middleware and controllers, then to the database, and finally the server sends a response back to the client.

Follow-up: Why separate controllers, services, and models?

It follows separation of concerns, so each part has a specific responsibility and the application becomes easier to maintain, test, and scale.

Follow-up: Where does MongoDB fit into this architecture?

MongoDB is the database layer, while Mongoose or the MongoDB driver is used by the backend to communicate with it.

Now start express same as node

Bilkul. Express.js ko bhi Node ki tarah systematic series mein karenge, aur jo concepts already JavaScript/Node mein properly ho chuke hain unko unnecessarily repeat nahi karunga.

Express Series Rules

- **Only genuine fresher MERN interview questions**
- **🔴 Must Know / 🟡 Important**
- Same question **straightforward + alternative wording/scenario**
- Answer = **3–4 natural interview lines**
- Difference = **4–5 lines**
- Follow-up questions + answers
- **No practical/coding questions** — woh separate series mein
- No 5–10 year experienced-level unnecessary topics
- Agar genuinely sirf 30 questions hain, **30 hi karenge**; number artificially nahi badhayenge.

EXPRESS.JS INTERVIEW

Q1–Q25 — Part 1/2

Q1. What is Express.js?

Same question can be asked as:

- What is Express?
- Why do we use Express with Node.js?
- What is Express.js used for?
- Is Express a Node.js framework?

Answer:

Express.js is a lightweight web framework built on top of Node.js. It makes backend development easier by providing features for routing, middleware, request handling, and building APIs. Instead of working directly with Node's HTTP module, Express provides a simpler and more structured way to create web applications and REST APIs.

Follow-up: Is Express a replacement for Node.js?

No. Express runs on Node.js and provides higher-level features that make server-side development easier.

Q2. Why do we use Express.js instead of the Node.js HTTP module?

Same question:

- Why is Express preferred over plain Node.js?
- What problems does Express solve?
- Why not create APIs using only the http module?

Answer:

Node.js's HTTP module can create servers, but handling routes, middleware, request parsing, and errors manually can become difficult as the application grows. Express provides simple routing and middleware mechanisms that make these tasks easier. It also gives a cleaner structure for building and maintaining APIs.

Follow-up: Can we build an API without Express?

Yes. Node.js provides the built-in http module, but Express makes the development process much simpler.

Q3. What are the main features of Express.js?

Same question:

- What are the advantages of Express?
- Why is Express popular?
- What does Express provide?

Answer:

Express provides simple routing, middleware support, request and response handling, and easy API development. It also supports error-handling mechanisms and can be extended using many npm packages. Its lightweight structure allows developers to organize backend applications according to their requirements.

Follow-up: Is Express opinionated?

No. Express is relatively unopinionated, so developers have flexibility in deciding their application structure.

Q4. What is routing in Express.js?

Same question:

- How does routing work in Express?
- How do you create routes in Express?

- What is an Express route?

Answer:

Routing determines how an Express application responds to a particular HTTP method and URL. For example, a GET /users route can return users, while a POST /users route can create a new user. Express provides methods such as get(), post(), put(), patch(), and delete() for defining routes.

Follow-up: Why separate routes?

Separating routes keeps the API organized and makes the application easier to maintain as it grows.

Q5. What is middleware in Express.js? 🔴🔥

Same question:

- What is Express middleware?
- Why do we use middleware?
- What happens between request and response?
- Explain the middleware concept in Express.

Answer:

Middleware is a function that runs during the request-response cycle. It can access the request and response objects and can either perform an operation, modify the request, send a response, or pass control to the next middleware. Middleware is commonly used for authentication, validation, logging, and error handling.

Follow-up: How does middleware pass control?

It calls the next() function to pass control to the next middleware or route handler.

Q6. What is `next()` in Express middleware? 🔴

Same question:

- What does `next()` do?
- Why do we use `next` in middleware?
- What happens if middleware doesn't call `next()`?

Answer:

`next()` is a function provided by Express that passes control to the next middleware or route handler in the request-response cycle. It allows multiple middleware functions to work together for the same request. If middleware neither sends a response nor calls `next()`, the request can remain pending.

Follow-up: Can `next()` pass an error?

Yes. Calling `next(error)` passes the error to Express's error-handling middleware.

Q7. What are the different types of middleware in Express?

Same question:

- What types of middleware does Express provide?
- Explain application-level and router-level middleware.
- What is built-in middleware?

Answer:

Express commonly uses application-level, router-level, built-in, third-party, and error-handling middleware. Application-level middleware applies to the application, while router-level middleware applies to a specific router. Built-in middleware includes functions such as `express.json()`, while third-party middleware is installed separately.

Follow-up: Give an example of built-in middleware.

`express.json()` is a common built-in middleware used to parse JSON request bodies.

Q8. What is `express.json()`? 🚫🔥

Same question:

- How does Express parse JSON?
- Why do we use `express.json()`?
- What happens if we don't use `express.json()`?

Answer:

`express.json()` is built-in Express middleware that parses incoming requests containing JSON data. After parsing, the JSON data becomes available through `req.body`. It is commonly required when an API receives JSON data from a frontend application.

Follow-up: What happens without it?

For JSON request bodies, `req.body` may not contain the parsed data, so the server may not be able to access the submitted JSON correctly.

Q9. What is `express.urlencoded()`? 🟡

Same question:

- How do you parse form data in Express?
- What is URL-encoded middleware?
- `express.json()` vs `express.urlencoded()`?

Answer:

`express.urlencoded()` is middleware used to parse URL-encoded request bodies, which are commonly sent by traditional HTML forms. After parsing, the submitted values are available through `req.body`. It is different from `express.json()`, which is used for JSON request bodies.

Follow-up: When would you use it?

It is useful when the client sends data using the `application/x-www-form-urlencoded` content type.

Q10. What is `req` and `res` in Express?

Same question:

- What are request and response objects?
- What information is available in `req`?
- How do you send a response in Express?

Answer:

`req` represents the incoming HTTP request and contains information such as parameters, query values, headers, and body data. `res` represents the response that the server sends back to the client. We use methods such as `res.json()`, `res.send()`, and `res.status()` to create the response.

Follow-up: How do you send JSON?

We can use `res.json(data)`.

Q11. What is `req.params`?

Same question:

- What are route parameters?
- How do you get an ID from a URL?
- `params` vs query parameters?

Answer:

`req.params` contains dynamic values defined in the route path. For example, in `/users/:id`, the value provided for `id` can be accessed using `req.params.id`. Route parameters are commonly used when identifying a specific resource.

Example:

```
GET /users/25
```

```
req.params.id → 25
```

Follow-up: Are route parameters optional?

A normal route parameter such as `:id` is required for that route to match.

Q12. What is `req.query`?

Same question:

- What are query parameters?
- How do you read filters from a URL?
- `req.query` vs `req.params`?

Answer:

`req.query` contains query parameters provided after the `?` in a URL. They are commonly used for filtering, searching, sorting, and pagination. For example, `/users?role=admin` allows us to access role through `req.query.role`.

Follow-up: Are query parameters required?

No. Query parameters are generally optional and depend on what the API needs.

Q13. What is `req.body`?

Same question:

- How do you access data sent by the client?
- Where is POST data stored?
- How do you get form/API data in Express?

Answer:

`req.body` contains data sent by the client in the request body. It is commonly used with POST, PUT, and PATCH requests when creating or updating resources. The appropriate body-parsing middleware, such as `express.json()`, must be configured according to the request content type.

Follow-up: Where would login credentials normally come from?

For a typical JSON login request, the email and password would be available through `req.body`.

Q14. Difference between `req.params`, `req.query`, and `req.body`?

Same question:

- Params vs query vs body?
- Where do URL ID, filters, and submitted data come from?
- How do you get different types of request data?

Answer:

`req.params` contains values defined as dynamic parts of the URL, such as `/users/:id`. `req.query` contains optional values after `?`, commonly used for filtering or searching. `req.body` contains data sent inside the request body, commonly used when creating or updating resources. So they represent three different ways of receiving client data.

Example:

```
/users/10?active=true
```

```
params → 10
```

```
query → true
```

```
body → submitted JSON data
```

Q15. What is `res.send()` VS `res.json()`?

Same question:

- How do you send a response in Express?
- `send` vs `json`?
- Which method do you use for API responses?

Answer:

res.send() can send different types of responses such as strings, objects, or buffers. **res.json()** is specifically intended for sending JSON responses and automatically handles JSON serialization. In REST APIs, **res.json()** is commonly used when returning structured data to the frontend.

Follow-up: Can **res.send()** send an object?

Yes, Express can serialize an object when using **res.send()**, but **res.json()** clearly communicates that the response is JSON.

Q16. What is `res.status()`?

Same question:

- How do you set an HTTP status code?
- How do you return 404/201/400 from Express?
- How do you send status and response together?

Answer:

res.status() is used to set the HTTP status code of the response. It can be chained with methods such as **json()** or **send()** to return both the status and response data. This allows the API to clearly communicate whether an operation succeeded or failed.

Example:

```
res.status(201).json({  
  message: "User created"  
});
```

Follow-up: Does **res.status()** itself send the response?

No. It sets the status code; a method such as **json()** or **send()** normally sends the response.

Q17. What is `express.Router()`? 🔴🔥

Same question:

- Why do we use Express Router?
- How do you create separate route files?
- What is router-level routing?

Answer:

`express.Router()` creates a separate router object that can contain related routes and middleware. It helps us divide a large application's routes into smaller modules, such as authentication, users, or products. These routers can then be mounted on the main Express application using `app.use()`.

Follow-up: Why is Router useful?

It keeps the application organized and prevents all routes from being placed in one large file.

Q18. What is `app.use()` in Express? 🔴🔥

Same question:

- How do you register middleware?
- How do you mount a router?
- What does `app.use('/api', router)` mean?

Answer:

`app.use()` is commonly used to register middleware or mount a router on an Express application. If a path is provided, the middleware or router can be applied to requests matching that path. It is one of the main methods used to build the request-processing flow of an Express application.

Follow-up: What does this mean?

```
app.use("/api/users", userRouter);
```

It mounts `userRouter` under the `/api/users` path, so its routes are accessed through that base URL.

Q19. What is the difference between `app.use()` and `app.get()`?

Same question:

- `app.use` vs `app.get`?
- Middleware vs route handler?
- When would you use each?

Answer:

`app.use()` is mainly used to register middleware or mount routers and can apply to multiple HTTP methods. `app.get()` specifically defines a route that handles HTTP GET requests. So `app.use()` is generally part of the middleware or routing structure, while `app.get()` defines a specific GET endpoint.

Follow-up: Can `app.use()` handle POST requests?

Yes. Middleware registered with `app.use()` can run for different HTTP methods unless restricted by its configuration.

Q20. What is route middleware?

Same question:

- How do you apply middleware to only one route?
- Can middleware be used for a specific endpoint?
- Route-specific middleware?

Answer:

Route middleware is middleware applied to a particular route instead of the entire application. It is useful when only certain endpoints need a specific check, such as authentication or

validation. We place the middleware between the route path and the final route handler.

Follow-up: Give an example use case.

An authentication middleware can be applied only to protected routes such as /profile or /admin.

Q21. What is error-handling middleware in Express? 🚫🔥

Same question:

- How do you handle errors globally in Express?
- What is Express error middleware?
- How do errors reach the error handler?

Answer:

Error-handling middleware is used to handle errors in a centralized way in an Express application. It has four parameters: err, req, res, and next. Middleware or route handlers can pass an error using next(error), and Express can then forward it to the error-handling middleware.

Follow-up: How can Express identify error-handling middleware?

It has four parameters: (err, req, res, next).

Q22. What is CORS in Express? 🚫

Same question:

- Why am I getting a CORS error?
- How do you enable CORS in Express?
- Can frontend and backend run on different origins?

Answer:

CORS stands for Cross-Origin Resource Sharing and controls whether a browser allows requests from one origin to another. In a MERN application, the frontend and backend may run on different origins during development. Express can use the cors package to configure which origins, methods, and headers are allowed.

Follow-up: Is CORS a backend security mechanism?

It is mainly a browser-enforced cross-origin access control mechanism; it should not be treated as a replacement for authentication or authorization.

Q23. What is REST API in Express.js? 🔴🔥

Same question:

- What is a RESTful API?
- How do you design APIs in Express?
- Explain REST principles.
- What is a REST endpoint?

Answer:

A REST API is an API style where resources are accessed through URLs and standard HTTP methods. For example, GET can retrieve data, POST can create a resource, PUT or PATCH can update it, and DELETE can remove it. Express is commonly used to build these REST endpoints.

Follow-up: Is REST the same as Express?

No. REST is an architectural style, while Express is a framework used to implement APIs and web applications.

Q24. What is the difference between PUT and PATCH?

Same question:

- PUT vs PATCH?
- Which method is used for updating data?
- How do you partially update a resource?

Answer:

Both PUT and PATCH are used for updating resources, but they have different common semantics. PUT is generally used to replace the complete representation of a resource, while PATCH is used for partial updates. For example, updating only a user's name would commonly use PATCH.

Follow-up: Which one would you use for partial updates?

PATCH, because it is designed for modifying only part of a resource.

Q25. What is API versioning?

Same question:

- Why do we use /api/v1?
- How do you maintain old API versions?
- What happens when an API changes?

Answer:

API versioning allows us to introduce changes to an API without immediately breaking existing clients. A common approach is to include the version in the URL, such as /api/v1/users and /api/v2/users. This allows different clients to continue using the version they support while newer clients can use the updated version.

Follow-up: Why is versioning important?

It provides backward compatibility and gives clients time to migrate to newer API versions.

EXPRESS PART 1 COMPLETE

 Must Know

Q1–8, Q10–19, Q21–24

 Important

Q9, Q20, Q25

25/50 complete.

Next Q26–Q50 mein mainly **authentication/authorization, JWT, cookies, sessions, protected routes, validation, security, rate limiting, file uploads, API structure, error handling, production concepts** aayenge — aur jo Node mein already padha hai usko repeat nahi karunga.

Next

Express.js Interview — Part 2/2

Q26–Q50 | Final 25

Ab Express ke **authentication, authorization, JWT, cookies, validation, security, file upload, API structure** wale genuinely important fresher questions karte hain.

Q26. What is authentication and authorization in Express.js?

Same question can be asked as:

- Authentication vs authorization?
- How do you protect APIs?
- How do you check whether a user can access a resource?

Answer:

Authentication verifies who the user is, usually through login credentials or a token. Authorization checks what that authenticated user is allowed to access. For example, login verifies the user's identity, while checking whether the user is an admin is authorization. In Express, middleware is commonly used to implement these checks.

Follow-up: Which comes first?

Authentication generally comes first, because we need to know the user's identity before checking their permissions.

Q27. What is JWT?

Same question:

- What is JSON Web Token?
- How does JWT authentication work?
- Why do we use JWT in MERN applications?

Answer:

JWT stands for JSON Web Token and is commonly used to represent authenticated user information between a client and server. After successful login, the server can issue a token, which the client sends with later requests. The server verifies the token to identify the user and allow access to protected resources.

Follow-up: Does JWT store the user's password?

No. A JWT should not contain sensitive information such as a user's password.

Q28. What are the three parts of a JWT?

Same question:

- Explain JWT structure.

- What is inside a JWT?
- Header vs payload vs signature?

Answer:

A JWT has three parts: header, payload, and signature, separated by dots. The header contains information about the token, the payload contains claims or data, and the signature helps verify that the token has not been modified. The payload is encoded, not encrypted, so sensitive information should not be stored in it.

Follow-up: Is JWT encrypted?

Not by default. JWT is normally encoded and signed, so its payload should not be treated as secret data.

Q29. Where should a JWT be stored on the client?

Same question:

- Where do you store an access token?
- localStorage vs cookies?
- How do you securely store JWT?

Answer:

The storage choice depends on the application's security requirements. A common secure approach is to keep sensitive authentication tokens in HttpOnly cookies so JavaScript cannot directly access them. If tokens are stored in browser storage such as localStorage, they can be exposed if an XSS vulnerability occurs. The overall authentication design should also consider CSRF and other security protections.

Follow-up: Can JavaScript access an HttpOnly cookie?

No. HttpOnly prevents client-side JavaScript from reading that cookie.

Q30. What is a cookie in Express.js?

Same question can be asked as:

- How do you set cookies in Express?
- What are cookies used for?
- How does the server send a cookie to the browser?

Answer:

A cookie is a small piece of data stored by the browser and associated with a website. In Express, cookies can be used for things such as authentication, preferences, or session identifiers. The server can set cookies in the response, and the browser can send them back with later requests.

Follow-up: What is an HttpOnly cookie?

It is a cookie that cannot be accessed through client-side JavaScript, which helps reduce certain XSS-related risks.

Q31. What is the difference between cookies and localStorage?

Same question:

- Cookie vs localStorage?
- Where should authentication data be stored?
- Which is safer for tokens?

Answer:

Cookies are automatically sent with matching HTTP requests, while localStorage data must be manually accessed and sent by JavaScript. Cookies can be configured with security attributes such as HttpOnly, Secure, and SameSite. localStorage is accessible to JavaScript, so an XSS vulnerability can expose stored tokens. The appropriate choice depends on the application's authentication and security design.

Follow-up: Which can be HttpOnly?

Cookies. localStorage cannot be made HttpOnly.

Q32. What is a protected route in Express? 🔴🔥

Same question:

- How do you protect an API endpoint?
- How do you prevent unauthorized users from accessing a route?
- What is authentication middleware?

Answer:

A protected route is an endpoint that can only be accessed after the required authentication or authorization checks succeed. In Express, middleware is commonly placed before the actual route handler to verify the user's token or session. If the check fails, the middleware returns an appropriate error response instead of allowing the request to continue.

Follow-up: What status code is commonly used when authentication is missing or invalid?

401 Unauthorized.

Q33. What is role-based authorization? 🔴

Same question:

- How do you create admin-only routes?
- How do you implement roles in Express?
- How can an admin access different APIs from a normal user?

Answer:

Role-based authorization gives users different permissions based on their assigned roles. For example, an application may have user and admin roles. Authentication first identifies the user, and authorization middleware can then check the user's role before allowing access to a particular endpoint.

Follow-up: Can a normal user access an admin route?

No. If the user is authenticated but does not have the required role, the server should deny access, commonly with a 403 response.

Q34. What is the difference between access token and refresh token? 🔴🔥

Same question:

- Why do we use refresh tokens?
- How does token refresh work?
- Short-lived vs long-lived tokens?

Answer:

An access token is normally short-lived and is used to access protected APIs. A refresh token is generally longer-lived and is used to obtain a new access token when the old one expires. This allows access tokens to have a shorter lifetime while still providing a convenient login experience. Refresh tokens should be stored and handled securely.

Follow-up: Why make access tokens short-lived?

If an access token is stolen, a shorter lifetime reduces the period during which it can be misused.

Q35. What is session-based authentication? 🔴

Same question:

- JWT vs session authentication?
- How does session authentication work?
- What is a server-side session?

Answer:

In session-based authentication, the server creates a session after successful login and associates it with the user. The client usually receives a session identifier through a cookie and sends it with later requests. The server uses that identifier to find the corresponding session and identify the user.

Follow-up: JWT vs sessions?

JWT authentication can keep authentication information in a token, while session authentication generally keeps session state on the server.

Q36. How do you validate user input in Express? 🚫🔥

Same question:

- How do you validate API requests?
- Why is request validation important?
- How do you validate email, password, etc.?

Answer:

I validate incoming request data before processing it or sending it to the database. Validation can check things such as required fields, data types, length, format, and allowed values. It can be implemented using validation libraries or custom middleware. Validation should be performed on the server even if the frontend already validates the data.

Follow-up: Why validate on the backend if frontend validation exists?

Frontend validation improves user experience, but it cannot be trusted for security because clients can send requests directly to the backend.

Q37. What is sanitization?

Same question:

- Validation vs sanitization?
- How do you clean user input?
- How do you protect APIs from malicious input?

Answer:

Sanitization means cleaning or transforming input so that unwanted or potentially dangerous data is removed or handled safely.

Validation checks whether input meets the expected rules, while sanitization modifies or cleans the input when necessary. Both can help reduce security and data-quality problems.

Follow-up: Are validation and sanitization the same?

No. Validation checks whether data is acceptable, while sanitization cleans or transforms the data.

Q38. What is rate limiting?

Same question:

- How do you prevent API abuse?
- How do you protect a login API from too many requests?
- What is request throttling?

Answer:

Rate limiting restricts how many requests a client can make within a specific period. It helps protect APIs from excessive traffic, brute-force login attempts, and certain forms of abuse. In Express, rate-limiting middleware can be configured for the whole application or for sensitive endpoints such as login and password-reset APIs.

Follow-up: Would you apply the same limit to every endpoint?

Not necessarily. Sensitive endpoints such as login may require stricter limits than normal public APIs.

Q39. What is Helmet in Express? 🟡

Same question:

- How do you add security headers in Express?
- What is helmet used for?
- How can Express improve HTTP header security?

Answer:

Helmet is a collection of Express middleware that helps set various security-related HTTP headers. These headers can reduce exposure to certain common web security risks. It is commonly added as part of the security configuration of an Express application, along with proper authentication, validation, and other protections.

Follow-up: Does Helmet make an application completely secure?

No. It provides useful security headers but cannot replace proper authentication, validation, authorization, and secure coding practices.

Q40. How do you handle 404 errors in Express? 🟠

Same question:

- What happens when no route matches?
- How do you create a Not Found handler?
- How do you return 404 from Express?

Answer:

A 404 handler can be placed after all valid routes so that requests that don't match any endpoint are handled separately. The server can return a 404 status with a suitable JSON message. This gives the client a clear response instead of leaving the request without a meaningful result.

Follow-up: Is 404 a server error?

No. It is a client-side HTTP status category indicating that the requested resource or route was not found.

Q41. How do you handle errors globally in Express? 🚫🔥

Same question:

- How do you create centralized error handling?
- How do you avoid repeating try/catch everywhere?
- Explain global error middleware.

Answer:

Express allows us to create centralized error-handling middleware that receives errors from different parts of the application. Routes or middleware can pass errors using `next(error)`, and the central handler can decide the status code and response format. This keeps error responses consistent and avoids duplicating error-handling logic throughout the application.

Follow-up: What parameters does error middleware have?

`err`, `req`, `res`, and `next`.

Q42. What is the purpose of `next(error)`? 🚫

Same question:

- How do you pass an error to Express?
- How does Express know a request has failed?
- What does `next(err)` do?

Answer:

`next(error)` tells Express that an error has occurred during request processing. Express then skips normal middleware and routes and

passes the error to the appropriate error-handling middleware. This allows errors to be handled centrally instead of being handled separately in every layer.

Follow-up: What happens if you call next() without an error?

Express continues the normal middleware chain.

Q43. How do you upload files in Express?

Same question:

- How do you handle image uploads?
- How do you upload a profile picture?
- Which middleware is commonly used for file uploads?

Answer:

File uploads are commonly handled using middleware such as Multer in Express. The client generally sends the file using multipart/form-data, and the middleware processes the uploaded file. The application can then store the file locally or upload it to a cloud storage service. File type, size, and other validation should also be applied.

Follow-up: Why can't express.json() handle normal file uploads?

Because file uploads commonly use multipart/form-data, not JSON.

Q44. What is multipart/form-data?

Same question:

- Which content type is used for file uploads?
- Why do we use FormData?
- JSON vs multipart form data?

Answer:

multipart/form-data is an HTTP content type used to send multiple parts of data in a single request. It is commonly used when a request contains files along with text fields. In a browser, the FormData API can be used to construct such a request. Express applications commonly use middleware such as Multer to process it.

Follow-up: Can JSON directly contain a file?

Not as a normal binary file upload. File uploads are generally handled using multipart form data or another appropriate mechanism.

Q45. What is MVC architecture in Express?

Same question:

- How do you structure an Express application?
- What are Model, View, and Controller?
- Explain MVC in a MERN backend.

Answer:

MVC stands for Model, View, and Controller and is a common way to organize applications by separating responsibilities. Models handle data and database interaction, controllers handle request-related application logic, and views are responsible for presentation. In a MERN backend, React usually handles the frontend UI, so Express applications often focus mainly on routes, controllers, models, services, and middleware.

Follow-up: Why use MVC?

It separates responsibilities, making the code easier to understand, maintain, test, and scale.

Q46. What is the difference between controller and middleware? ●

Same question:

- Controller vs middleware?
- Where should business logic go?
- Should authentication be inside the controller?

Answer:

Middleware performs operations during the request-response cycle and can decide whether the request should continue. A controller generally handles the main logic for a particular endpoint after the required middleware has run. For example, authentication can be middleware, while creating a user or returning user data can be handled by a controller.

Follow-up: Can middleware send a response?

Yes. Middleware can send a response and end the request instead of calling next().

Q47. How do you organize a scalable Express.js project? ●

Same question:

- How would you structure a large Express application?
- What folders do you create in a MERN backend?
- How do you maintain clean Express code?

Answer:

I would separate the application into clear modules such as routes, controllers, models, middleware, services, configuration, and utilities. Routes would define endpoints, controllers would handle request logic, and services could contain reusable business logic. Database configuration and environment variables should also be

kept separate from application logic. This makes the code easier to maintain and scale.

Follow-up: Why not put everything in server.js?

A single large file becomes difficult to understand, test, and maintain as the application grows.

Q48. How do you secure an Express.js API? 🔴🔥

Same question:

- What security practices do you follow in Express?
- How do you protect Express APIs?
- How would you secure a MERN backend?

Answer:

I would implement authentication and authorization, validate and sanitize incoming data, and securely hash passwords. I would protect sensitive configuration using environment variables and configure CORS carefully. I would also use HTTPS, security headers such as Helmet, rate limiting for sensitive endpoints, and regularly check dependencies for vulnerabilities.

Follow-up: Is CORS enough to secure an API?

No. CORS is only one part of browser-based cross-origin control and does not replace authentication or authorization.

Q49. How do you make an Express API scalable and maintainable? 🔴

Same question:

- How do you build scalable Express applications?
- What are Express best practices?
- How would you prepare an API for production?

Answer:

I would keep the code modular by separating routes, controllers, services, models, and middleware. I would use consistent validation and error-handling patterns and keep configuration in environment variables. Database queries should be optimized, and logging and monitoring should be added for production. For scalability, the application should remain as stateless as possible so multiple instances can handle requests behind a load balancer.

Follow-up: Why is statelessness useful?

It allows requests to be handled by different server instances without depending on data stored only in one particular instance.

Q50. How would you handle a production Express API?

Same question:

- What things do you consider before deploying an Express API?
- How do you make an Express application production-ready?
- What are your Express production best practices?

Answer:

Before production, I would configure environment variables, proper error handling, request validation, authentication, authorization, security headers, CORS, and rate limiting. I would also optimize database operations and avoid exposing sensitive information in responses or logs. Proper logging, monitoring, HTTPS, and a reliable process or deployment strategy should also be in place.

Follow-up: Should detailed server errors be returned directly to users?

No. Users should receive a safe, meaningful error message, while detailed technical information should be logged securely on the server.

EXPRESS.JS COMPLETE — 50/50

Must Know

Q1–8, Q10–19, Q21–24, Q26–38, Q40–42, Q46–50

Important

Q9, Q20, Q25, Q39, Q43–45

Highest-priority Express topics

Agar revision ke time bahut pressure ho, sabse pehle ye pakke karo:

Middleware → Routing → req.params/query/body → app.use() → Router → Error handling → REST → JWT → Authentication vs Authorization → Protected routes → Roles → Validation → Cookies → PUT/PATCH → CORS → Security.

Express ke 50 questions complete. Ab iske baad unnecessary Express internals add karke preparation ko huge nahi karenge.

Aur kuch questions agar important bche h to vo bhi dedo

Haan, 50 ke baad **sirf 5 genuinely useful bonus questions** aur bachte hain. Iske baad Express ke liye stop karna better hai—warna unnecessary advanced questions aa jayenge.

Express.js — Bonus Q51–Q55

Q51. What is `express.static()`?

Same question: How do you serve static files in Express? / How do you serve images, CSS, or frontend files?

`express.static()` is built-in Express middleware used to serve static files such as images, CSS, JavaScript, and HTML files. It allows clients to access files directly from a specified directory.

Follow-up: Can Express serve images using it?

Yes. For example, an uploads or public directory can be served using `express.static()`.

Q52. What is the difference between middleware and a route handler? ●

Same question: Middleware vs controller? / What is the difference between middleware and endpoint logic?

Middleware runs during the request-response cycle and can perform common tasks such as authentication, validation, or logging. A route handler is responsible for handling a specific endpoint and usually generating the final response. Middleware can pass control using `next()`, while a route handler normally completes the request.

Follow-up: Can middleware send the final response?

Yes. It can send a response instead of calling `next()`.

Q53. What is request validation middleware? ●

Same question: Where should API validation happen? / How do you validate requests before the controller?

Validation middleware checks incoming request data before it reaches the main controller logic. It can verify required fields, data types, formats, and allowed values. If the data is invalid, it can immediately return an appropriate error response; otherwise, it passes control to the next middleware or controller.

Follow-up: Why is this better than validating inside every controller?

It keeps validation reusable and prevents controllers from becoming unnecessarily large.

Q54. What is the difference between `res.send()`, `res.json()`, and `res.end()`? ●

Same question: How do you terminate an Express response? / Which method should you use for API responses?

`res.send()` sends a response and can handle different data types. `res.json()` is specifically used to send JSON data, so it is commonly

preferred for REST APIs. `res.end()` simply ends the response and is useful when no response body is required.

Follow-up: Which one would you commonly use for a MERN API?

I would commonly use `res.json()` because MERN APIs generally exchange structured JSON data.

Q55. What happens if you send two responses for the same request? 🟡

Same question: Why do I get Cannot set headers after they are sent? / Can we call `res.json()` twice?

An HTTP request should normally receive only one response. If the application tries to send another response after the response has already been sent, Express can throw an error such as Cannot set headers after they are sent. This usually happens because the code continues executing after sending a response or multiple code paths try to respond.

Follow-up: How do you prevent it?

After sending a response in a conditional branch, I make sure the execution does not continue to another response path.

✅ **Final Express Count: 55**

Ab genuinely enough hai.

Iske baad jo questions aayenge woh mostly advanced Express internals ya experienced-level architecture honge, jo **fresher MERN interview ke liye priority nahi hain.**

Chalo ab react ke jitne bhi important question hai jitne bhi means agar tumhen lagta hai ki pressure ke liye 16 important question hai to 60 de do agar tum lagta hai sirf 50 hi important hai to 50 de do to main lagta hai agar 30 important hai to 30 de do matlab hamen befaltu ki chijen bhi nahin karni theek hai aur koi important chij chhod nahin hai theek hai to is type se preparation karni hai to ab batao kitne total kitne questions karne padenge react ke interview ko crack karne ke liye ek fresher ko

Bilkul. Is baar main **number artificially nahi badhaunga**. React mein jo genuinely fresher MERN interview ke liye important hai, **sirf utne hi** karenge.

React Fresher Interview — Total 50 Questions

Mere hisaab se **50 questions enough hain** React ke interview ke liye, provided ki hum unke **follow-ups + alternative wording + scenario-based wording** bhi cover karein.

Tier 1 — Must Know: 30 questions

Ye questions almost har React interview mein kisi na kisi form mein aa sakte hain:

1. What is React?
2. Why is React used?
3. What are the main features of React?
4. What is JSX?
5. Why do we use JSX?
6. JSX vs HTML
7. What are components?
8. Functional vs class components
9. What are props?
10. Props vs state
11. What is state?
12. How does state update work?
13. What is the Virtual DOM?
14. Virtual DOM vs Real DOM
15. What is reconciliation?
16. What is the key prop?
17. Why are keys important in lists?
18. What are event handlers in React?
19. How do you handle forms in React?
20. Controlled vs uncontrolled components
21. What is conditional rendering?
22. How do you render lists?
23. What is useState?

- 24. What is useEffect?
 - 25. How does the dependency array work?
 - 26. What is the cleanup function in useEffect?
 - 27. What is useContext?
 - 28. What is prop drilling?
 - 29. What is React Router?
 - 30. How do protected/private routes work?
-

● Tier 2 — Very Important: 15 questions

- 31. What are Hooks?
 - 32. Rules of Hooks
 - 33. Why can't Hooks be called inside conditions/loops?
 - 34. useState vs useReducer
 - 35. useEffect vs useLayoutEffect
 - 36. What is useRef?
 - 37. useMemo vs useCallback
 - 38. What is React.memo?
 - 39. What causes a component to re-render?
 - 40. How can you optimize unnecessary re-renders?
 - 41. What is lifting state up?
 - 42. What is component composition?
 - 43. What is lazy loading/code splitting?
 - 44. How do you handle API calls in React?
 - 45. How do you handle loading and error states?
-

● Tier 3 — Important but Lower Priority: 5 questions

- 46. What are custom Hooks?
 - 47. What is Context API and when should you use it?
 - 48. What is error handling/Error Boundary in React?
 - 49. What is Strict Mode?
 - 50. What is the difference between client-side rendering and server-side rendering?
-

But ek important baat

50 questions ka matlab sirf 50 answers ratna nahi hai.

Hum har question ko is format mein karenge:

**Question → Same question ke alternative wordings → Interview-level answer
→ Follow-up → Follow-up answer**

Example:

“What is useEffect?”

Interviewer isi ko ye bhi puch sakta hai:

- Why do we use useEffect?
- When does useEffect run?
- What happens when dependency array is empty?
- What happens if dependency array is omitted?
- How do you perform cleanup in useEffect?
- How do you fetch API data in useEffect?

To 1 main question ke andar uske natural follow-ups cover honge.

Isliye actual preparation:

50 main questions + unke follow-ups = React ka complete fresher interview preparation.

Aur haan — **practical/coding questions abhi nahi**, jaise tumne bola tha. Unki separate series rakhenge.

React = 50. Node = 60 ke around, Express = 55.

Ab React ko bhi isi standard par start karna best rahega.

Iske alava bhi agar koi important bacha hai to vah bhi de do ek question Maine yah bhi suna hai ki tum kaise Apne react application ko optimise karoge is type se bhi

Haan, **bilkul sahi suna**. “How would you optimize a React application?” type ke **scenario/best-practice questions** bhi fresher interviews mein important hote

hain. Mere 50 mein ye angle partially covered tha, lekin **standalone interview questions ke roop mein kuch important ones add karne chahiye.**

Main unnecessary advanced questions nahi add karunga. **Final React list ko 60 questions** karna better hai.

Additional 10 — React Interview Questions

51. How would you optimize a React application?

Same question:

- How do you improve React application performance?
- How do you make a React app faster?
- What steps would you take if a React application is slow?

I would first identify the actual performance bottleneck instead of optimizing unnecessarily. I would reduce unnecessary re-renders using techniques such as React.memo, useMemo, and useCallback where they are actually beneficial. I would also use lazy loading and code splitting for large components, optimize images and API requests, and avoid unnecessary state updates. For large lists, techniques such as virtualization can also improve performance.

Follow-up: What can cause unnecessary re-renders?

Unnecessary state updates, changing object or function references, parent re-renders, and poorly managed component state can cause unnecessary re-renders.

52. What causes a React component to re-render?

A component can re-render when its state changes, when its parent re-renders, or when the values of its props change. Context updates can also cause consuming components to re-render. A re-render does not necessarily mean the DOM will be completely updated because React performs reconciliation to determine the required changes.

53. How do you optimize a large list in React?

Same question:

- How would you render thousands of items efficiently?
- How do you improve large-list performance?

For a very large list, I would avoid rendering every item at the same time. I can use pagination or list virtualization so that only the required items are rendered. I would also use stable keys and avoid unnecessary calculations or re-renders inside each list item.

54. What is lazy loading in React? ●

Lazy loading means loading a component only when it is actually needed instead of loading everything when the application starts. React provides `lazy()` for dynamically importing components, and `Suspense` can display a fallback while the component loads. This can reduce the initial JavaScript bundle and improve initial loading performance.

55. What is code splitting in React? ●

Code splitting divides the application's JavaScript bundle into smaller chunks that can be loaded when required. This prevents the browser from downloading the entire application upfront. React's `lazy()` and dynamic imports are commonly used to achieve this.

Follow-up: Lazy loading vs code splitting?

Code splitting creates separate bundles or chunks, while lazy loading is a technique of loading those chunks only when they are needed.

56. How do you optimize API calls in React? ●

I would avoid unnecessary API calls by triggering requests only when required and keeping dependencies correct in `useEffect`. I would use caching or data-fetching libraries when appropriate and avoid fetching the same data repeatedly. Pagination, debouncing for search, and proper loading and error handling can also improve the user experience and application performance.

57. What is debouncing in React? 🟡

Same question:

- How would you optimize a search input?
- How do you prevent an API call on every keystroke?

Debouncing delays an operation until the user stops performing an action for a specified period. For example, in a search box, instead of calling the API after every keystroke, we can wait until the user stops typing. This reduces unnecessary API requests and improves performance.

58. How do you manage state in a large React application? 🔴

Same question:

- How would you handle global state?
- Context API vs Redux?
- How do you avoid unnecessary prop drilling?

I would keep state as close as possible to the components that actually need it. For shared state, I can use Context API or a state-management library such as Redux depending on the application's complexity. I would avoid putting every piece of state globally because unnecessary global state can make the application harder to maintain and can cause unnecessary re-renders.

59. How do you handle loading, error, and empty states in React? 🔴

Same question:

- What happens while an API request is loading?
- How do you handle API failures in the UI?
- How do you handle an empty API response?

I would maintain clear UI states for loading, success, error, and empty results. While the request is running, I would show a loader or skeleton; if it fails, I would display a meaningful error message; and if there is no data, I would show an appropriate empty-state

message. This gives users clear feedback instead of leaving the interface blank.

60. How would you improve the user experience of a React application? 🟡

Same question:

- How do you make a React application production-ready?
- What would you improve in a React application?
- What React best practices do you follow?

I would focus on performance, responsive design, accessibility, clear loading and error states, and efficient API handling. I would also use reusable components, proper state management, lazy loading where appropriate, and meaningful error messages. Before deployment, I would test the application and check performance using appropriate browser and React developer tools.

✅ FINAL REACT COUNT: 60

Ab main 60 par genuinely stop karunga.

Because ab humne cover kar liya:

React fundamentals → JSX → components → props/state → Virtual DOM → reconciliation → events/forms → Hooks → useEffect → Context → Router → authentication/protected routes → performance → optimization → API handling → lazy loading → code splitting → state management → large lists → debouncing → production practices.

Aur “How would you optimize...?”, “What would you do if...?”, “How would you handle...?” jaise interview ke scenario questions bhi ab included hain.

Iske baad React ke questions add karna mostly unnecessary repetition/advanced territory hoga for a fresher.

/research Superb ab isi traha node ke bhi most important scenario interview questions dedo answers ekdum simple human English me hone chahiye na jyada chote na jyada bde 3 lines bhot h

Bilkul. Is baar **sirf Node.js ke scenario-based questions** karenge—jo interviewer practical situation dekar puch sakta hai. Already covered **Promise, async/await, callback hell, CORS, JWT/authentication basics, MongoDB connection, clustering, optimization, scalability, security, architecture** ko repeat nahi kar raha.

Answers ~3 lines, simple human English, jaise tum interview mein naturally bol sako.

Node.js Scenario-Based Interview Questions

1. Your Node.js server suddenly becomes very slow. What would you check first? ●

I would first check whether the problem is caused by CPU usage, memory usage, database queries, or blocking code. I would use logs and monitoring tools to identify the bottleneck before making changes.

Same question:

- What would you do if your Node API becomes slow in production?
 - How would you debug a slow Node.js application?
-

2. Your API is taking 5 seconds to respond. How would you find the reason? ●

I would check the API logs and measure each important operation, especially database queries and external API calls. Then I would identify which operation is taking the most time and optimize that specific part.

Same question:

- How would you debug a slow API?
 - How do you find the bottleneck in a Node API?
-

3. Your Node.js application crashes unexpectedly. What would you do? ●

I would first check the server logs and error stack to find the reason for the crash. I would fix the root cause and make sure important errors are handled properly. In production, I would also use a process manager so the application can restart when necessary.

Same question:

- What if your Node server goes down?
 - How would you handle a production crash?
-

4. Your server works locally but crashes in production. What could be the reason? ●

I would first compare the environment variables, Node.js version, dependencies, database configuration, and production settings. I would check the production logs to identify the exact error instead of assuming the code is wrong.

Same question:

- Why does an application work on localhost but not on production?
 - How would you debug a deployment issue?
-

5. Your Node.js application is consuming too much memory. What would you check? ●

I would check for memory leaks, large objects being kept in memory, unnecessary caching, and resources that are not being released. I would use monitoring or profiling tools to find what is continuously increasing memory usage.

Same question:

- How would you debug high memory usage?
 - What can cause memory problems in Node.js?
-

6. A database query is making your API slow. What would you do? ●

I would check the query and database execution time first. I would optimize the query, add appropriate indexes, fetch only the required data, and avoid unnecessary database calls.

Same question:

- How would you optimize database-heavy APIs?
 - What if MongoDB is slowing down your Node API?
-

7. Your API receives thousands of requests at the same time. How would you handle it? ●

I would keep the API stateless and optimize database and API operations first. For higher traffic, I could run multiple Node.js instances behind a load balancer and use caching where appropriate.

Same question:

- How would you scale a Node.js application?
 - What would you do if traffic suddenly increases?
-

8. A user sends a very large request body to your API. What would you do? ●

I would limit the maximum request body size so that users cannot send unnecessarily large data. I would also validate the input before processing it to protect the server from excessive resource usage.

Same question:

- How do you prevent large payloads from affecting your server?
 - How would you protect an API from oversized requests?
-

9. Your API receives thousands of requests from the same client in a short time. What would you do? ●

I would use rate limiting to control how many requests a client can make within a certain time. This helps protect the API from abuse and excessive traffic, especially on sensitive endpoints such as login.

Same question:

- How would you prevent API abuse?
 - How would you protect a login endpoint from excessive requests?
-

10. Your Node.js server is blocked by a CPU-heavy operation. How would you handle it? ●

I would avoid running heavy CPU-intensive work directly on the main thread because it can block the Event Loop. I could move that work to Worker Threads or a separate service so normal requests remain responsive.

Same question:

- What if a Node API performs a heavy calculation?
 - How do you handle CPU-intensive tasks in Node?
-

11. Your API needs to process a 2 GB file. Would you load it completely into memory? ●

No, I would use streams so the file can be processed in smaller chunks. This avoids using a large amount of memory and is much more suitable for handling very large files.

Same question:

- How would you handle very large files?
 - Why are streams useful in Node.js?
-

12. Your application needs to send a large file to a client. What would you use? 🟡

I would use a readable stream instead of loading the complete file into memory first. The data can then be sent to the client gradually, which is more memory-efficient.

Same question:

- How would you efficiently download a large file?
 - Would you use `readFile()` for a huge file?
-

13. Your API sometimes returns old data even after it has been updated. What could be the reason? 🟡

One possible reason is caching. I would check whether the API, database, or another layer is returning cached data. I would make sure the cache is updated or invalidated whenever the underlying data changes.

Same question:

- How can caching cause stale data?
 - What would you check if an API returns outdated information?
-

14. Your application needs the same data repeatedly. How would you improve performance? 🔴

I would consider caching frequently requested data so the application does not repeatedly perform the same expensive operation. I would also decide how long the data should remain cached based on how frequently it changes.

Same question:

- How would you reduce repeated database queries?
 - When would you use caching in Node.js?
-

15. Your Node application needs to call an external API, but that API is sometimes unavailable. What would you do? ●

I would handle the external API failure gracefully instead of allowing it to crash or break the entire application. I would use proper error handling, timeouts, and where appropriate, retry logic with a reasonable limit.

Same question:

- How do you handle third-party API failures?
 - What if a payment or email service is temporarily down?
-

16. Your API is returning sensitive information in its response. What would you do? ●

I would review the response object and return only the fields that the client actually needs. Sensitive information such as passwords, secret keys, or internal credentials should never be exposed through API responses.

Same question:

- How do you prevent sensitive data leakage?
 - What data should never be returned from an API?
-

17. Your application contains API keys directly in the source code. What would you do? ●

I would remove the secrets from the source code and store them in environment variables or a proper secrets-management system. I would also make sure those secrets are not committed to the repository.

Same question:

- How would you secure API keys?
 - What would you do if you accidentally hardcoded a secret?
-

18. Your Node application needs different configurations for development and production. How would you manage them? 📌

I would use environment variables for configuration that changes between environments. For example, database URLs, ports, API keys, and other environment-specific settings can be configured separately without changing the application code.

Same question:

- How do you manage environment-specific configuration?
 - Development and production use different databases—how would you handle that?
-

19. Your server needs to perform some work after the response has been sent. What would you consider? 🟡

I would first check whether that work really needs to happen during the request. If it can safely happen independently, I would move it to a background job or separate process so the user does not have to wait for it.

Same question:

- How would you handle background tasks?
 - How do you avoid making users wait for non-critical operations?
-

20. Your Node API is receiving invalid input from the frontend. What would you do? 📌

I would never rely only on frontend validation. I would validate the data again on the server before processing it or sending it to the database, and return a clear error when the input is invalid.

Same question:

- Why validate data on the backend?
 - What if someone directly calls your API without using your frontend?
-

21. Your application has multiple modules that need to communicate with each other. What could you use? 🟡

For simple event-based communication within a Node application, I could use EventEmitter. If the communication is between separate services, I would consider a proper messaging or event system depending on the application requirements.

Same question:

- How can different parts of a Node application communicate?
 - When would you use EventEmitter?
-

22. Your server needs to shut down while requests are still being processed. What would you do? 🟡

I would use graceful shutdown so the server can stop accepting new requests and properly finish or clean up existing work. I would also close database and other connections before the process exits.

Same question:

- How would you safely stop a Node server?
 - What is graceful shutdown?
-

23. Your Node application has many repeated pieces of code. How would you improve it? 🔴

I would identify common functionality and move it into reusable modules or functions. This reduces duplication and makes future changes easier because the same logic is maintained in one place.

Same question:

- How do you keep Node.js code maintainable?
 - How would you reduce duplicate backend logic?
-

24. Your API has many routes and the project is becoming difficult to maintain. What would you do? 🚫

I would separate routes, controllers, services, models, and middleware into appropriate modules. This gives each part a clear responsibility and makes the application easier to understand, test, and maintain.

Same question:

- How would you structure a growing Node.js project?
 - What would you do when server.js becomes too large?
-

25. A Node.js API is working correctly but its response time is inconsistent. How would you investigate it? 🟡

I would monitor the response time and check whether the delay comes from the database, external APIs, CPU usage, or other operations. I would use logs and performance monitoring to identify patterns instead of guessing the cause.

Same question:

- Why can API response time fluctuate?
 - How would you debug inconsistent API performance?
-

🎯 Ye 25 enough hain

Inmein **actual interview mein scenario bana ke puchne wale major Node.js areas** cover ho gaye:

Performance → debugging → crashes → memory → database → scaling → large requests → rate limiting → CPU work → streams → caching → external APIs → security → environment → background work → validation → events → graceful shutdown → maintainability.

Aur importantly, **already-covered JS/Node questions ko repeat nahi kiya.**

Express bhi for freshers kuch bhi random question mat kro research kro and then btao ki kon konse questions mujhe krne hi chahiye agar sirf 5 questions

milte h to 5 do 10 milte h to 10 do means number matter nhi krta /research /think and answers bhi vaise tumhe kya lgta h puchte h scenario based questions freshers se bhi?

Haan, maine specifically **fresher/junior Express interviews ke recent patterns** ko check karke dekha. Current Express documentation + recent junior/fresher interview material mein repeatedly **middleware, routing/order, request data, error handling, REST APIs, authentication/security aur production basics** aa rahe hain. ([Express.js](#))

Aur **haan** — **freshers se scenario-based questions bhi puchte hain**. Lekin usually 10 saal experience wale system-design scenarios nahi. Example: “*req.body undefined aa raha hai, kya check karoge?*”, “*protected route kaise banaoge?*”, “*API 404 de rahi hai, debug kaise karoge?*” Ye basically wahi fundamentals hote hain, bas real situation mein puchhe jaate hain. Recent junior-oriented material bhi middleware order, route matching aur error handling ko on-the-spot reasoning ke form mein highlight karta hai. ([TechPrep](#))

Express Scenario Questions — 20 genuinely important

Main **20 par stop karunga**. Isse zyada karna tumhari preparation ko unnecessarily huge karega.

1. Your `req.body` is coming as `undefined`. What would you check?

First, I would check whether the correct body-parsing middleware such as `express.json()` is added. I would also check whether the client is actually sending JSON with the correct Content-Type.

Same wording:

- Why is `req.body` undefined?
 - POST request data isn't coming to Express. What will you check?
-

2. Your middleware is running, but the next middleware is never called. What could be wrong? ●

I would check whether the middleware is calling `next()` after completing its work. If it neither sends a response nor calls `next()`, the request can remain pending. ([Express.js](#))

Same:

- Why is my Express request hanging?
 - What happens if middleware doesn't call `next()`?
-

3. You added authentication middleware, but the protected route is still accessible. What would you check? ●

I would check whether the authentication middleware is actually attached to that route and whether it correctly stops unauthorized requests. I would also verify that it calls `next()` only after successful authentication.

Same:

- Why is my protected route not protected?
 - Authentication middleware is not working—how would you debug it?
-

4. Your authentication middleware rejects every user. How would you debug it? ●

I would check whether the token is actually being received, whether I am reading the correct header or cookie, and whether the token verification is using the correct secret. I would also check the decoded user information before calling `next()`.

Same:

- JWT middleware always returns 401. Why?
 - Login works but protected APIs don't.
-

5. Your API returns 404 even though the route exists. What would you check? ●

I would check the HTTP method, URL, router mounting path, and route definition. I would also check whether the router is actually mounted using `app.use()` and whether route or middleware order is affecting the request.

Same:

- Route not found in Express?
- Why am I getting 404 for an existing endpoint?

Express routing depends on the matching HTTP method and route path.
([Express.js](#))

6. You created a router but its routes are not working. What would you check? ●

I would first check whether the router is imported and mounted correctly with `app.use()`. Then I would verify the base path and make sure the final URL is the combination of the mounted path and router route.

Same:

- Express Router isn't working.
 - Why is my separate route file not responding?
-

7. One middleware should run only for `/admin` routes. How would you implement it? ●

I would apply the middleware to the admin router or the `/admin` route instead of registering it globally. This keeps the authentication or authorization logic limited to the routes that actually need it.

Same:

- How do you apply middleware to selected routes?
 - How would you create admin-only middleware?
-

8. Your error handler isn't catching an error. What would you check? ●

I would check whether the error is being passed to Express using `next(error)`, and whether the error-handling middleware has the correct four-argument signature. I would also check that it is registered after the normal routes and middleware. ([Express.js](#))

Same:

- Why is my global error handler not working?
 - Express is not catching my error.
-

9. An API throws an error inside an async route. How would you handle it? ●

I would make sure the rejected Promise or thrown error reaches Express's error-handling middleware. In current Express 5, rejected Promises from route handlers are forwarded automatically; in older Express 4 patterns, the error generally needs to be passed through `next(error)` or a wrapper. ([Express.js](#))

Same:

- How do you handle async errors in Express?
 - What happens if an async controller throws an error?
-

10. Your API sends a response and then throws `Cannot set headers after they are sent`. What happened? ●

It usually means the application tried to send more than one response for the same request. I would check whether a code path sends a response and then continues executing, or whether multiple branches are responding.

Same:

- Why am I getting `Cannot set headers after they are sent`?
 - Can Express send two responses for one request?
-

11. You need to return different responses for normal users and admins. How would you do it? ●

I would first authenticate the user and then use authorization middleware to check the user's role. If the role has permission, I would call next(); otherwise, I would return a 403 response.

Same:

- How would you implement role-based authorization?
 - How do you create an admin-only API?
-

12. Your frontend and Express backend are running on different ports and the browser shows a CORS error. What would you do? ●

I would configure CORS on the backend to allow the frontend's origin and the required methods and headers. I would also check whether credentials are being used because cookies require appropriate CORS configuration.

Same:

- Frontend cannot call Express API because of CORS.
 - How would you fix a CORS issue?
-

13. A client sends invalid data to your API. Should the controller directly save it to MongoDB? ●

No. I would validate the request data before sending it to the database. If the data is invalid, I would return a suitable 400-level response instead of processing it.

Same:

- Where should API validation happen?
- How do you prevent invalid data from reaching the database?

Express specifically recommends not trusting user input and validating/handling it properly. ([Express.js](#))

14. Your login API is being hit hundreds of times by the same IP. What would you do? ●

I would add rate limiting or brute-force protection to the login endpoint. I would also monitor repeated failed attempts and apply stricter limits to sensitive authentication endpoints. ([Express.js](#))

Same:

- How would you protect a login API from brute-force attacks?
 - How do you prevent API abuse?
-

15. Your Express application is exposing too much information in error responses. What would you change? ●

I would return a simple and safe error message to the client and keep detailed technical information in server-side logs. Production responses should not expose stack traces, database details, or internal implementation information. ([Express.js](#))

Same:

- Should you send `err.stack` to the frontend?
 - How do you handle errors securely in production?
-

16. You need to upload a profile picture through an Express API. How would you handle it? ●

I would receive the file using multipart/form-data and use suitable upload middleware such as Multer. I would also validate the file type and size before storing it locally or sending it to cloud storage.

Same:

- How would you implement image upload?
 - Why can't `express.json()` handle a normal file upload?
-

17. Your `app.js` has hundreds of routes and is becoming difficult to maintain. What would you do? ●

I would separate routes into different Express routers based on features such as users, authentication, and products. I would also separate controllers, middleware, services, and models so each part has a clear responsibility.

Same:

- How would you structure a large Express application?
 - How would you make an Express project maintainable?
-

18. A route works before adding middleware but stops working after adding it. How would you debug it? ●

I would check the middleware order first because Express processes middleware in the order it is registered. I would also check whether the middleware sends a response, throws an error, or forgets to call `next()`. ([Express.js](#))

Same:

- Why did adding middleware break my route?
 - Does middleware order matter in Express?
-

19. Your Express API works in development but is not secure enough for production. What would you check? ●

I would check HTTPS, secure cookie settings, input validation, rate limiting, security headers, dependency vulnerabilities, and whether sensitive configuration is properly protected. I would also avoid exposing detailed error information in production. ([Express.js](#))

Same:

- How would you make an Express API production-ready?
 - What security checks would you perform before deployment?
-

20. You have to debug an Express API that is failing randomly. What would be your approach? 🟡

I would reproduce the issue and check server logs, request details, database operations, and external API calls. Then I would isolate the failing part instead of changing multiple things at once, and add proper error logging if the problem is difficult to reproduce.

Same:

- How would you debug a production Express API?
 - An API sometimes works and sometimes fails—what would you check?
-

Ab important conclusion

Haan, freshers ko scenario-based questions mil sakte hain. But don't imagine that interviewer suddenly asks some huge real-world architecture problem.

For a fresher, scenario usually looks like:

“This particular thing is not working. What would you check?”

or

“Suppose you have to implement X. How would you do it?”

or

“What would happen if X?”

And ye 20 scenarios exactly isi type ki thinking prepare karte hain.

Express ke liye ab tumhare paas:

55 core questions + 20 scenario questions

Lekin **20 scenario ko 20 completely new topics mat samajhna**—ye core 55 ko real interview situations mein apply karna sikhate hain.

Meri recommendation: Express ko yahin STOP karo. Iske baad random “Express interview questions 100/200” lists karna tumhare liye preparation nahi, overload hoga. Recent junior/fresher material bhi fundamentals around routing, middleware, request/response, error handling, REST, authentication and security par cluster karta hai. ([Toolmingo](#))

Ab chlo mongodb interview questions dedo for freshers phele theory do then queries wale krenge answers 3 4 lines ke hi dena

Bilkul 👍 MongoDB ko bhi **overload nahi karenge**. Pehle sirf **fresher-level theory**, phir separate series mein **MongoDB queries/practical questions** karenge.

Mere hisaab se theory ke liye **35 genuinely important questions** enough hain. Answers **3–4 lines**, simple interview English mein rahenge, aur jahan useful hoga wahan follow-up bhi denge.

MongoDB Interview Questions — Theory

Q1. What is MongoDB? ●

MongoDB is a NoSQL, document-oriented database that stores data in flexible JSON-like documents. It stores data in collections instead of traditional tables. It is commonly used with Node.js and Express in MERN applications.

Follow-up: Why is MongoDB called a NoSQL database?

> Because it does not use the traditional relational table-based structure and does not require fixed schemas like SQL databases.

Q2. Why do we use MongoDB? ●

MongoDB provides a flexible document structure, which makes it easy to work with changing application data. It also supports horizontal scaling and works well with JavaScript-based applications. It is commonly used for modern web applications.

Q3. What is a document in MongoDB? ●

A document is a single record in MongoDB, stored in a BSON format. It is similar to a JSON object and contains field-value pairs. For example, one document can represent one user or one product.

Q4. What is a collection in MongoDB? ●

A collection is a group of related documents. It is roughly similar to a table in a relational database, but its documents do not have to follow exactly the same structure. For example, a users collection can contain user documents.

Q5. MongoDB vs SQL databases? ● 🔥

SQL databases store data mainly in tables with rows and columns, while MongoDB stores data as documents inside collections. SQL databases usually have a predefined schema, whereas MongoDB provides a more flexible schema. MongoDB is also designed around document-based data modeling and can scale horizontally.

Q6. What is BSON? ●

BSON stands for Binary JSON and is the format MongoDB uses internally to store documents. It supports JSON-like data along with additional data types such as dates and ObjectIds. MongoDB converts documents to BSON for storage and processing.

Follow-up: Is BSON the same as JSON?

> No. BSON is a binary representation with additional data types, while JSON is a text-based data format.

Q7. What is `_id` in MongoDB? ● 🔥

`_id` is a unique identifier for every MongoDB document. If we don't provide an `_id`, MongoDB automatically generates one, usually as an ObjectId. It helps uniquely identify and retrieve a document.

Q8. What is ObjectId? ●

ObjectId is a commonly used MongoDB identifier type for the `_id` field. It is designed to be unique and contains information that helps

distinguish generated IDs. It is commonly seen as a 24-character hexadecimal string when represented as text.

Q9. What does schema-less mean in MongoDB? 📌

Schema-less means documents in the same collection do not necessarily have to contain exactly the same fields or structure. This provides flexibility when application requirements change. However, applications can still enforce validation rules when needed.

Follow-up: Does MongoDB have no schema at all?

MongoDB has flexible schemas by default, but schema validation can still be applied when required.

Q10. What is Mongoose? 📌🔥

Mongoose is an ODM library for Node.js and MongoDB. It allows us to define schemas and models and provides features such as validation, middleware, and convenient database operations. It makes working with MongoDB from Node.js easier.

Q11. MongoDB vs Mongoose? 📌

MongoDB is the actual database system, while Mongoose is a Node.js library used to interact with MongoDB. MongoDB stores and manages the data, whereas Mongoose provides schemas, models, validation, and other application-level features.

Q12. What is a Mongoose Schema? 📌🔥

A Mongoose Schema defines the structure and rules for documents used by a particular model. We can specify fields, data types, default values, validation rules, and other options. It helps maintain consistency in application data.

Q13. What is a Mongoose Model? 🔴🔥

A Mongoose Model is created from a schema and provides an interface for interacting with a MongoDB collection. We use models to create, read, update, and delete documents. For example, a User model can be used to work with user documents.

Q14. Schema vs Model in Mongoose? 🔴

A schema defines how the data should look and what rules it should follow. A model is created from that schema and is used to interact with the corresponding MongoDB collection. In simple terms, the schema defines the structure, while the model works with the data.

Q15. What is CRUD in MongoDB? 🔴🔥

CRUD stands for Create, Read, Update, and Delete. These are the four basic operations used to manage data in a database. In MongoDB, these operations are performed on documents within collections.

Q16. What is an index in MongoDB? 🔴🔥

An index is a data structure that helps MongoDB find documents more efficiently. Without a suitable index, MongoDB may need to scan many documents to find matching data. Indexes can improve read performance but also require additional storage and can affect write performance.

Q17. Why can too many indexes be a problem? 🟡

Indexes improve reads, but they also consume storage and need to be updated when documents are inserted or modified. Having unnecessary indexes can therefore increase write overhead. I would create indexes based on actual query patterns.

Q18. What is a unique index? ●

A unique index ensures that a particular field cannot contain duplicate values. For example, we can create a unique index on an email field when every user must have a different email address. MongoDB rejects conflicting duplicate values.

Q19. What is MongoDB aggregation? ●🔥

Aggregation is used to process and transform multiple documents and produce calculated results. It works through stages such as \$match, \$group, \$sort, and \$project. It is useful for tasks such as reports, totals, filtering, and data analysis.

Q20. What is an aggregation pipeline? ●

An aggregation pipeline is a sequence of stages through which MongoDB processes documents. Each stage performs an operation and passes its result to the next stage. This allows complex data processing to be broken into smaller steps.

Q21. What is \$lookup in MongoDB? ●

\$lookup is an aggregation stage used to combine documents from two collections. It is similar to a join in SQL. For example, it can be used to combine orders with the corresponding user information.

Q22. What is embedding in MongoDB? ●

Embedding means storing related data directly inside another document. It can be useful when the related data is usually accessed together and is not too large. For example, an address can sometimes be embedded inside a user document.

Q23. What is referencing in MongoDB? ●

Referencing means storing a reference, usually an ObjectId, to a document in another collection. It is useful when related data is large, shared, or frequently managed separately. For example, an order can reference a separate user document.

Q24. Embedding vs referencing? ● 🔥

Embedding stores related data inside the same document, while referencing stores the related data separately and connects it using an identifier. Embedding can be faster when related data is always needed together. Referencing is better when the related data is large, shared, or independently maintained.

Q25. What is normalization and denormalization in MongoDB? ●

Normalization keeps related data separated to reduce duplication, while denormalization stores some related data together to improve read performance. MongoDB often uses a combination of both approaches depending on how the application accesses the data.

Q26. What is `populate()` in Mongoose? ● 🔥

`populate()` is a Mongoose feature used to replace a referenced ObjectId with the related document's data. It makes working with referenced documents easier at the application level. For example, an order's `userId` can be populated with the corresponding user information.

Q27. What is validation in MongoDB/Mongoose? ●

Validation ensures that data meets certain rules before it is stored. With Mongoose, we can define rules such as required fields, data types, minimum or maximum values, and custom validation. This helps maintain consistent application data.

Q28. What are transactions in MongoDB? ●

A transaction allows multiple database operations to be treated as one logical unit. If an operation fails, the transaction can be rolled back so partial changes are not left behind. Transactions are useful when multiple related updates must succeed together.

Q29. What is atomicity in MongoDB? ●

Atomicity means an operation is completed completely or not applied at all. MongoDB provides atomicity for operations on a single document, and transactions can be used when atomic behavior is needed across multiple documents or collections.

Q30. What is replication in MongoDB? ●

Replication means maintaining multiple copies of data across MongoDB servers. MongoDB uses replica sets to provide redundancy and high availability. If the primary server fails, another eligible member can become the primary.

Q31. What is sharding in MongoDB? ●

Sharding is a technique used to distribute data across multiple servers. It helps MongoDB handle very large datasets and high workloads by spreading data and operations across different machines. It is mainly relevant to larger-scale applications.

Q32. Replication vs sharding? ●

Replication creates multiple copies of data mainly for high availability and redundancy. Sharding distributes different portions of data across servers mainly for scalability. In large systems, both can be used together.

Q33. What is MongoDB Atlas? ●

MongoDB Atlas is a cloud-based service for hosting and managing MongoDB databases. It handles infrastructure-related tasks such as deployment, monitoring, backups, and scaling options. It is commonly used to host MongoDB databases for web applications.

Q34. How do you secure a MongoDB database? ●🔥

I would use authentication, proper user permissions, secure network access, and environment variables for database credentials. I would also use encrypted connections and avoid exposing the database publicly without appropriate access controls.

Q35. What happens if MongoDB connection is lost? ●

The application may fail to perform database operations until the connection is restored. I would handle connection errors properly, log the issue, and use the MongoDB driver's or Mongoose's connection-management features to recover when possible.

🎯 Priority

● Must do — 27

Q1–16, Q18–24, Q26–27, Q33–35

● Do after that — 8

Q17, Q25, Q28–32

Sabse important MongoDB topics:

MongoDB basics → Document/Collection → BSON → `_id/ObjectId` → MongoDB vs SQL → Mongoose → Schema/Model → CRUD → Indexes → Aggregation → `$lookup` → Embedding/Referencing → `populate()` → Validation → Transactions → Security.

Ab **queries wali series alag** rakhenge—usmein CRUD queries, operators, aggregation, find(), update, delete, sorting, pagination, indexes, Mongoose queries etc. karenge.

Now all most important queries interview questions of mongodb

Bilkul. Ab **sirf MongoDB query/practical interview questions** karte hain. Theory repeat nahi karunga. Freshers ke liye genuinely important MongoDB + Mongoose queries ko cover karenge, including **CRUD, operators, sorting, pagination, aggregation, lookup, populate, indexes**.

Main isko unnecessarily 100+ nahi banaunga. ~45 **questions** mein query side properly cover ho jayegi.

MongoDB Query Interview Questions

1. How do you insert one document?

```
db.users.insertOne({
  name: "Pranjal",
  age: 21,
  email: "pranjal@example.com"
})
```

Follow-up: Multiple documents?

```
db.users.insertMany([
  { name: "Aman", age: 22 },
  { name: "Riya", age: 20 }
])
```

2. How do you find all documents?

```
db.users.find()
```

Follow-up: Find one document?

```
db.users.findOne({ name: "Pranjal" })
```

● 3. How do you find a user by a specific field?

```
db.users.find({ age: 21 })
```

Same question: Find all users whose age is 21.

● 4. How do you find documents using multiple conditions?

```
db.users.find({  
  age: 21,  
  city: "Roorkee"  
})
```

This performs an **AND** condition.

● 5. How do you use \$or?

```
db.users.find({  
  $or: [  
    { age: 20 },  
    { city: "Delhi" }  
  ]  
})
```

It returns documents matching **either condition**.

● 6. How do you find users whose age is greater than 20?

```
db.users.find({  
  age: { $gt: 20 }  
})
```

Important comparison operators:

```
$gt  greater than  
$gte greater than or equal  
$lt  less than  
$lte less than or equal  
$ne  not equal
```

7. How do you find users between two ages?

```
db.users.find({  
  age: { $gte: 18, $lte: 25 }  
})
```

Same question: Find users whose age is between 18 and 25.

8. How do you find documents where a field is not equal to a value?

```
db.users.find({  
  city: { $ne: "Delhi" }  
})
```

9. How do you find documents where a field exists?

```
db.users.find({  
  phone: { $exists: true }  
})
```

Without the field:

```
db.users.find({  
  phone: { $exists: false }  
})
```

10. How do you find users whose age is in a given list?

```
db.users.find({  
  age: { $in: [18, 21, 25] }  
})
```

Follow-up: Opposite?

```
db.users.find({  
  age: { $nin: [18, 21, 25] }  
})
```

UPDATE QUERIES

11. How do you update one document?

```
db.users.updateOne(  
  { name: "Pranjal" },  
  { $set: { age: 22 } }  
)
```

12. How do you update multiple documents?

```
db.users.updateMany(  
  { city: "Delhi" },  
  { $set: { country: "India" } }  
)
```

13. What is \$set?

```
db.users.updateOne(  
  { name: "Pranjal" },  
  { $set: { age: 22 } }  
)
```

\$set updates a field without replacing the entire document.

14. How do you increment a numeric field?

```
db.products.updateOne(  
  { name: "Laptop" },  
  { $inc: { stock: 5 } }  
)
```

Same: Increase/decrease a value.

15. How do you remove a field from a document?

```
db.users.updateOne(  
  { name: "Pranjal" },
```

```
{ $unset: { phone: "" } }  
)
```

16. How do you rename a field?

```
db.users.updateMany(  
  {},  
  { $rename: { "name": "fullName" } }  
)
```

17. How do you add an item to an array?

```
db.users.updateOne(  
  { name: "Pranjal" },  
  { $push: { skills: "React" } }  
)
```

18. How do you add multiple items to an array?

```
db.users.updateOne(  
  { name: "Pranjal" },  
  {  
    $push: {  
      skills: {  
        $each: ["Node", "MongoDB"]  
      }  
    }  
  }  
)
```

19. How do you remove an item from an array?

```
db.users.updateOne(  
  { name: "Pranjal" },  
  { $pull: { skills: "React" } }  
)
```



DELETE

20. How do you delete one document?

```
db.users.deleteOne({  
  name: "Pranjal"  
})
```

21. How do you delete multiple documents?

```
db.users.deleteMany({  
  age: { $lt: 18 }  
})
```



PROJECTION / SORT / LIMIT

22. How do you select only specific fields?

```
db.users.find(  
  {},  
  { name: 1, email: 1 }  
)
```

This is called **projection**.

23. How do you exclude a field?

```
db.users.find(  
  {},  
  { password: 0 }  
)
```

Very common interview example: **don't return passwords**.

24. How do you sort documents?

Ascending:

```
db.users.find().sort({ age: 1 })
```

Descending:

```
db.users.find().sort({ age: -1 })
```

1 = ascending, -1 = descending.

25. How do you limit the number of results?

```
db.users.find().limit(10)
```

Returns maximum 10 documents.

26. How do you skip documents?

```
db.users.find().skip(10)
```

Useful for pagination.

27. How would you implement pagination in MongoDB?

```
db.users.find()  
  .skip((page - 1) * limit)  
  .limit(limit)
```

For example, page 2 with 10 items:

```
.skip(10)  
.limit(10)
```

MONGOOSE QUERIES

28. How do you find all users using Mongoose?

```
const users = await User.find();
```

● 29. How do you find one user using Mongoose?

```
const user = await User.findOne({  
  email: "test@gmail.com"  
});
```

● 30. How do you find a document by ID?

```
const user = await User.findById(userId);
```

Same question: Find user using MongoDB _id.

● 31. How do you create a document using Mongoose?

```
const user = await User.create({  
  name: "Pranjal",  
  email: "test@gmail.com"  
});
```

● 32. How do you update a document using Mongoose?

```
const user = await User.findByIdAndUpdate(  
  userId,  
  { age: 22 },  
  { new: true }  
);
```

Follow-up: Why new: true?

It returns the updated document instead of the old document.

● 33. How do you delete a document using Mongoose?

```
await User.findByIdAndDelete(userId);
```

● 34. How do you select specific fields in Mongoose?

```
const users = await User.find()  
  .select("name email");
```

Exclude:

```
.select("-password")
```

● 35. How do you sort using Mongoose?

```
const users = await User.find()  
  .sort({ createdAt: -1 });
```

🔥 AGGREGATION

● 36. What is an aggregation query? Give an example.

```
db.orders.aggregate([  
  {  
    $group: {  
      _id: "$status",  
      total: { $sum: 1 }  
    }  
  }  
])
```

This groups orders by status and counts them.

● 37. How do you filter documents using \$match?

```
db.orders.aggregate([  
  {  
    $match: {  
      status: "completed"  
    }  
  }  
])
```

Interview: What is \$match?

It filters documents that match a condition.

38. How do you group documents using \$group?

```
db.orders.aggregate([
  {
    $group: {
      _id: "$userId",
      totalOrders: { $sum: 1 }
    }
  }
])
```

39. How do you calculate the total amount?

```
db.orders.aggregate([
  {
    $group: {
      _id: null,
      total: { $sum: "$amount" }
    }
  }
])
```

40. How do you calculate the average value?

```
db.products.aggregate([
  {
    $group: {
      _id: null,
      averagePrice: { $avg: "$price" }
    }
  }
])
```

🔴 41. How do you sort aggregation results?

```
db.products.aggregate([
  { $sort: { price: -1 } }
])
```

🔴 42. How do you limit aggregation results?

```
db.products.aggregate([
  { $sort: { price: -1 } },
  { $limit: 5 }
])
```

Returns the top 5 expensive products.

🔥 LOOKUP / POPULATE

🔴 43. How do you join two collections in MongoDB?

Using \$lookup:

```
db.orders.aggregate([
  {
    $lookup: {
      from: "users",
      localField: "userId",
      foreignField: "_id",
      as: "user"
    }
  }
])
```

🔴 44. How do you populate a referenced document in Mongoose?

```
const orders = await Order.find()
  .populate("userId");
```

Follow-up: \$lookup vs populate()?

\$lookup is a MongoDB aggregation operation, while populate() is a Mongoose feature that resolves referenced documents.

INDEX

45. How do you create an index?

```
db.users.createIndex({  
  email: 1  
})
```

For unique email:

```
db.users.createIndex(  
  { email: 1 },  
  { unique: true }  
)
```

Follow-up: Why create an index?

It can make queries faster by allowing MongoDB to find matching documents more efficiently.

FINAL PRIORITY

Must Practice

1–16, 20–16? Correction: 1–16, 20–27, 28–39, 41–45

इनमें especially:

- find() / findOne()
- insertOne() / insertMany()
- updateOne() / updateMany()
- \$set, \$inc, \$push, \$pull, \$unset
- \$gt, \$gte, \$lt, \$lte, \$in, \$nin
- \$or
- Projection
- sort()

- limit() / skip()
- Pagination
- Mongoose CRUD
- select()
- Aggregation
- \$match
- \$group
- \$sum / \$avg
- \$lookup
- populate()
- Indexes

Queries ke liye ye set genuinely enough hai for a fresher MERN interview. Ab iske baad random operators ki 100-query list karne ki zarurat nahi hai.

Ok sql queries interview questions dedo ab

Bilkul. SQL ke liye bhi **random 100 queries nahi** karenge. Fresher MERN interview ke perspective se jo queries genuinely important hain, unhi ko cover karenge.

Main **40 SQL query interview questions** ko enough maanta hoon. Isme basic CRUD se lekar **JOIN, GROUP BY, HAVING, subqueries, aggregate functions, window functions aur common interview scenarios** cover ho jayenge.

SQL Query Interview Questions — Fresher

Assume tables:

employees

id
 name
 department_id
 salary
 city
 email

manager_id

departments

id

department_name

1. How do you fetch all employees?

```
SELECT * FROM employees;
```

2. How do you fetch only specific columns?

```
SELECT name, salary  
FROM employees;
```

Same: How do you select particular fields from a table?

3. How do you find employees from a specific city?

```
SELECT *  
FROM employees  
WHERE city = 'Delhi';
```

4. How do you find employees whose salary is greater than 50,000?

```
SELECT *  
FROM employees  
WHERE salary > 50000;
```

● 5. How do you find employees whose salary is between 30,000 and 60,000?

```
SELECT *  
FROM employees  
WHERE salary BETWEEN 30000 AND 60000;
```

● 6. How do you find employees whose names start with 'A'?

```
SELECT *  
FROM employees  
WHERE name LIKE 'A%';
```

Follow-up: Names ending with A?

```
WHERE name LIKE '%A';
```

Containing A:

```
WHERE name LIKE '%A%';
```

● 7. How do you find employees from multiple cities?

```
SELECT *  
FROM employees  
WHERE city IN ('Delhi', 'Mumbai', 'Pune');
```

● 8. How do you sort employees by salary?

Highest first:

```
SELECT *  
FROM employees  
ORDER BY salary DESC;
```

Lowest first:

```
SELECT *  
FROM employees  
ORDER BY salary ASC;
```

● 9. How do you find the highest salary?

```
SELECT MAX(salary)
FROM employees;
```

● 10. How do you find the lowest salary?

```
SELECT MIN(salary)
FROM employees;
```

● 11. How do you find the average salary?

```
SELECT AVG(salary)
FROM employees;
```

● 12. How do you count the total number of employees?

```
SELECT COUNT(*)
FROM employees;
```

● 13. How do you count employees in each department?

```
SELECT department_id, COUNT(*) AS total
FROM employees
GROUP BY department_id;
```

Important: GROUP BY is used to create groups before applying aggregate functions.

● 14. How do you find the average salary of each department?

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id;
```

🔴 15. How do you find departments having more than 5 employees?

```
SELECT department_id, COUNT(*) AS total
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

★ Follow-up: WHERE vs HAVING?

WHERE filters individual rows before grouping, while HAVING filters groups after GROUP BY.

🔥 JOINS

🔴 16. What is an INNER JOIN? Write a query.

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.id;
```

It returns only employees whose department has a matching record.

🔴 17. What is a LEFT JOIN? Write a query.

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.id;
```

It returns all employees, even if their department doesn't have a matching record.

🔴 18. Find employees who don't belong to any department.

```
SELECT e.*
FROM employees e
```

```
LEFT JOIN departments d
ON e.department_id = d.id
WHERE d.id IS NULL;
```

🔥 **Very common interview pattern.**

🔴 19. INNER JOIN vs LEFT JOIN?

INNER JOIN returns only matching records from both tables. **LEFT JOIN** returns all records from the left table and matching records from the right table. If there is no match, the right-side columns contain NULL.

🔴 20. How do you join more than two tables?

```
SELECT e.name, d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.id;
```

You can continue adding additional JOIN clauses for other related tables.

🔥 SUBQUERIES

🔴 21. How do you find employees earning more than the average salary?

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

★ **Very important fresher interview question.**

● 22. How do you find the second-highest salary?

One common approach:

```
SELECT MAX(salary)
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

Follow-up: What if duplicate salaries exist?

Then I would use DISTINCT or a ranking/window-function approach depending on whether I need the second distinct salary or second row.

● 23. How do you find the third-highest salary?

Using DISTINCT:

```
SELECT salary
FROM (
    SELECT DISTINCT salary
    FROM employees
    ORDER BY salary DESC
    LIMIT 3
) x
ORDER BY salary
LIMIT 1;
```

Note: LIMIT syntax varies by database.

● 24. How do you find employees having the maximum salary?

```
SELECT *
FROM employees
WHERE salary = (
    SELECT MAX(salary)
```

```
FROM employees  
);
```

25. How do you find duplicate emails?

```
SELECT email, COUNT(*) AS total  
FROM employees  
GROUP BY email  
HAVING COUNT(*) > 1;
```

 Very common practical interview question.

NULL / CONDITIONAL

26. How do you find employees whose manager is NULL?

```
SELECT *  
FROM employees  
WHERE manager_id IS NULL;
```

Follow-up: Why not `manager_id = NULL`?

Because NULL represents a missing or unknown value, so we use IS NULL or IS NOT NULL to check it.

27. How do you replace NULL values in a query?

```
SELECT name, COALESCE(city, 'Unknown') AS city  
FROM employees;
```

`COALESCE()` returns the first non-NULL value.

28. How do you use a CASE statement?

```
SELECT name,  
CASE  
  WHEN salary >= 80000 THEN 'High'  
  WHEN salary >= 50000 THEN 'Medium'  
  ELSE 'Low'
```

```
END AS salary_level  
FROM employees;
```

Useful for conditional output.

INSERT / UPDATE / DELETE

29. How do you insert a new employee?

```
INSERT INTO employees  
(name, department_id, salary, city)  
VALUES  
('Rahul', 2, 50000, 'Delhi');
```

30. How do you update an employee's salary?

```
UPDATE employees  
SET salary = 60000  
WHERE id = 101;
```

 **Follow-up:** What happens if you forget WHERE?

The update can affect every row in the table, so I always verify the condition before running an UPDATE.

31. How do you delete an employee?

```
DELETE FROM employees  
WHERE id = 101;
```

32. How do you delete duplicate records?

One common pattern is using ROW_NUMBER():

```
WITH duplicates AS (  
  SELECT id,  
         ROW_NUMBER() OVER (  
           PARTITION BY email
```



```
        ORDER BY id
    ) AS rn
FROM employees
)
DELETE FROM employees
WHERE id IN (
    SELECT id
    FROM duplicates
    WHERE rn > 1
);
```

Exact syntax can vary between database systems.



ADVANCED BUT FRESHER-RELEVANT



33. How do you find the top 3 highest-paid employees?

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

Note: SQL Server commonly uses TOP 3 instead of LIMIT.



34. How do you find the highest-paid employee in each department?

Using a window function:

```
SELECT *
FROM (
    SELECT e.*,
        RANK() OVER (
            PARTITION BY department_id
            ORDER BY salary DESC
        ) AS rnk
    FROM employees e
) x
WHERE rnk = 1;
```

🔥 Good interview-level query.

🟡 35. How do you rank employees based on salary?

```
SELECT name, salary,  
       RANK() OVER (ORDER BY salary DESC) AS salary_rank  
FROM employees;
```

🟡 36. RANK vs DENSE_RANK?

Both assign rankings based on values, but `RANK()` leaves gaps after ties, while `DENSE_RANK()` does not. For example, if two employees are ranked 1st, the next rank is 3 with `RANK()` but 2 with `DENSE_RANK()`.

🟡 37. How do you find employees earning more than their department's average salary?

```
SELECT *  
FROM employees e  
WHERE salary > (  
    SELECT AVG(e2.salary)  
    FROM employees e2  
    WHERE e2.department_id = e.department_id  
);
```

This is a good **correlated subquery** pattern.

🟡 38. How do you get the total salary paid by each department?

```
SELECT department_id,  
       SUM(salary) AS total_salary  
FROM employees  
GROUP BY department_id;
```

🟡 39. How do you find departments where average salary is greater than 60,000?

```
SELECT department_id,  
       AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department_id  
HAVING AVG(salary) > 60000;
```

🟡 40. How do you find employees whose salary is higher than their manager's salary?

```
SELECT e.name, e.salary  
FROM employees e  
JOIN employees m  
ON e.manager_id = m.id  
WHERE e.salary > m.salary;
```

🔥 This is a classic **self-join scenario** and worth knowing.

🎯 What you REALLY need to practice

🔴 Must-do

1–32

Especially:

SELECT → WHERE → LIKE → IN → BETWEEN → ORDER BY → aggregate functions →
GROUP BY → HAVING → **JOINS** → subqueries → second-highest salary → duplicates
→ NULL → CRUD.

🟡 After that

33–40

These cover the slightly harder interview scenarios:

Top N → window functions → ranking → correlated subqueries → self join.

So **40 SQL queries are enough for your fresher MERN interview preparation**. Anything beyond this should be done only if a particular company/job description specifically demands advanced SQL.

Now theory questions

Bilkul. Ab **SQL theory** karte hain — queries repeat nahi karunga. Fresher MERN interview ke liye sirf genuinely important concepts. **Answers 3–4 lines**, simple human English, interview mein directly bolne layak.

SQL Theory Interview Questions — Fresher

1. What is SQL? ●

SQL stands for Structured Query Language. It is used to store, retrieve, update, and manage data in relational databases. We can also use SQL to define database structures and control access to data.

Same question: What is SQL used for?

2. What is a database? ●

A database is an organized collection of data that can be stored, accessed, and managed efficiently. In a relational database, data is usually stored in tables containing rows and columns.

3. What is an RDBMS? ●

RDBMS stands for Relational Database Management System. It stores data in related tables and uses SQL to manage that data. MySQL, PostgreSQL, and SQL Server are common examples.

4. SQL vs NoSQL? ●🔥

SQL databases store structured data in tables with predefined schemas, while NoSQL databases can use flexible structures such as

documents. SQL commonly uses relationships and joins, whereas NoSQL databases often focus on flexible document-based or other non-relational models.

5. What is a table? 📌

A table is a structure used to store related data in a relational database. It consists of rows and columns, where each row represents a record and each column represents an attribute of that record.

6. What is a row and a column? 📌

A row represents one complete record in a table, while a column represents a particular attribute of that record. For example, in a users table, one row can represent one user and email can be a column.

7. What is a primary key? 📌 🔥

A primary key uniquely identifies each record in a table. It cannot contain NULL values and a table can have only one primary key, although that key can contain multiple columns.

8. What is a foreign key? 📌 🔥

A foreign key is a column that references a key in another table, usually its primary key. It is used to establish relationships between tables and maintain referential integrity.

9. Primary key vs foreign key? 📌

A primary key uniquely identifies records within its own table. A foreign key creates a relationship by referencing a key in another table and can contain duplicate values.

10. What is a candidate key? 🟡

A candidate key is a column or combination of columns that can uniquely identify a record. A table can have multiple candidate keys, but one of them is selected as the primary key.

11. What is a composite key? 🟡

A composite key is a key made up of two or more columns that together uniquely identify a record. It is useful when a single column cannot uniquely identify a row.

12. What is a UNIQUE constraint? 🔴

The UNIQUE constraint ensures that values in a column or combination of columns are not duplicated. Unlike a primary key, the exact handling of NULL values can depend on the database system.

13. Primary key vs UNIQUE key? 🔴

A primary key uniquely identifies each row and cannot be NULL. A UNIQUE constraint also prevents duplicate values, but a table can have multiple UNIQUE constraints and its NULL behavior depends on the database system.

14. What is a NOT NULL constraint? 🔴

NOT NULL ensures that a column must have a value when a row is inserted or updated. It is used for fields that are required, such as a user's name or email.

15. What is a DEFAULT constraint? 🟡

DEFAULT provides a value automatically when no value is supplied for a column during insertion. For example, a user's status could have a default value of 'active'.

SQL COMMANDS

16. What are DDL, DML, DQL, DCL and TCL?

DDL is used to define database structures, such as **CREATE** and **ALTER**. **DML** manages data, such as **INSERT**, **UPDATE**, and **DELETE**; **DQL** is commonly used for retrieving data with **SELECT**. **DCL** manages permissions, while **TCL** manages transactions such as **COMMIT** and **ROLLBACK**.

17. DELETE vs DROP vs TRUNCATE?

DELETE removes selected rows and can use a **WHERE** condition. **TRUNCATE** removes all rows from a table more directly, while **DROP** removes the table itself along with its structure. Their exact transactional behavior can vary by database system.

18. WHERE vs HAVING?

WHERE filters individual rows before grouping, while **HAVING** filters groups after **GROUP BY**. **WHERE** is generally used for row-level conditions, while **HAVING** is mainly used with aggregate results.

19. GROUP BY vs ORDER BY?

GROUP BY combines rows with the same values into groups, usually for aggregate calculations. **ORDER BY** only sorts the result based on one or more columns.

20. What are aggregate functions?

Aggregate functions perform calculations on multiple rows and return a result. Common examples are **COUNT()**, **SUM()**, **AVG()**, **MIN()**, and **MAX()**.

JOINS

21. What is a JOIN?

A JOIN combines related data from two or more tables using a matching condition. It allows us to retrieve information that is stored across different tables.

22. What are the different types of JOINS?

The commonly used joins are INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN. They differ mainly in whether they return only matching rows or also unmatched rows from one or both tables.

23. INNER JOIN vs LEFT JOIN?

INNER JOIN returns only rows that have a match in both tables. LEFT JOIN returns all rows from the left table and matching rows from the right table; unmatched right-side values become NULL.

24. What is a SELF JOIN?

A self join is when a table is joined with itself. It is useful when records in the same table are related, such as employees and their managers.

NORMALIZATION

25. What is normalization?

Normalization is the process of organizing data to reduce unnecessary duplication and improve data consistency. It usually involves splitting data into related tables and defining proper relationships between them.

26. What is 1NF? 🟡

First Normal Form means each column should contain atomic values and there should not be repeating groups of data. In simple terms, one cell should represent one value rather than a list of values.

27. What is 2NF? 🟡

Second Normal Form means the table should already be in 1NF and every non-key attribute should depend on the entire primary key. It mainly addresses partial dependency when a composite key is used.

28. What is 3NF? 🟡

Third Normal Form means the table should be in 2NF and non-key attributes should not depend on other non-key attributes. It helps remove transitive dependencies and unnecessary duplication.

29. What is denormalization? 🟡

Denormalization intentionally introduces some data duplication to improve read performance or simplify certain queries. It can reduce the number of joins, but it also increases storage and can make updates more complex.

🔥 INDEX / PERFORMANCE

30. What is an index? 🔴🔥

An index is a data structure that helps the database find rows faster without scanning the entire table. It can significantly improve read performance, but indexes also consume storage and can add overhead to inserts and updates.

31. Why shouldn't we create indexes on every column? 📌

Indexes improve read performance but require additional storage and must be maintained when data changes. Too many indexes can therefore slow down INSERT, UPDATE, and DELETE operations.

32. What is a clustered index? 🟡

A clustered index determines how table data is physically organized in systems that support this concept. Because the data itself is organized around the clustered index, a table typically has only one clustered organization.

33. What is a non-clustered index? 🟡

A non-clustered index is a separate structure that stores indexed values along with references to the corresponding table rows. A table can generally have multiple non-clustered indexes.

🔥 TRANSACTIONS

34. What is a transaction? 📌 🔥

A transaction is a group of database operations treated as one logical unit. The goal is to ensure that related operations either complete successfully together or are rolled back when appropriate.

35. What are ACID properties? 📌 🔥

ACID stands for Atomicity, Consistency, Isolation, and Durability. These properties help ensure that transactions are completed reliably, maintain valid data, remain properly isolated from each other, and persist after a successful commit.

Follow-up:

- **Atomicity:** all or nothing

- **Consistency:** data remains valid
 - **Isolation:** transactions don't improperly interfere
 - **Durability:** committed data persists
-

36. COMMIT vs ROLLBACK?

COMMIT permanently saves the changes made by a transaction.
ROLLBACK undoes changes that have not yet been committed, depending on the database and transaction state.

IMPORTANT INTERVIEW CONCEPTS

37. What is a subquery?

A subquery is a query written inside another SQL query. It is useful when the result of one query is needed as a condition or input for another query, such as finding employees earning above the average salary.

38. What is a view?

A view is a virtual table based on a SQL query. It can simplify complex queries and control which data users can access, while the underlying data remains in the original tables.

39. What is a stored procedure?

A stored procedure is a pre-defined set of SQL statements stored in the database and executed when called. It can be useful for reusable database operations, although its usage depends on the application's architecture.

40. What is a trigger?

A trigger is database logic that automatically executes when certain events occur, such as INSERT, UPDATE, or DELETE. It can be used for tasks such as auditing or enforcing specific database actions.

41. What is referential integrity?

Referential integrity ensures that relationships between tables remain valid. For example, a foreign key should not reference a non-existing parent record when the database's constraints prohibit it.

42. What is a NULL value?

NULL represents a missing, unknown, or not-applicable value; it is not the same as zero or an empty string. Because NULL represents the absence of a value, we use IS NULL or IS NOT NULL to check it.

43. What is a constraint in SQL?

Constraints are rules applied to table columns to maintain data accuracy and integrity. Common constraints include PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT.

44. What is a CHECK constraint?

A CHECK constraint ensures that values satisfy a specified condition. For example, we can require that an employee's salary must be greater than zero.

45. What is a deadlock?

A deadlock occurs when two or more transactions wait for resources held by each other, so none of them can continue. Databases detect and resolve deadlocks using mechanisms such as aborting or rolling back one of the transactions.

Fresher Priority

Must know extremely well

1–5, 7–9, 12–19, 21–25, 30–31, 34–36, 41–43

Know after that

6, 10–11, 26–29, 32–33, 37–40, 44–45

Total = 45 theory + 40 query questions.

For a **fresher MERN interview**, this is a strong SQL set. You don't need to memorize every advanced database feature; you need to be able to **explain these naturally and write the common queries without getting stuck**.

Jo tumne mongodb ke upar question diye h unke answers 3 lines ke krdo sirf

Bilkul. MongoDB ke **35 theory questions** ke answers ko ab **sirf 3 lines**, simple human English aur interview-ready bana raha hoon. Questions same rahenge, unnecessary details hata raha hoon.

MongoDB Theory — 3-Line Answers

1. What is MongoDB?

MongoDB is a NoSQL, document-oriented database. It stores data in flexible JSON-like documents instead of tables. It is commonly used with Node.js in MERN applications.

2. Why do we use MongoDB?

MongoDB provides a flexible data structure and is easy to work with changing application requirements. It also supports scalability and works naturally with JavaScript applications.

3. What is a document in MongoDB? ●

A document is a single record in MongoDB. It stores data as field-value pairs in BSON format. For example, one document can represent one user.

4. What is a collection in MongoDB? ●

A collection is a group of related documents. It is similar to a table in SQL databases. For example, a users collection can store user documents.

5. MongoDB vs SQL databases? ●

SQL databases store data in tables, while MongoDB stores data in documents. SQL usually has a predefined schema, whereas MongoDB provides a more flexible schema.

6. What is BSON? ●

BSON stands for Binary JSON. MongoDB uses BSON internally to store documents. It also supports additional data types such as ObjectId and Date.

7. What is _id in MongoDB? ●

_id uniquely identifies each document in a collection. If we don't provide it, MongoDB automatically generates one. It is usually an ObjectId.

8. What is ObjectId? ●

ObjectId is a commonly used MongoDB data type for uniquely identifying documents. It is commonly used as the value of _id. When represented as text, it appears as a 24-character hexadecimal string.

9. What does schema-less mean in MongoDB? ●

It means documents in the same collection don't have to contain exactly the same fields. This gives MongoDB flexibility when application requirements change. Validation rules can still be added when needed.

10. What is Mongoose? ●

Mongoose is an ODM library for Node.js and MongoDB. It provides schemas, models, validation, and convenient database operations. It makes MongoDB easier to use from Node.js applications.

11. MongoDB vs Mongoose? ●

MongoDB is the actual database, while Mongoose is a Node.js library used to work with MongoDB. MongoDB stores the data, while Mongoose provides application-level features like schemas and validation.

12. What is a Mongoose Schema? ●

A Schema defines the structure and rules for documents. It can specify fields, data types, defaults, and validation rules. It helps maintain consistency in application data.

13. What is a Mongoose Model? ●

A Model is created from a Mongoose Schema. It provides methods to interact with a MongoDB collection. For example, a User model can create and find user documents.

14. Schema vs Model in Mongoose? ●

A Schema defines the structure and rules of the data. A Model is created from that Schema and is used to interact with the database. In simple terms, Schema defines and Model operates.

15. What is CRUD in MongoDB? ●

CRUD stands for Create, Read, Update, and Delete. These are the basic operations used to manage documents. MongoDB provides methods for performing each of these operations.

16. What is an index in MongoDB? ●

An index helps MongoDB find documents faster during queries. It improves read performance by avoiding unnecessary document scans. However, indexes also require storage and can add write overhead.

17. Why can too many indexes be a problem? ●

Indexes improve read performance but consume additional storage. They also need to be updated when data changes. Therefore, unnecessary indexes can slow down write operations.

18. What is a unique index? ●

A unique index prevents duplicate values in a field. For example, we can create a unique index on email so two users cannot have the same email.

19. What is MongoDB aggregation? ●

Aggregation is used to process and analyze multiple documents. It uses stages such as `$match`, `$group`, and `$sort`. It is useful for reports, calculations, and complex data processing.

20. What is an aggregation pipeline? ●

An aggregation pipeline is a sequence of stages that process documents step by step. Each stage performs an operation and passes its result to the next stage.

21. What is `$lookup` in MongoDB? ●

`$lookup` is an aggregation stage used to combine data from two collections. It works similarly to a JOIN in SQL. It is useful when related data is stored separately.

22. What is embedding in MongoDB? ●

Embedding means storing related data inside the same document. It is useful when the related data is usually accessed together. For example, an address can be embedded inside a user document.

23. What is referencing in MongoDB? ●

Referencing means storing the ID of a related document instead of storing the complete data. It is useful when related data is large or managed separately. For example, an order can reference a user by `userId`.

24. Embedding vs referencing? ●

Embedding stores related data inside the same document, while referencing stores it separately using an ID. Embedding is useful for

closely related data, while referencing is better for large or independently managed data.

25. What is normalization and denormalization in MongoDB? ●

Normalization separates related data to reduce duplication.

Denormalization stores some related data together to improve read performance. MongoDB can use either approach depending on the application's requirements.

26. What is populate() in Mongoose? ●

populate() is a Mongoose feature used with referenced documents. It replaces a referenced ObjectId with the related document's data. It makes working with relationships easier at the application level.

27. What is validation in MongoDB/Mongoose? ●

Validation checks whether data follows specific rules before it is stored. Mongoose supports rules such as required fields, data types, and custom validation. It helps maintain consistent data.

28. What are transactions in MongoDB? ●

A transaction groups multiple database operations into one logical unit. If something fails, the changes can be rolled back. They are useful when multiple operations must succeed together.

29. What is atomicity in MongoDB? ●

Atomicity means an operation happens completely or not at all. MongoDB provides atomicity for operations on a single document. Transactions can provide atomic behavior across multiple documents.

30. What is replication in MongoDB? ●

Replication keeps multiple copies of data on different MongoDB servers. MongoDB uses replica sets for high availability and redundancy. If the primary fails, another member can become primary.

31. What is sharding in MongoDB? ●

Sharding distributes data across multiple servers. It helps MongoDB handle very large datasets and high traffic. It is mainly useful for applications that need large-scale horizontal scaling.

32. Replication vs sharding? ●

Replication creates multiple copies of data mainly for availability and redundancy. Sharding distributes different portions of data across servers for scalability. Large systems can use both together.

33. What is MongoDB Atlas? ●

MongoDB Atlas is a cloud-based service for hosting and managing MongoDB databases. It provides features such as deployment, monitoring, backups, and scaling. It is commonly used to host databases for web applications.

34. How do you secure a MongoDB database? ●

I would use authentication, proper user permissions, secure network access, and environment variables for credentials. I would also use encrypted connections and avoid unnecessary public database access.

35. What happens if MongoDB connection is lost? ●

The application may not be able to perform database operations until the connection is restored. I would handle connection errors properly and log them. I would also use appropriate connection-management and recovery mechanisms.

Aur bhi do important questions iske projection vgera bhi to h

Haan, bilkul. **Projection** important hai, aur kuch aur MongoDB concepts bhi hain jo fresher MERN interview mein genuinely aa sakte hain. Pehle wale 35 ko repeat nahi kar raha.

MongoDB — Additional Important Theory Questions

36. What is Projection in MongoDB? ●

Projection means selecting only the required fields from a document. It helps reduce unnecessary data in the response. In MongoDB, we can use 1 to include and 0 to exclude fields.

37. What are MongoDB comparison operators? ●

Comparison operators are used to compare field values in queries. Common operators are \$gt, \$gte, \$lt, \$lte, \$eq, \$ne, \$in, and \$nin.

38. What are MongoDB logical operators? ●

Logical operators combine multiple conditions in a query. Common operators are \$and, \$or, \$not, and \$nor. They help us create more flexible queries.

39. What are array operators in MongoDB? ●

Array operators are used to query and modify array fields. Common examples include \$in, \$all, \$elemMatch, \$push, \$pull, and \$addToSet.

40. What is \$elemMatch? ●

\$elemMatch is used to match documents where an array contains an element satisfying multiple conditions. It is especially useful when working with arrays of objects.

41. What is \$addToSet? ●

\$addToSet adds a value to an array only if that value does not already exist. It is useful when we want to maintain unique values inside an array.

42. \$push VS \$addToSet? ●

\$push adds a value to an array even if it already exists. \$addToSet adds the value only when it is not already present. So \$addToSet is useful when duplicates should be avoided.

43. What is \$project in aggregation? ●

\$project is an aggregation stage used to select, exclude, or create fields in the output documents. It is commonly used to shape the result returned by an aggregation pipeline.

44. \$project vs projection? ●

Projection is the general concept of selecting specific fields from documents. **\$project** is specifically an aggregation pipeline stage used to shape documents during aggregation.

45. What is \$unwind? ●

\$unwind is an aggregation stage that separates array elements into individual documents. For example, an array containing three products can produce three separate documents in the pipeline.

46. What is \$sort in aggregation? ●

\$sort sorts documents based on one or more fields. We can sort in ascending order using 1 and descending order using -1. It is commonly combined with **\$limit** to get top results.

47. What is \$count in aggregation? ●

\$count counts the number of documents reaching that stage of the aggregation pipeline. It is useful when we need the total number of matching documents.

48. What is \$addFields? ●

\$addFields adds new fields or calculates fields in aggregation results. The existing fields are preserved unless explicitly modified.

49. What is the difference between find() and findOne()? ●

find() returns all documents that match the condition, while **findOne()** returns only one matching document. **find()** is useful when multiple results are expected, while **findOne()** is useful for a single record.

50. What is the difference between updateOne() and updateMany()? ●

updateOne() updates the first matching document. **updateMany()** updates all documents that match the given condition. We choose between them based on whether one or multiple records should change.

51. What is the difference between deleteOne() and deleteMany()? ●

deleteOne() removes one matching document, while **deleteMany()** removes all documents matching the condition. We should be

especially careful with `deleteMany()` because it can affect multiple records.

52. What is `findById()` in Mongoose? ●

`findById()` is a Mongoose method used to find a document using its `_id`. It is essentially a convenient way to query a document by its identifier.

53. What is `findByIdAndUpdate()`? ●

`findByIdAndUpdate()` finds a document using its `_id` and updates it. We can pass options such as `{ new: true }` to return the updated document.

54. What is `lean()` in Mongoose? ●

`lean()` makes Mongoose return plain JavaScript objects instead of full Mongoose documents. It can improve read performance when we don't need Mongoose document methods or features.

55. What is `exec()` in Mongoose? ●

`exec()` executes a Mongoose query and returns a Promise. It can make query execution explicit and is commonly used with `await` in asynchronous code.

56. What is `distinct()` in MongoDB? ●

`distinct()` returns unique values for a specified field. For example, it can be used to get all different cities from a users collection without duplicates.

57. What is `upsert` in MongoDB? ●

Upsert means update if a matching document exists, otherwise insert a new document. In MongoDB, we can enable it using the `upsert: true` option in an update operation.

58. What is `countDocuments()`? ●

`countDocuments()` returns the number of documents matching a specified filter. It is useful when we need the total number of records, especially for pagination.

59. What is a compound index? ●

A compound index is an index created using multiple fields. It is useful when queries commonly filter or sort using a combination of fields.

60. What is a text index? ●

A text index allows MongoDB to search text within string fields. It can be useful for basic text-search requirements such as searching product names or descriptions.

🎯 Ab MongoDB theory ka scope

60 theory questions + query questions ke baad fresher MERN ke liye MongoDB ka coverage kaafi strong ho gaya hai.

Sabse zyada focus karo:

Projection → Operators → Arrays → Aggregation (\$match, \$group, \$project, \$unwind, \$lookup) → Mongoose methods → populate() → Indexes → Upsert → Pagination.

Ye concepts **queries wali series mein bhi directly use honge**, isliye theory ratne ke bajay query practice ke saath automatically strong ho jayenge.